A Second Course in Algorithms

Mihir Rao

Contents

1 Course Goals and Introduction to Maximum Flow **4**
1.1 The maximum flow problem . . . 4
1.2 A naive greedy algorithm . . . 5
1.3 Residual networks . . . 5
1.4 The Ford–Fulkerson algorithm . . . 6
1.5 Termination . . . 7
1.6 Optimality conditions . . . 7

2 Augmenting Path Algorithms for Maximum Flow **8**
2.1 Cuts . . . 8
2.2 Optimality conditions and the max-flow/min-cut theorem . . . 8
2.3 The Edmonds–Karp algorithm . . . 10
2.4 Dinic's algorithm and blocking flows . . . 12

3 The Push-Relabel Algorithm for Maximum Flow **14**
3.1 Preflows and pushes . . . 14
3.2 Heights and invariants . . . 15
3.3 The algorithm . . . 16
3.4 Example . . . 16
3.5 Analysis . . . 17
3.6 Proof of the key lemma . . . 18

4 Applications of Maximum Flows and Minimum Cuts **20**
4.1 The minimum cut problem . . . 20
4.2 Image segmentation . . . 20
4.3 Bipartite matching . . . 22
4.4 Hall's theorem . . . 23

5 Minimum-Cost Bipartite Matching **25**
5.1 Optimality conditions . . . 25
5.2 Reduced costs and invariants . . . 26
5.3 The Hungarian algorithm . . . 27
5.4 Example . . . 30

6 Generalizations of Maximum Flow and Bipartite Matching **31**
6.1 The minimum-cost flow problem . . . 31
6.2 Maximum-cardinality non-bipartite matching . . . 32
6.3 Minimum-cost non-bipartite matching . . . 33

7 Linear Programming: Introduction and Applications **35**
7.1 Linear programs . . . 35
7.2 Geometric intuition . . . 35
7.3 Flow problems as linear programs . . . 37

7.4 Fitting a line . . . 37
7.5 Computing a linear classifier . . . 38
7.6 Non-separable data . . . 39

8 Linear Programming Duality (Part 1) 40
8.1 A warm-up example . . . 40
8.2 The dual linear program . . . 41
8.3 Max-flow/min-cut revisited . . . 42
8.4 A recipe for taking duals . . . 43
8.5 Weak duality . . . 44

9 Linear Programming Duality (Part 2) 45
9.1 Recap: primal, dual, and weak duality . . . 45
9.2 Complementary slackness . . . 46
9.3 The Hungarian algorithm revisited . . . 47
9.4 Strong LP duality . . . 48
9.5 The separating hyperplane theorem . . . 49
9.6 Farkas's lemma . . . 50

10 The Minimax Theorem and Algorithms for Linear Programming 51
10.1 Zero-sum games . . . 51
10.2 From LP duality to minimax . . . 52
10.3 The simplex method . . . 53
10.4 The ellipsoid method . . . 54
10.5 Interior-point methods . . . 56

11 Online Learning and the Multiplicative Weights Algorithm 57
11.1 Online algorithms . . . 57
11.2 Online decision-making . . . 57
11.3 The multiplicative weights algorithm . . . 58
11.4 Proof of the regret bound . . . 59
11.5 Minimax revisited . . . 61

12 Applications of Multiplicative Weights to Games and Linear Programs 62
12.1 Extensions of the multiplicative weights guarantee . . . 62
12.2 Minimax revisited (again) . . . 62
12.3 Linear classifiers revisited . . . 64
12.4 Maximum flow revisited . . . 66

13 Online Scheduling and Online Steiner Tree 68
13.1 Online makespan scheduling . . . 68
13.2 The online Steiner tree problem . . . 69
13.3 Analysis of the greedy algorithm . . . 71

14 Online Bipartite Matching 73
14.1 Deterministic algorithms . . . 73
14.2 Online fractional bipartite matching . . . 74
14.3 Analysis: a false start . . . 76
14.4 Analysis with a splitting function . . . 76
14.5 The optimal splitting function . . . 77

15 Introduction to Approximation Algorithms 79
15.1 Approximation algorithms . . . 79
15.2 Metric Steiner tree . . . 80
15.3 Set coverage . . . 81

15.4 Influence maximization . . . 83

16 The Traveling Salesman Problem **85**
16.1 Inapproximability of general TSP . . . 85
16.2 Metric TSP and the MST heuristic . . . 86
16.3 Christofides's algorithm . . . 87
16.4 Asymmetric TSP . . . 89

17 Linear Programming and Approximation Algorithms **92**
17.1 Greedy set cover without costs . . . 92
17.2 Greedy set cover with costs . . . 93
17.3 Interpretation via LP duality . . . 94
17.4 LP rounding for vertex cover . . . 95
17.5 A primal-dual algorithm for vertex cover . . . 96

18 Five Essential Tools for the Analysis of Randomized Algorithms **98**
18.1 Linearity of expectation and MAX 3SAT . . . 98
18.2 Tail inequalities, hashing, and Markov's inequality . . . 99
18.3 Chebyshev's inequality . . . 99
18.4 Chernoff bounds . . . 100
18.5 The union bound . . . 101
18.6 Randomized rounding for edge-disjoint paths . . . 102

19 Beating Brute-Force Search **104**
19.1 Vertex cover and fixed-parameter tractability . . . 104
19.2 TSP and dynamic programming . . . 105
19.3 Schöning's random search for 3SAT . . . 106

20 The Maximum Cut Problem and Semidefinite Programming **109**
20.1 A semidefinite relaxation . . . 109
20.2 Randomized hyperplane rounding . . . 111

Lecture 1: Course Goals and Introduction to Maximum Flow

The course has three goals.

1. Learn efficient algorithms for fundamental, *well-solved* problems: problems flexible enough to model many applications that can still be solved quickly and exactly, in theory and in practice. The first half covers the maximum flow problem, the minimum cut problem, graph matching problems, and linear programming.
2. Learn tools for *not-so-well-solved* problems: NP-hard problems, which are not expected to have exact polynomial-time algorithms, and online problems, where an algorithm must make irrevocable decisions without knowing the whole input. The focus is on algorithms guaranteed to output a solution reasonably close to an optimal one.
3. Provide a gateway to the study of advanced algorithms.

1.1 The maximum flow problem

Definition 1.1 [Maximum flow]. An instance consists of a directed graph $G = (V, E)$, a source $s \in V$, a sink $t \in V$, and a nonnegative integral capacity u_e for each edge $e \in E$. A flow is a nonnegative vector $\{f_e\}_{e \in E}$ satisfying

- *capacity constraints*: $f_e \leq u_e$ for every $e \in E$;
- *conservation constraints*: for every vertex $v \neq s, t$, the total flow $\sum f_e$ on edges entering v equals the total flow on edges leaving v.

The value of a flow is the total flow leaving s (equivalently, as shown in Lecture 2, the total flow entering t). The goal is a maximum flow, one of maximum value.

Since the point is to push flow from s to t, assume without loss of generality that s has no incoming edges and t has no outgoing edges. Each capacity is the maximum amount of "stuff" an edge can carry. A flow can be pictured as a superposition of paths, but algorithms track only the amount f_e on each edge. In the left network of Figure 1, the flow sends 2 units on $s \to v \to t$, 1 on $s \to v \to w \to t$, and 2 on $s \to w \to t$. Its value 5 is maximum, since at most $3 + 2 = 5$ units can leave s.

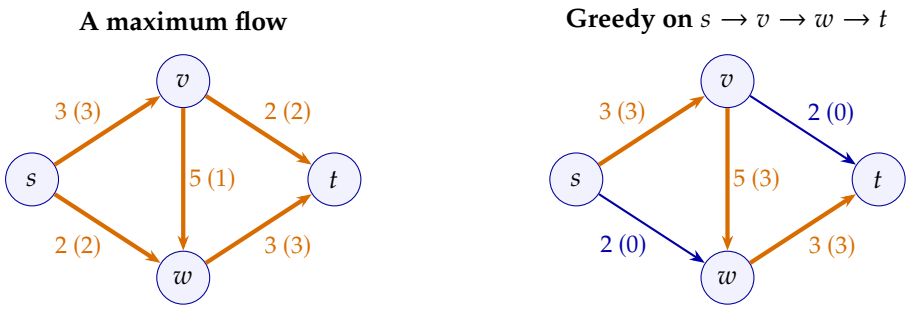


Figure 1: The running example. Each edge is labeled u_e (f_e), and flow-carrying edges are thick. Left: a maximum flow, of value 5. Right: the naive greedy algorithm, choosing the zig-zag path first, gets stuck at value 3.

Maximum flow directly models routing traffic through a transportation network, packets through a data network, or oil through a distribution network, and many less obvious problems reduce to it.

1.2 A naive greedy algorithm

A priori it is not clear that maximum flow has an efficient algorithm. A first try is greedy: start from the zero flow and repeatedly send more flow along some s–t path.

Algorithm 1.1: Naive greedy algorithm

Input: Network $G = (V, E)$, s, t, capacities u.
Output: A flow f.

```
f_e ← 0 for all e ∈ E
while some s–t path P has f_e < u_e for every e ∈ P do
    Δ ← min_{e∈P}(u_e − f_e)                    // room on P
    for every edge e of P do f_e ← f_e + Δ
return f
```

Finding P is a reachability question in the subgraph of edges with $f_e < u_e$, answered in $O(m)$ time by BFS or DFS. If several paths exist, one is chosen arbitrarily. The greedy algorithm can fail. On the running example, if it first picks the zig-zag path $s \to v \to w \to t$, then $\Delta = \min\{3, 5, 3\} = 3$. This saturates (s, v) and (w, t), after which every s–t path contains a full edge, so the algorithm halts with value 3 instead of 5 (Figure 1, right).

1.3 Residual networks

From the stuck flow, the maximum flow is reached by sending 2 more units along (s, w), *backward* along (v, w), undoing 2 of the 3 units sent there, and then along (v, t). Residual networks formalize these "undo" operations.

Definition 1.2 [Residual network]. Given a flow f in G, the residual network G_f has vertex set V

and two edges for each edge $e = (v, w)$ of G:

- a FORWARD EDGE (v, w) with RESIDUAL CAPACITY $u_e - f_e$, the room remaining on e;
- a REVERSE EDGE (w, v) with residual capacity f_e, the flow on e that can be undone.

Unless stated otherwise, edges with zero residual capacity are omitted.

If G already has both (v, w) and (w, v), then G_f has parallel edges, which causes no problems. The paths searched for by the greedy algorithm are exactly the s–t paths of G_f that use only forward edges.

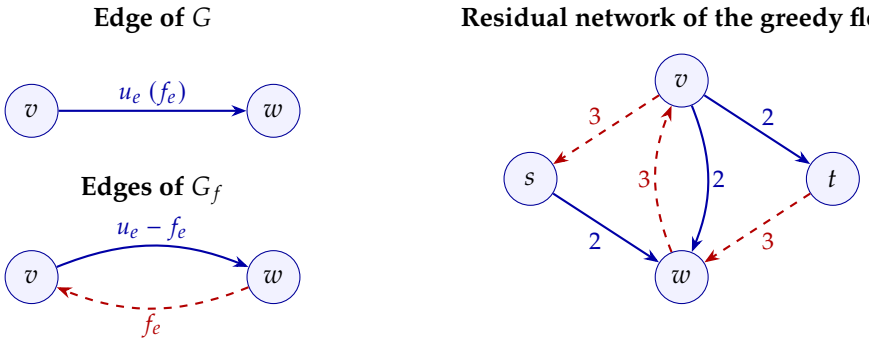


Figure 2: Left: an edge of G with capacity u_e and flow f_e yields a forward edge (solid) and a reverse edge (dashed) in G_f. Right: the residual network of the greedy flow of Figure 1, with four zero-capacity edges omitted. It contains the path $s \to w \to v \to t$.

1.4 The Ford–Fulkerson algorithm

Running the greedy algorithm in the current residual network gives a correct algorithm.

Algorithm 1.2: Ford–Fulkerson algorithm

Input: Network $G = (V, E)$, s, t, capacities u.
Output: A maximum flow f.

$f_e \leftarrow 0$ for all $e \in E$
while G_f has an s–t path P (of positive residual capacities) **do**
  $\Delta \leftarrow$ minimum residual capacity of an edge of P
  // augment f along P
  for each $e \in E$ whose forward edge is in P **do** $f_e \leftarrow f_e + \Delta$
  for each $e \in E$ whose reverse edge is in P **do** $f_e \leftarrow f_e - \Delta$
return f

Each search takes $O(m)$ time by BFS or DFS. In the residual network of Figure 2, the path $s \to w \to v \to t$ has residual capacities $2, 3, 2$, so $\Delta = 2$; augmenting gives the maximum flow of Figure 1.

1.5 Termination

Proposition 1.3 [Ford–Fulkerson terminates]. *With integral capacities, the Ford–Fulkerson algorithm halts, and it halts with a flow.*

Proof. Two invariants hold by induction on the number of iterations. First, f is a flow. The choice of Δ keeps every f_e between 0 and u_e. At a vertex $v \neq s, t$ on P, the two edges of P at v are forward or reverse in four combinations, and in each, flow in and flow out of v change equally: two forward edges raise both by Δ, two reverse edges lower both by Δ, and in the mixed cases two edges entering (or two leaving) v change by $+\Delta$ and $-\Delta$. Second, f is integral, since all residual capacities, and hence Δ, are integers.
So $\Delta \geq 1$, and each iteration raises the value by Δ. The value is at most the total capacity of the edges leaving s, so the algorithm halts, and by the first invariant it halts with a flow. □

Rational capacities reduce to integral ones by clearing denominators; with irrational capacities the algorithm need not terminate.

1.6 Optimality conditions

A recurring question in the course is: *how do we know when we are done?* Showing that a flow is *not* maximum is easy: exhibit a flow of higher value. Certifying that a flow *is* maximum is harder, which motivates a paradigm.

1. Identify optimality conditions: sufficient conditions for a feasible solution to be optimal. This step is structural, not necessarily algorithmic.
2. Design an algorithm that terminates with the optimality conditions satisfied. Such an algorithm is necessarily correct.

For maximum flow the first step is the following claim, proved in Lecture 2 as part of a stronger theorem.

Theorem 1.4 [Optimality conditions for maximum flow]. *If f is a flow in G such that the residual network G_f has no s–t path, then f is a maximum flow.*

The Ford–Fulkerson algorithm halts only when this condition holds, so it completes the second step: it terminates with a maximum flow.

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 1: Course Goals and Introduction to Maximum Flow.

Lecture 2: Augmenting Path Algorithms for Maximum Flow

Throughout, $G = (V, E)$ is a directed graph with source s, sink t, and integral capacities u_e, with $n = |V|$ and $m = |E|$. A flow satisfies capacity and conservation constraints, and G_f denotes the residual network of a flow f, keeping only edges of positive residual capacity. The Ford–Fulkerson algorithm augments along s–t paths of G_f until none remains. This lecture proves that it then has a maximum flow, and makes the choice of paths efficient.

2.1 Cuts

Definition 2.1 [(s,t)-cut]. An (s,t)-cut of G is a partition of V into sets A, B with $s \in A$ and $t \in B$. Let $\delta^+(A)$ and $\delta^-(A)$ be the sets of edges sticking out of A (tail in A, head in B) and into A (tail in B, head in A). The capacity of the cut is

$$\sum_{e \in \delta^+(A)} u_e .$$

A minimum cut is one of minimum capacity.

A cut sorts the edges into four classes: inside A, inside B, out of A, and into A. Only edges out of A count toward the capacity. In the network of Figure 3, the cut $(\{s, w\}, \{v, t\})$ has capacity 3 (three outgoing edges of capacity 1), while $(\{s\}, \{v, w, t\})$ has capacity 101.

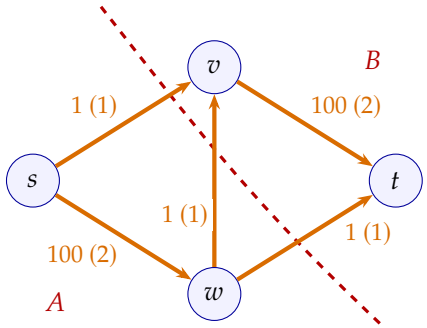


Figure 3: A network with each edge labeled u_e (f_e). The flow has value 3, and the dashed cut $(A, B) = (\{s, w\}, \{v, t\})$ has capacity 3, which certifies that the flow is maximum.

2.2 Optimality conditions and the max-flow/min-cut theorem

Theorem 2.2 [Optimality conditions for maximum flow]. *For a flow f in G, the following are equivalent:*

(1) f is a maximum flow of G;

(2) there is an (s,t)-cut (A, B) whose capacity equals the value of f;

(3) the residual network G_f has no s–t path.

The proof shows (2)⇒(1)⇒(3)⇒(2). The first step rests on a lemma.

Lemma 2.3 [Flow across a cut]. *For every flow f and every (s,t)-cut (A,B),*

$$\textit{value of } f = \sum_{e\in\delta^+(A)} f_e - \sum_{e\in\delta^-(A)} f_e \le \textit{capacity of } (A,B).$$

Proof. Since s has no incoming edges, the value of f is $\sum_{e\in\delta^+(s)} f_e - \sum_{e\in\delta^-(s)} f_e$. For each $v \in A \setminus \{s\}$, conservation gives $\sum_{e\in\delta^+(v)} f_e - \sum_{e\in\delta^-(v)} f_e = 0$. Adding these equations,

$$\text{value of } f = \sum_{v\in A}\Big(\sum_{e\in\delta^+(v)} f_e - \sum_{e\in\delta^-(v)} f_e\Big).$$

Now count by edges. An edge with both endpoints in B does not appear. An edge (v,w) inside A contributes f_e at v and $-f_e$ at w, net zero. An edge out of A contributes f_e and an edge into A contributes $-f_e$, which gives the equality: the net flow across every cut equals the value. The inequality follows from $f_e \le u_e$ on $\delta^+(A)$ and $f_e \ge 0$ on $\delta^-(A)$. □

Proof of Theorem 2.2. (2)⇒(1). By Lemma 2.3, every flow value is at most every cut capacity. If f attains the capacity of some cut, no flow can have larger value.
(1)⇒(3). If G_f has an s–t path, augmenting along it as in Ford–Fulkerson gives a flow of strictly larger value, so f is not maximum.
(3)⇒(2). Let

$$A = \{v \in V : \ G_f \text{ has an } s\text{–}v \text{ path}\},$$

the set where a graph search of G_f from s gets stuck, and $B = V \setminus A$. Then $s \in A$, and $t \notin A$ by (3), so (A,B) is an (s,t)-cut. No edge of G_f leaves A, since otherwise the search would reach further. Translating back to G:

- every $e \in \delta^+(A)$ is saturated, $f_e = u_e$, since otherwise its forward edge in G_f would leave A;
- every $e \in \delta^-(A)$ has $f_e = 0$, since otherwise its reverse edge in G_f would leave A.

So the inequality of Lemma 2.3 is an equality, and the value of f equals the capacity of (A,B). □

Corollary 2.4. *If f is a flow such that G_f has no s–t path, then f is a maximum flow. In particular, the Ford–Fulkerson algorithm, and any algorithm that halts with such a flow, is correct.*

Corollary 2.5 [Max-flow/min-cut theorem]. *In every network, the maximum value of a flow equals the minimum capacity of an (s,t)-cut.*

Proof. Lemma 2.3 gives $\leq$. Applying (3)$\Rightarrow$(2) to a maximum flow, which satisfies (3), produces a cut of equal capacity. □

Corollary 2.6. *Given a maximum flow, a minimum cut can be computed in linear time.*

Proof. Compute the set A above by BFS or DFS of G_f from s; the proof shows that $(A, V \setminus A)$ is a minimum cut. □

In practice, minimum cuts are usually computed by this reduction to maximum flow.

2.3 The Edmonds–Karp algorithm

Ford–Fulkerson is only PSEUDOPOLYNOMIAL: it runs in polynomial time when the capacities are polynomially bounded integers, but with large capacities it can choose bad paths over and over. The EDMONDS–KARP ALGORITHM is Ford–Fulkerson in which each augmenting path is a *shortest* s–t path of G_f, with the fewest edges. BFS finds one in $O(m)$ time; Dijkstra's algorithm is unnecessary.

Algorithm 2.1: Edmonds–Karp algorithm

Input: Network $G = (V, E)$, s, t, capacities u.
Output: A maximum flow f.

$f_e \leftarrow 0$ for all $e \in E$
while G_f has an s–t path **do**
  $P \leftarrow$ an s–t path of G_f with the fewest edges *// BFS, $O(m)$*
  $\Delta \leftarrow$ minimum residual capacity of an edge of P
  augment f along P by Δ
return f

As a special case of Ford–Fulkerson, it is correct.

Theorem 2.7 [RUNNING TIME OF EDMONDS–KARP]. *The Edmonds–Karp algorithm runs in $O(m^2 n)$ time.*

Since m ranges between about n and n^2, this is between n^3 and n^5: slow, but polynomial regardless of the capacities. For a flow f, let $d(f)$ be the number of edges on a shortest s–t path of G_f, or $+\infty$ if there is none.

Lemma 2.8 [EDMONDS–KARP PROGRESS LEMMA].

(a) *$d(f)$ never decreases during the Edmonds–Karp algorithm.*

(b) *$d(f)$ increases at least once every m iterations.*

A path with n or more edges repeats a vertex, and removing the cycle shortens it, so $d(f) \in \{1, \ldots, n-1\} \cup \{+\infty\}$. By the lemma, after at most mn iterations $d(f) = +\infty$ and the algorithm halts. Each iteration is one BFS plus an $O(n)$ augmentation, which proves Theorem 2.7.

For the analysis, run BFS from s in G_f, giving layers $V_0 = \{s\}, V_1, V_2, \ldots$, where V_{i+1} consists of the vertices not in earlier layers that are one hop from V_i. Each edge of G_f is FORWARD (from V_i to V_{i+1}), SIDEWAYS (within a layer), or BACKWARD (from V_i to V_j with $j < i$). By the definition of BFS, no edge skips a layer forward. The LAYERED NETWORK L_f is the subgraph of G_f of forward edges; vertices beyond the layer of t may be discarded. Every s–t path of L_f has exactly $d(f)$ edges, and every other s–t path of G_f contains a sideways or backward edge and so has at least $d(f) + 1$ edges. Thus L_f encodes all shortest augmenting paths.

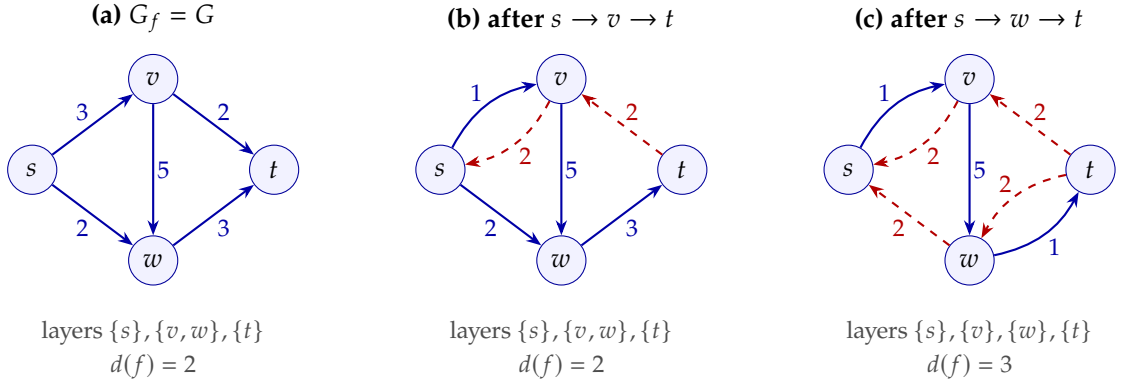


Figure 4: Edmonds–Karp on the running example of Lecture 1. Solid edges are forward residual edges and dashed edges are reverse edges, labeled with residual capacities. (a) Initially $d(f) = 2$. (b) After 2 units on $s \to v \to t$, $d(f)$ is still 2. (c) After 2 units on $s \to w \to t$, $d(f) = 3$; the last augmentation sends 1 unit on $s \to v \to w \to t$.

Example: On the running example (Figure 4), $d(f) = 2$ initially, with shortest paths $s \to v \to t$ and $s \to w \to t$. After sending 2 units on the upper path, the layers are unchanged and $d(f) = 2$. After sending 2 units on the lower path, no two-hop path remains: the layers become $\{s\}, \{v\}, \{w\}, \{t\}$ and $d(f) = 3$. Sending 1 unit on the unique shortest path $s \to v \to w \to t$ gives a maximum flow, of value 5.

Proof of Lemma 2.8. (a) Augmenting along a shortest path P only changes edges of P, which lie in L_f. The only new residual edges are reverses of edges of P, which go from some V_{i+1} to V_i, that is, backward. Hence every edge of the new residual network still goes from a layer V_i to a layer V_j with $j \le i + 1$ (layers taken before the augmentation), so an s–t path still needs at least $d(f)$ edges.
(b) Consider a run of iterations during which $d(f) = c$ stays constant. Let F be the edge set of L_f and $V_0, \ldots, V_c$ its layers before the first of them; F and the V_i stay fixed during the run. As in (a), every new edge goes backward, from some V_{i+1} to V_i. An s–t path with c edges must climb one layer per edge, so it uses only forward edges, none of them new: it lies in F. Inductively, every iteration of the run augments along a path of edges of F. Each augmentation saturates at least one edge $(v, w) \in F$, the one with minimum residual capacity, which then leaves the residual network. It can return only by an augmentation along (w, v), but (w, v) goes backward and every augmentation in the run uses only edges of F. So each iteration

permanently removes an edge of F, and the run lasts at most $|F| \le m$ iterations. □

2.4 Dinic's algorithm and blocking flows

Dinic's algorithm, developed independently and around the same time, resembles Edmonds–Karp but builds L_f explicitly and augments along many shortest paths at once. Its modularity leads to faster algorithms.

Definition 2.9 [Blocking flow]. A blocking flow g in a network is a feasible flow such that every s–t path contains an edge saturated by g ($g_e = u_e$).

Blocking flows are exactly the possible outputs of the naive greedy algorithm of Lecture 1, so a blocking flow need not be maximum: the greedy flow of value 3 on $s \to v \to w \to t$ is blocking, while the maximum is 5. Conversely, every maximum flow is blocking, since G_f then has no s–t path, in particular none of forward edges.

Algorithm 2.2: Dinic's algorithm

```
Input: Network G = (V, E), s, t, capacities u.
Output: A maximum flow f.
f_e ← 0 for all e ∈ E
while G_f has an s–t path do
    construct the layered network L_f from G_f by BFS                    // O(m)
    compute a blocking flow g in L_f (capacities: residual capacities)
    for each edge (v, w) of L_f with g_vw > 0 do
        if (v, w) is the forward edge of e ∈ E then
            f_e ← f_e + g_vw
        else
            f_e ← f_e − g_vw                          // reverse edge of e = (w, v)
return f
```

Dinic's algorithm halts only when G_f has no s–t path, so it returns a maximum flow by Corollary 2.4.

Lemma 2.10 [Dinic progress lemma]. *If h is obtained from f by augmenting by a blocking flow g of L_f, then $d(h) > d(f)$.*

Proof. Let $c = d(f)$, with layers $V_0, \dots, V_c$ and edge set F of L_f. As in Lemma 2.8(a), the new residual edges are reverses of edges of F and go backward, so every edge of G_h climbs at most one layer, and $d(h) \ge c$. Suppose G_h had an s–t path with exactly c edges. It climbs one layer per edge, so all its edges are old forward edges, that is, edges of F, each with positive residual capacity in G_h. The residual capacity of $e \in F$ in G_h is its capacity in L_f minus g_e, so g saturates no edge of this path, an s–t path of L_f. This contradicts that g is blocking in L_f. □

On the running example, L_f initially contains $s \to v \to t$ and $s \to w \to t$ but not the sideways

edge (v, w). A blocking flow sends 2 units on each path, giving the residual network of Figure 4(c), where d has risen from 2 to 3. The next layered network is the single path $s \to v \to w \to t$, and its blocking flow of 1 unit completes a maximum flow.

Since $d(f) \leq n - 1$ until it becomes infinite, Dinic's algorithm performs at most n iterations: maximum flow reduces to n blocking flow computations in layered networks, and the running time is $O(n \cdot BF)$, where BF is the time to compute a blocking flow in a layered network. The proof of Lemma 2.8(b) shows how to compute a blocking flow in $O(m^2)$ time, by repeatedly saturating a path of L_f. A DFS-based method computes one in $O(mn)$ time, so Dinic's algorithm runs in $O(n^2m)$, improving on Edmonds–Karp. With fancy data structures a blocking flow can be computed in near-linear time, giving maximum flow algorithms with running time close to $O(mn)$.

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 2: Augmenting Path Algorithms for Maximum Flow.

Lecture 3: The Push-Relabel Algorithm for Maximum Flow

The algorithms so far are augmenting path algorithms: they maintain a flow and increase its value each iteration (Ford–Fulkerson, Edmonds–Karp in $O(m^2n)$ time, and Dinic's algorithm in $O(n^2m)$ time). The push-relabel paradigm of the mid-1980s works differently, and push-relabel algorithms are often the method of choice in practice.

In the network of Figure 5, with k large, the maximum flow value is k. Ford–Fulkerson and Edmonds–Karp augment k times and re-explore the long path through x each time, taking $\Omega(k^2)$ time. It would be better to route k units from s to x in one step and then distribute them over the k paths to t, in $O(k)$ time overall. If fewer than k units could continue from x, the remainder would be sent back to the source. Push-relabel algorithms organize this idea for arbitrary networks.

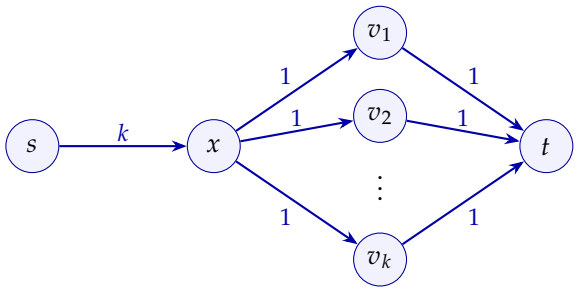


Figure 5: One edge of large capacity k followed by k disjoint unit-capacity paths from x to t (three drawn). Augmenting path algorithms take $\Omega(k^2)$ time here.

3.1 Preflows and pushes

Definition 3.1 [Preflow and excess]. A preflow is a nonnegative vector $\{f_e\}_{e \in E}$ satisfying the capacity constraints $f_e \leq u_e$ and the *relaxed conservation constraints*: for every vertex $v \neq s$, the flow entering v is at least the flow leaving v. The excess of $v \neq s, t$ is

$$\alpha_f(v) = \text{flow entering } v - \text{flow leaving } v \geq 0.$$

A preflow is a flow if and only if every vertex $v \neq s, t$ has zero excess, so restoring feasibility means eliminating all excesses. The residual network G_f of a preflow is defined exactly as for a flow, again omitting edges of zero residual capacity.

Relaxed conservation allows augmenting along a single edge at a time.

Algorithm 3.1: Push(v)

```
choose an outgoing edge (v, w) of v in G_f
Δ ← min{α_f(v), residual capacity of (v, w)}
push Δ units along (v, w)
```

Pushing updates f as before: if (v, w) is the forward version of $e = (v, w) \in E$, then f_e increases by Δ; if it is the reverse version of $e = (w, v)$, then f_e decreases by Δ. If the residual capacity is the binding constraint, the push is SATURATING; otherwise the excess at v is binding and the push is NON-SATURATING, emptying v (a tie counts as non-saturating). Afterwards w has positive excess.

3.2 Heights and invariants

Pushing excess around arbitrarily need not terminate: excess could circle forever around a directed cycle. To organize the pushes, maintain a HEIGHT $h(v)$, a nonnegative integer, for each vertex. Picture $h(v)$ as the altitude of v, so residual edges go uphill, stay flat, or go downhill.

Definition 3.2 [INVARIANTS].

1. $h(s) = n$;
2. $h(t) = 0$;
3. for every edge (v, w) of the current residual network, $h(v) \leq h(w) + 1$.

The third invariant says residual edges descend gradually, by at most one per hop. For example, if v has residual edges to vertices of heights 3, 4, and 6, then $h(v) \leq 4$.

Lemma 3.3 [INVARIANTS DISCONNECT s AND t]. *If the invariants hold for a preflow f, then G_f has no s–t path. Consequently, if f is also a flow, it is a maximum flow.*

Proof. An s–t path may be taken to have at most $n - 1$ edges. Along it the height drops from n to 0, but by at most 1 per edge, so by at most $n - 1$ in total, a contradiction. The second statement is the optimality condition of Lecture 2. □

The two paradigms are mirror images. Augmenting path algorithms maintain a feasible flow and work toward disconnecting s and t in the residual network. Push-relabel algorithms maintain that s and t are disconnected in the residual network and work toward feasibility, that is, the conservation constraints.

3.3 The algorithm

The invariants require $h(s) = n$ and $h(t) = 0$; set $h(v) = 0$ for every other vertex too. The zero preflow would violate the third invariant, since edges out of s would descend from n to 0. So the initial preflow saturates every edge out of s, removing these edges from the residual network.

Algorithm 3.2: Push-relabel algorithm

Input: Network $G = (V, E)$, s, t, capacities u.
Output: A maximum flow f.

$h(s) \leftarrow n$; $h(v) \leftarrow 0$ for all $v \neq s$
$f_e \leftarrow u_e$ for every edge e out of s; $f_e \leftarrow 0$ for all other edges
while some vertex $v \neq s, t$ has $\alpha_f(v) > 0$ **do**
    choose such a vertex v of maximum height $h(v)$ *// ties arbitrarily*
    if G_f has an edge (v, w) with $h(v) = h(w) + 1$ **then**
        Push(v) along such an edge *// downhill only*
    else
        $h(v) \leftarrow h(v) + 1$ *// relabel*
return f

After initialization all three invariants hold and f is a preflow. Each iteration processes the highest vertex with excess, pushing downhill if possible and otherwise RELABELING it.

Lemma 3.4 [INVARIANTS ARE MAINTAINED]. *The three invariants hold throughout the algorithm.*

Proof. Neither s nor t is ever relabeled, so the first two invariants hold. A relabel at v changes only $h(v)$, and is performed only when every residual edge out of v is flat or uphill; after it, each such edge descends by at most 1. A push from v to w changes only the preflow. The only possible new residual edge is (w, v), and since the push went downhill, this edge goes uphill. □

By Lemmas 3.3 and 3.4, if the algorithm terminates, then it terminates with a maximum flow: the loop exits only when no vertex has excess, so the final preflow is a flow.

3.4 Example

Consider the network of Figure 6(a), whose maximum flow value is 3, as certified by the cut $(\{s, w\}, \{v, t\})$. With $n = 4$, initialization sets $h(s) = 4$ and saturates (s, v) and (s, w), so v and w have excesses 1 and 100 (panel b). The algorithm then proceeds as follows; one choice is made among ties.

1. Both v and w have height 0. Process v: it has no downhill edge, so it is relabeled to $h(v) = 1$.
2. Now v is the highest vertex with excess, and (v, w) and (v, t) go downhill. Pushing to w along (v, w) is non-saturating: $\alpha(v) = 0$ and $\alpha(w) = 101$.

3. w is relabeled to 1, then saturates (w, t) with 1 unit, so $\alpha(w) = 100$.
4. w is relabeled to 2, then pushes 1 unit on (w, v) and 1 unit on the reverse of (v, w), both saturating: $\alpha(w) = 98$ and $\alpha(v) = 2$ (panel c).
5. w is higher than v but its only residual edge goes to s. It is relabeled three times, to height 5, and then pushes its entire excess 98 back to s along the reverse edge (w, s).
6. v pushes 2 units along (v, t), and the algorithm halts with a maximum flow of value 3.

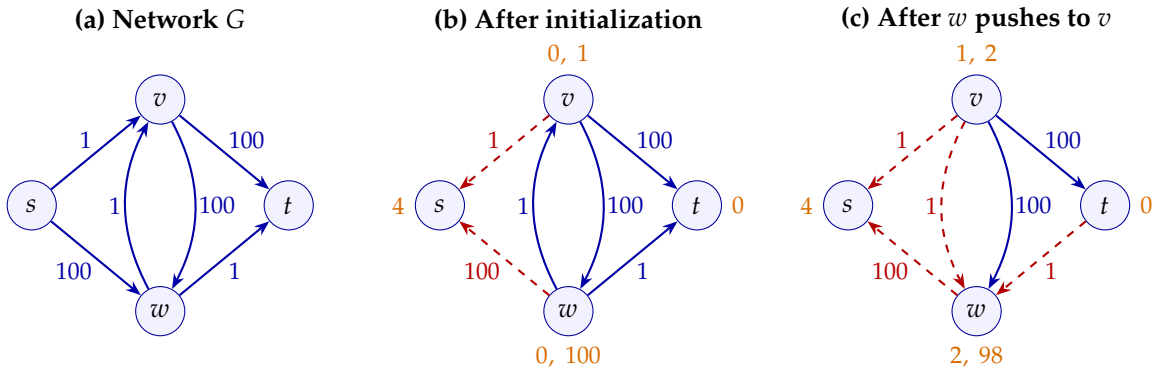


Figure 6: Push-relabel on a network with maximum flow value 3. Residual edges are labeled with residual capacities; dashed edges are reverse edges. Labels give h for s, t and the pair h, α_f for v, w.

3.5 Analysis

Theorem 3.5 [Running time of push-relabel]. *The push-relabel algorithm terminates after $O(n^2)$ relabels and $O(n^3)$ pushes.*

The hidden constants are at most 2. With suitable data structures, which find a highest vertex with excess in $O(1)$ amortized time, the algorithm runs in $O(n^3)$ time. Unlike for augmenting paths, there is no obvious progress measure; the analysis uses the relabels themselves. It rests on the following lemma, proved in the next subsection.

Lemma 3.6 [Key lemma]. *If v has positive excess in the preflow f, then G_f has a path from v to s.*

Corollary 3.7 [Height bound]. *Every vertex always has height at most $2n$.*

Proof. A vertex v is relabeled only when it has excess. Then by Lemma 3.6, G_f has a v–s path, which may be taken to have at most $n - 1$ edges. Each edge descends by at most 1 and $h(s) = n$, so $h(v) \le 2n - 1$ before the relabel and $h(v) \le 2n$ after it. □

Corollary 3.8 [Relabel bound]. *Each vertex is relabeled at most $2n$ times, so the algorithm performs $O(n^2)$ relabels.*

Assume G has no parallel edges, which is without loss of generality since parallel edges in the same direction can be merged by adding capacities; then $m = O(n^2)$.

Lemma 3.9 [Saturating pushes]. *Between two saturating pushes on the same residual edge (v, w) in the same direction, each of v and w is relabeled at least twice.*

Proof. At a saturating push on (v, w), $h(v) = h(w) + 1$, and (v, w) leaves the residual network. It reappears only after a push on (w, v), which requires $h(w) = h(v) + 1$. Heights never decrease, so w must first be relabeled at least twice. Then a second push on (v, w) requires $h(v) = h(w) + 1$ again, so v must be relabeled at least twice. □

With Corollary 3.8, each edge suffers $O(n)$ saturating pushes, for $O(mn) = O(n^3)$ in total. Bounding non-saturating pushes is the one place where the highest-vertex rule matters.

Lemma 3.10 [Non-saturating pushes]. *Between any two consecutive relabels, of any vertices, there are at most n non-saturating pushes.*

Proof. Call the interval between two consecutive relabels a phase. When a non-saturating push occurs at v, the rule chooses v as a highest vertex with excess, so

$$h(v) \geq h(w) \quad \text{for every vertex } w \text{ with excess.}$$

Heights do not change during a phase, and pushes move excess only downhill, so this inequality persists until the next relabel. The push empties v, and v can regain excess only by receiving flow from a higher vertex with excess, which does not exist during the phase. Hence each vertex has at most one non-saturating push per phase. □

Proof of Theorem 3.5. Corollary 3.8 bounds relabels by $O(n^2)$. Lemma 3.9 bounds saturating pushes by $O(n)$ per edge, so $O(mn)$ in total. There are $O(n^2)$ phases, and by Lemma 3.10 each contains at most n non-saturating pushes, so there are $O(n^3)$ of them. □

3.6 Proof of the key lemma

The intuition is that excess reached v from s somehow, and the reverse edges of that flow form a trail back to s.

Proof of Lemma 3.6. Fix a preflow f and let

$$A = \{v \in V : \ G \text{ has an } s\text{–}v \text{ path } P \text{ with } f_e > 0 \text{ for all } e \in P\},$$

the set where a graph search from s along flow-carrying edges gets stuck. For $v \in A$, the reverse edges of such a path form a v–s path in G_f. So it suffices to show that every vertex with excess lies in A.

Let $B = V \setminus A$, which does not contain s and may be empty. Consider

$$\sum_{v \in B} \big(\text{flow out of } v - \text{flow into } v\big). \tag{3.1}$$

Since f is a preflow and $s \notin B$, every term is nonpositive. Counting by edges as in Lecture 2, edges inside A do not appear, edges inside B cancel, an edge out of B contributes f_e, and an edge into B contributes $-f_e$, so (3.1) equals

$$\sum_{e \in \delta^+(B)} f_e - \sum_{e \in \delta^-(B)} f_e.$$

Each term of the first sum is nonnegative. Each term of the second sum is zero: an edge of $\delta^-(B)$ goes from A to B, and if it carried flow, the search defining A would not have stopped. So (3.1) is both nonpositive and nonnegative, hence zero, and since its terms are nonpositive, each is zero. Thus every vertex of B satisfies conservation, and every vertex with excess lies in A. □

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 3: The Push-Relabel Algorithm for Maximum Flow.

Lecture 4: Applications of Maximum Flows and Minimum Cuts

Maximum flow directly models traffic in transportation networks, oil in distribution networks, and packets in communication networks, and many other problems are maximum flow or minimum cut in disguise. This lecture gives two examples, image segmentation and bipartite matching, and derives Hall's theorem from the max-flow/min-cut theorem.

4.1 The minimum cut problem

In the minimum (s,t)-cut problem, the input is as for maximum flow: a directed graph $G = (V, E)$, a source s, a sink t, and edge capacities. The feasible solutions are the (s,t)-cuts (A, B), partitions of V with $s \in A$ and $t \in B$, and the goal is to minimize

$$\text{capacity of } (A,B) = \sum_{e \in \delta^+(A)} u_e,$$

the total capacity of edges sticking out of A; edges sticking in do not count.

Proposition 4.1 [Minimum cut reduces to maximum flow]. *The minimum (s,t)-cut problem reduces in linear time to the maximum flow problem.*

Given a maximum flow f, a BFS or DFS from s in G_f gets stuck at a set A, and $(A, V \setminus A)$ is a cut whose capacity equals the value of f (Lecture 2). Since no cut has capacity less than the value of a flow, it is a minimum cut. Some algorithms solve minimum cut without maximum flow, especially in undirected graphs, but in practice this reduction is common.

4.2 Image segmentation

The task is to classify each pixel of an image as foreground or background. The input is an undirected graph $G = (V, E)$ whose vertices are pixels and whose edges join neighboring pixels, commonly a grid graph (sometimes with diagonals); the solution works for any graph. Each vertex v has nonnegative values a_v and b_v, and each edge e a nonnegative penalty p_e. A feasible output is a partition of V into a foreground X and a background Y, either possibly empty, and the goal is to

$$\text{maximize} \quad \sum_{v \in X} a_v + \sum_{v \in Y} b_v - \sum_{e \in \delta(X)} p_e, \tag{4.1}$$

where $\delta(X)$ is the set of edges with one endpoint on each side.

A vertex earns a_v in the foreground and b_v in the background. These values come from a prior: a_v

is large and b_v small if v is likely foreground, and vice versa. Alone they would make the problem trivial, assigning each pixel independently to the side with the larger prize. The penalties encode that images are mostly smooth: p_e is paid when neighboring pixels are separated. The objective is a massaged version of a maximum likelihood objective.

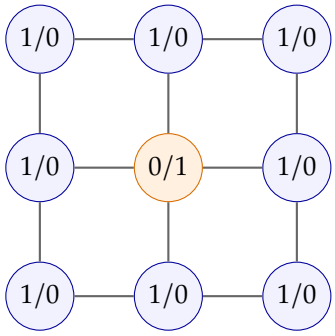


Figure 7: A 3×3 grid with each pixel labeled a_v/b_v. All boundary pixels favor the foreground, and the center pixel favors the background.

Example: In Figure 7, if every $p_e = 0$, the best partition puts the boundary in X and the center in Y, with value 9. If every $p_e = 1$, this partition cuts four edges and has value 5, while putting all nine pixels in X has value 8, which is optimal. The penalties recover a corrupted pixel inside a homogeneous region.

Theorem 4.2 [Segmentation reduces to minimum cut]. *The image segmentation problem reduces in linear time to the minimum (s,t)-cut problem, and hence to the maximum flow problem.*

The output, a partition of the vertices into two pieces, suggests a cut problem. The differences are that segmentation maximizes rather than minimizes, has no source or sink, is undirected, and has vertex parameters. Two transformations remove them.

Transforming the objective. Negating (4.1) gives the problem of minimizing $\sum_{e\in\delta(X)} p_e - \sum_{v\in X} a_v - \sum_{v\in Y} b_v$. Adding the constant $\sum_{v\in V} a_v + \sum_{v\in V} b_v$ to every solution removes the negative terms without changing the optimal partition, giving

$$\text{minimize} \quad \sum_{e\in\delta(X)} p_e + \sum_{v\in Y} a_v + \sum_{v\in X} b_v. \tag{4.2}$$

Transforming the graph. Build a directed graph $G' = (V', E')$ with

- $V' = V \cup \{s, t\}$, adding a new source and sink;
- for each edge $e = \{v, w\} \in E$, the two directed edges (v, w) and (w, v), each with capacity p_e;
- for each $v \in V$, an edge (s, v) with capacity a_v and an edge (v, t) with capacity b_v.

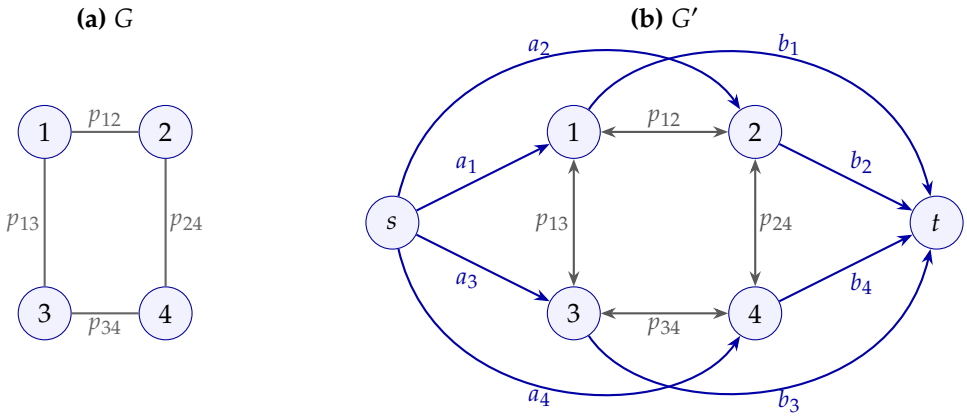


Figure 8: The reduction on a four-pixel example. (a) The pixel graph with penalties. (b) The network G'; each double-headed arrow is a pair of directed edges, each of capacity p_e.

Proof of Theorem 4.2. Partitions (X, Y) of V correspond bijectively to (s, t)-cuts of G', via $A = X \cup \{s\}$ and $B = Y \cup \{t\}$. The edges sticking out of $X \cup \{s\}$ are exactly

- (s, v) for each $v \in Y$, of capacity a_v;
- (v, t) for each $v \in X$, of capacity b_v;
- for each $e \in \delta(X)$, exactly one of its two directed copies (the other points back into A), of capacity p_e.

So the cut's capacity is $\sum_{v \in Y} a_v + \sum_{v \in X} b_v + \sum_{e \in \delta(X)} p_e$, the objective (4.2) of (X, Y). Hence a minimum cut of G' gives an optimal segmentation, and G' is built in linear time. □

4.3 Bipartite matching

In the BIPARTITE MATCHING problem, the input is an undirected bipartite graph $G = (V \cup W, E)$, every edge having one endpoint in V and one in W. A MATCHING is a set $S \subseteq E$ of edges sharing no endpoints, and the goal is a matching of maximum cardinality. A matching covering all vertices is PERFECT. For example, the 4-cycle is bipartite and has a perfect matching of size 2; a star is bipartite, but however many vertices it has, its maximum matching has size 1; a triangle is not bipartite and its maximum matching has size 1. Matching theory is nicer and its algorithms faster for bipartite graphs, and most applications are bipartite: workers to jobs, courses to rooms and time slots, residents to hospitals.

Proposition 4.3 [MATCHING REDUCES TO MAXIMUM FLOW]. *Maximum-cardinality bipartite matching reduces in linear time to maximum flow.*

Construct G' with vertex set $V \cup W \cup \{s, t\}$: direct every edge of G from V to W, and add edges (s, v) for all $v \in V$ and (w, t) for all $w \in W$. Edges at s or t get capacity 1, reflecting that each vertex

is matched at most once. The edges from V to W may get any capacity at least 1; for simplicity, give them infinite capacity.

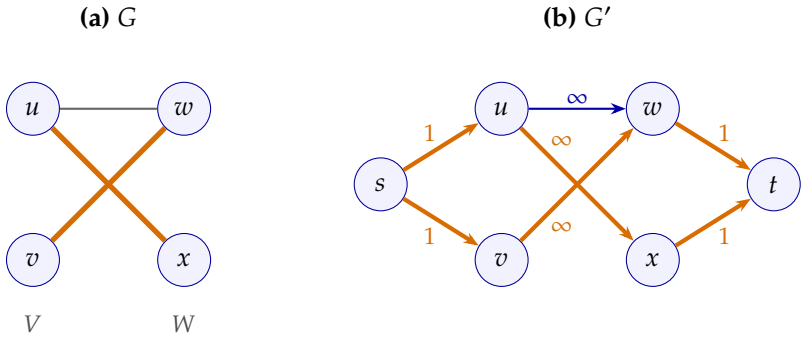


Figure 9: (a) A bipartite graph with a maximum matching $\{(u, x), (v, w)\}$ (thick). (b) The network G'; the matching corresponds to the integral flow of value 2 on the thick edges, with (v, w) corresponding to one unit on $s \to v \to w \to t$.

Matchings of G correspond one-to-one to integral flows of G', with an edge (v, w) corresponding to one unit on $s \to v \to w \to t$. Indeed, in an integral flow each $v \in V$ receives at most one unit from s, so at most one edge out of v carries flow, and each $w \in W$ sends at most one unit to t, so at most one edge into w carries flow. The flow-carrying edges therefore form a matching whose size is the flow value, and every matching arises this way. All the maximum flow algorithms so far return an integral maximum flow when capacities are integers, since the current (pre)flow, the residual capacities, and the augmentation amounts stay integral. So the V–W edges carrying flow in such a maximum flow of G' form a maximum matching.

4.4 Hall's theorem

Asking again *how do we know when we are done?*, the max-flow/min-cut theorem translates through this reduction into a famous optimality condition for bipartite matching. Let $G = (V \cup W, E)$ be bipartite with $|V| \le |W|$, renaming the sides if necessary, and now call a matching PERFECT if it matches every vertex of V; a perfect matching is maximum. For $S \subseteq V$, let

$$N(S) = \{w \in W : (v, w) \in E \text{ for some } v \in S\}$$

be the union of the neighborhoods of the vertices of S.

If some $S \subseteq V$ has $|N(S)| < |S|$, a CONSTRICTING SET, then G has no perfect matching, since the vertices of S can be matched only to the fewer vertices of $N(S)$. In Figure 10, the three vertices of S have only two neighbors, so no perfect matching exists. Hall's theorem says that constricting sets are the only obstacles.

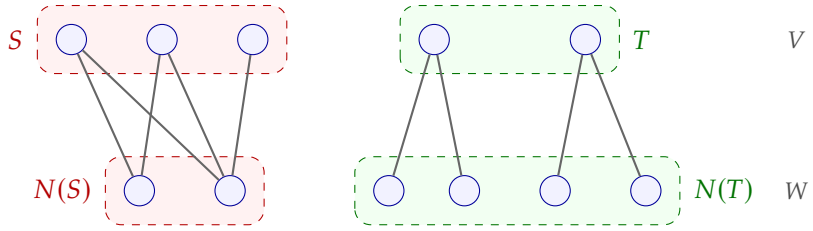


Figure 10: A bipartite graph with $|V| = 5$ and $|W| = 6$. The set S is constricting, $|N(S)| = 2 < 3 = |S|$, so there is no perfect matching. The set T is not constricting, since $|N(T)| = 4$.

Theorem 4.4 [Hall's theorem]. *A bipartite graph $(V \cup W, E)$ with $|V| \leq |W|$ has a perfect matching if and only if $|N(S)| \geq |S|$ for every $S \subseteq V$.*

So both the existence and the nonexistence of a perfect matching have short certificates: a matching, or a constricting set.

Proof. The "only if" direction was shown above. For "if", assume $|N(S)| \geq |S|$ for all $S \subseteq V$, and let G' be the network of the matching reduction. We claim that every (s,t)-cut (A,B) of G' has capacity at least $|V|$. Then by the max-flow/min-cut theorem the maximum flow value is at least $|V|$, and an integral flow of value $|V|$ corresponds to a perfect matching.
Let $S = A \cap V$. Since $s \in A$, the unit-capacity edges from s to the $|V| - |S|$ vertices of $V \setminus S$ all leave A. If some $w \in N(S)$ is not in A, then an infinite-capacity edge from S to w leaves A, and the cut has infinite capacity. Otherwise $N(S) \subseteq A$, and the unit-capacity edges from $N(S)$ to t all leave A. Hence

$$\text{capacity of } (A,B) \geq (|V| - |S|) + |N(S)| \geq |V|. \qquad \square$$

The same argument extends to every bipartite graph with $|V| \leq |W|$:

$$\text{size of a maximum matching} = \min_{S \subseteq V} \big(|V| - (|S| - |N(S)|)\big),$$

where the inequality $\leq$ holds because every matching leaves at least $|S| - |N(S)|$ vertices of S unmatched.

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 4: Applications of Maximum Flows and Minimum Cuts.

Lecture 5: Minimum-Cost Bipartite Matching

A bipartite graph $G = (V \cup W, E)$ has every edge joining V to W, and a matching is a set $M \subseteq E$ of edges with no shared endpoints. When a graph has many perfect matchings, some may be preferable to others, as when several workers can do a job but some do it better. To model this, each edge $e \in E$ has a cost c_e. Three assumptions are made for convenience; none is crucial.

1. $|V| = |W| = n$. Enforce this by adding dummy vertices with no incident edges to the smaller side.
2. G has at least one perfect matching. Enforce this by adding very expensive dummy edges, for instance one from the ith vertex of V to the ith vertex of W for each i.
3. $c_e \geq 0$ for all e. If the most negative cost is $-M$, add M to every cost; this adds nM to every perfect matching and so does not change the problem.

The minimum-cost perfect bipartite matching problem asks for the perfect matching M minimizing $\sum_{e \in M} c_e$. Negating costs gives the equivalent maximum-weight perfect bipartite matching problem.

With equal costs this is maximum-cardinality matching, which reduces to maximum flow (Lecture 4). With general costs there seems to be no natural reduction: capacities affect which flows are feasible but not their values, while costs do the opposite. The plan follows the push-relabel paradigm: identify optimality conditions, then maintain invariants implying them while working toward feasibility, that is, a perfect matching.

5.1 Optimality conditions

Definition 5.1 [Alternating and negative cycles]. Let M be a matching of G. A cycle C of G is M-alternating if every other edge of C belongs to M. An M-alternating cycle is negative if

$$\sum_{e \in C \cap M} c_e > \sum_{e \in C \setminus M} c_e,$$

and nonnegative otherwise.

Since G is bipartite, every cycle is even. Toggling an M-alternating cycle C, replacing $C \cap M$ by $C \setminus M$, gives a matching that matches exactly the same vertices: vertices off C are unaffected, and each vertex of C is matched to its other neighbor on C. If M is perfect, toggling C saves $\sum_{e \in C \cap M} c_e$ and costs $\sum_{e \in C \setminus M} c_e$, so it yields a cheaper perfect matching if and only if C is negative. A negative cycle is thus a short, convincing proof that M is not minimum-cost (Figure 11). The converse also holds.

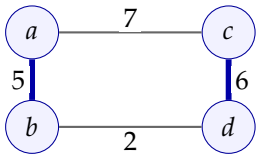


Figure 11: With $M = \{(a, b), (c, d)\}$ (thick edges, cost $5 + 6 = 11$), the cycle $a \to b \to d \to c \to a$ is M-alternating and negative. Toggling it gives the perfect matching $\{(a, c), (b, d)\}$ of cost $7 + 2 = 9$.

Theorem 5.2 [Optimality conditions]. *A perfect matching M of a bipartite graph has minimum cost if and only if there is no negative M-alternating cycle.*

Proof. The "only if" direction is the toggling argument above. Conversely, suppose no M-alternating cycle is negative, and let M' be any perfect matching. Every vertex has degree 1 in both M and M', so in the symmetric difference $M \oplus M'$ every vertex has degree 0 (matched to the same vertex in both) or 2. Hence $M \oplus M'$ is a union of vertex-disjoint cycles, each M-alternating. Since $(M \oplus M') \oplus M = M'$, the matching M' is obtained from M by toggling these disjoint cycles, which do not interfere. Each is nonnegative, so each toggle does not decrease the cost, and M' costs at least as much as M. □

5.2 Reduced costs and invariants

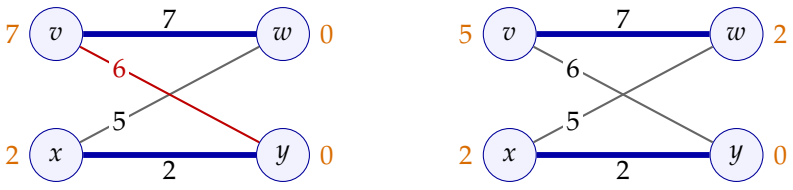
Analogous to heights in push-relabel, keep a price $p_v \in \mathbb{R}$ (positive or negative) for each vertex $v \in V \cup W$. The reduced cost of an edge $e = (v, w)$ is

$$c_e^p = c_e - p_v - p_w.$$

For the current matching M and prices p, the algorithm maintains two invariants.

1. Every edge of G has nonnegative reduced cost.
2. Every edge of M is tight: its reduced cost is zero.

For example, in Figure 12, making (v, w) and (x, y) tight with $p_w = p_y = 0$ forces $p_v = 7$ and $p_x = 2$, but then $c_{vy}^p = 6 - 7 = -1$ violates invariant 1. Resetting $p_v = 5$ and $p_w = 2$ keeps both matched edges tight and gives the other two edges reduced cost 1.



(a) $c_{vy}^p = 6 - 7 - 0 = -1$ **(b) both invariants hold**

Figure 12: The perfect matching $\{(v, w), (x, y)\}$ (thick edges), with edge costs in black and prices in orange.

Lemma 5.3 [Invariants imply optimality]. *If M is a perfect matching and both invariants hold, then M is a minimum-cost perfect matching.*

Proof. By Theorem 5.2, it suffices to show that every M-alternating cycle C is nonnegative. Every vertex of C is an endpoint of exactly one edge of $C \cap M$ and exactly one edge of $C \setminus M$. Adding and subtracting $\sum_{v \in C} p_v$ therefore gives

$$\sum_{e \in C \cap M} c_e = \sum_{e \in C \cap M} c_e^p + \sum_{v \in C} p_v \qquad \text{and} \qquad \sum_{e \in C \setminus M} c_e = \sum_{e \in C \setminus M} c_e^p + \sum_{v \in C} p_v.$$

The price terms agree. The first reduced-cost sum is 0 by invariant 2 and the second is at least 0 by invariant 1, so the left side of the first equation is at most that of the second. □

5.3 The Hungarian algorithm

It remains to compute some perfect matching while maintaining the invariants. The Hungarian algorithm, also called the Kuhn–Munkres algorithm, starts from $M = \emptyset$ and $p = 0$. Invariant 2 holds vacuously and invariant 1 holds because costs are nonnegative. While M is not perfect, it augments along a good path if there is one, and otherwise finds a good set and updates prices.

Good paths. A path P from v to w is good if

1. $v \in V$ and $w \in W$ are both unmatched in M (so P has odd length);
2. P alternates between edges not in M and edges in M, starting and ending with edges not in M;
3. every edge of P is tight.

Augmenting replaces M by $M \oplus P$. Since $|P \cap M| = |P \setminus M| - 1$, the matching grows by one edge. No reduced cost changes, so invariant 1 still holds, and every edge of P is tight, so invariant 2 still holds. After n augmentations M is perfect, and by Lemma 5.3 it is optimal. To find a good path, breadth-first search, modified to enforce M-alternation, suffices.

Algorithm 5.1: Searching for a good path

Input: Bipartite G, matching M, prices p.
Output: A good path, or "stuck".

```
Level 0 ← {r}, where r is the first unmatched vertex of V
for i = 1, 2, ... until stuck or an unmatched vertex w ≠ r is reached do
    if i is odd then
        Level i ← unseen vertices joined by a tight edge to level i − 1
    else
        Level i ← vertices matched in M to vertices of level i − 1
return the r–w tree path if an unmatched w ≠ r was reached, and stuck otherwise
```

Because G is bipartite, even levels lie in V and odd levels in W. This gives the following property.

(∗) If either endpoint of an edge $(v, w) \in M$ is reached, then both are, at consecutive levels i and $i + 1$ with i odd.

Indeed, a reached w is at an odd level i, and its partner v goes to level $i + 1$: $v \neq r$ since r is unmatched, and v cannot have been reached earlier, because every vertex at an even level ≥ 2 is the partner of a vertex at the previous level. Conversely, a reached $v \neq r$ is at an even level, as the partner of w at the previous level. In a non-bipartite graph, both endpoints of a matched edge could appear in the same level.

If the search reaches an unmatched $w \neq r$, then w is at an odd level, since every vertex at an even level after 0 is matched. All tree edges are tight and every tree path is M-alternating, so the r–w tree path is good. Figure 13 shows a search that gets stuck.

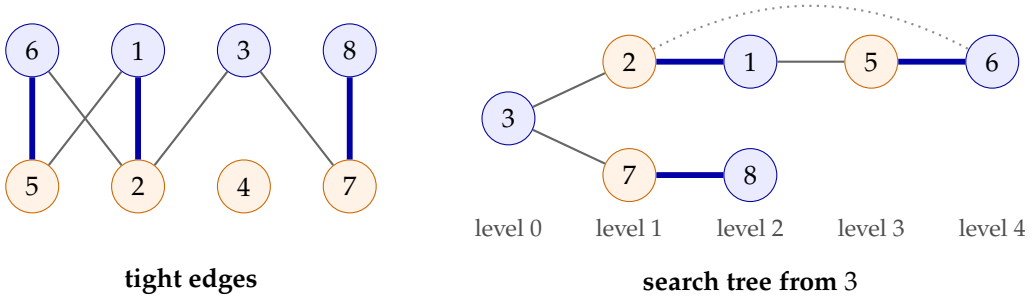


Figure 13: Left: the tight edges for a matching (thick edges), with V on top (blue) and W below (orange). Right: the search from the unmatched vertex 3. Level 2 contains only 1 and 8, the partners of level-1 vertices; ordinary BFS would also put 6 there. The dotted edge $(2, 6)$ is not used. The search gets stuck after level 4, and vertex 4 is never reached.

Good sets. Suppose the search gets stuck. Let $S \subseteq V$ be the vertices at even levels, and let

$$N(S) = \{w \in W : (v, w) \text{ is tight for some } v \in S\}.$$

Then $N(S)$ is exactly the set of vertices at odd levels: each odd-level vertex is joined by a tight edge to the previous level, and conversely the search gets stuck only when no tight edge leaves an even-level vertex to an unseen vertex. The set S is a GOOD SET:

1. S contains an unmatched vertex, namely r;
2. every vertex of $N(S)$ is matched in M to a vertex of S, since the search failed.

The PRICE UPDATE raises p_v by Δ for every $v \in S$ and lowers p_w by Δ for every $w \in N(S)$. For an edge (v, w) this has four effects.

1. $v \notin S$, $w \notin N(S)$: c^p_{vw} is unchanged.

2. $v \in S, w \in N(S)$: c^p_{vw} is unchanged.
3. $v \notin S, w \in N(S)$: c^p_{vw} rises by Δ.
4. $v \in S, w \notin N(S)$: c^p_{vw} falls by Δ.

By $(*)$, every edge of M is in category 1 or 2, so it stays tight and invariant 2 is preserved. Edges of category 3 may stop being tight, but none is in M, since vertices of $N(S)$ are matched into S. Only category 4 endangers invariant 1, and by the definition of $N(S)$ none of its edges is tight. So take

$$\Delta = \min\{c^p_{vw} : (v, w) \in E,\ v \in S,\ w \notin N(S)\} > 0.$$

Some such edge exists: otherwise every neighbor of S in G lies in $N(S)$, and the two properties of a good set give $|N(S)| < |S|$, violating Hall's condition and contradicting the assumption that G has a perfect matching.

Each price update makes progress. It makes some edge (v, w) tight with $v \in S$ and $w \notin N(S)$. All edges of the old search tree are in category 2 and stay tight, and M and r are unchanged. So the next search from r regrows the same tree, reaches v, and now also reaches w. The tree therefore gains a vertex of W with each update, so at most n updates occur between consecutive augmentations.

Algorithm 5.2: The Hungarian algorithm

Input: Bipartite $G = (V \cup W, E)$ with $|V| = |W| = n$, a perfect matching, and costs $c_e \geq 0$.
Output: A minimum-cost perfect matching.

$M \leftarrow \emptyset$; $p_v \leftarrow 0$ for all $v \in V \cup W$
while M is not a perfect matching **do**
  Build the search tree T from the first unmatched $r \in V$
  if T contains an unmatched vertex $w \in W$ **then**
    $P \leftarrow$ the r–w path in T; $M \leftarrow M \oplus P$
  else
    $S \leftarrow$ even levels of T; $N(S) \leftarrow$ odd levels of T
    $\Delta \leftarrow \min\{c^p_{vw} : (v, w) \in E,\ v \in S,\ w \notin N(S)\}$
    $p_v \leftarrow p_v + \Delta$ for all $v \in S$; $p_w \leftarrow p_w - \Delta$ for all $w \in N(S)$
return M

Theorem 5.4 [Hungarian algorithm]. *The Hungarian algorithm returns a minimum-cost perfect matching in $O(mn^2)$ time.*

Proof. Both invariants hold throughout, so the returned perfect matching is optimal by Lemma 5.3. There are at most n augmentations, since $|M| \leq n$, and at most n price updates between consecutive augmentations, since the tree can contain at most n vertices of W. Each iteration, building the tree and computing P or S, $N(S)$, and Δ, takes $O(m)$ time. □

5.4 Example

Run the algorithm on the graph of Figure 14, with $V = \{v, x\}$ and $W = \{w, y\}$.

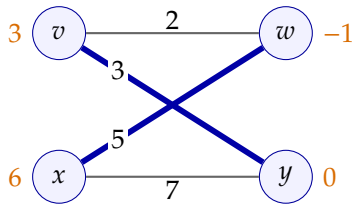


Figure 14: Edge costs, the final prices (orange), and the minimum-cost perfect matching $\{(v, y), (x, w)\}$ of cost 8 (thick edges). All initial prices are 0.

Iteration	Outcome	(p_v, p_w, p_x, p_y)	M
1	stuck at v: $S = \{v\}$, $N(S) = \emptyset$, $\Delta = 2$	$(2, 0, 0, 0)$	$\emptyset$
2	good path $v \to w$	$(2, 0, 0, 0)$	$\{(v, w)\}$
3	stuck at x: $S = \{x\}$, $N(S) = \emptyset$, $\Delta = 5$	$(2, 0, 5, 0)$	$\{(v, w)\}$
4	stuck: $S = \{x, v\}$, $N(S) = \{w\}$, $\Delta = 1$	$(3, -1, 6, 0)$	$\{(v, w)\}$
5	good path $x \to w \to v \to y$	$(3, -1, 6, 0)$	$\{(v, y), (x, w)\}$

Initially no edge is tight. In iteration 3, (x, w) becomes tight, while (v, y) and (x, y) have reduced costs 1 and 2. In iteration 4 the search from x reaches w, already matched to v, and gets stuck at v; the update with $\Delta = 1$ makes (v, y) tight, and the final search finds a good path to y.

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 5: Minimum-Cost Bipartite Matching.

Lecture 6: Generalizations of Maximum Flow and Bipartite Matching

Maximum flow, maximum-cardinality bipartite matching, and minimum-cost bipartite matching each have a natural generalization (Figure 15). Minimum-cost flow is a common generalization of maximum flow and minimum-cost bipartite matching, and matching also makes sense in non-bipartite graphs, with or without costs. Together with minimum spanning trees and shortest paths, these six problems are arguably the most fundamental problems of combinatorial optimization, the efficient optimization over large collections of discrete structures. All of them have reasonably efficient algorithms, slower than for the earlier problems, with roughly $O(mn)$ or $O(n^3)$ as a typical bound for a smart algorithm. These algorithms rest on the same principles as before: optimality conditions, progress measures, and well-chosen invariants.

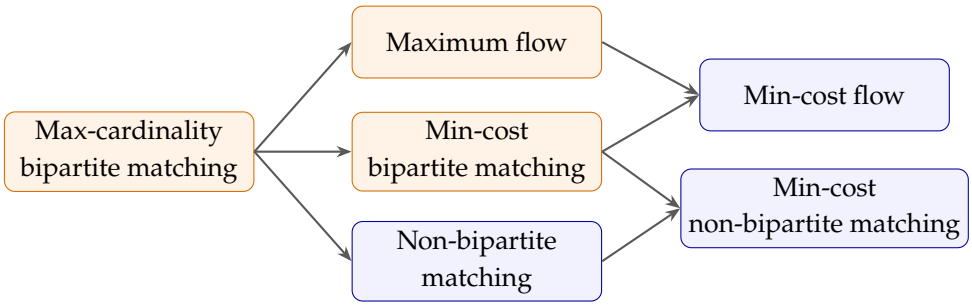


Figure 15: Six fundamental problems. Each arrow points from a problem to a generalization of it. The three problems in orange were treated in Lectures 1 to 5.

6.1 The minimum-cost flow problem

An instance of the minimum-cost flow problem consists of a directed graph $G = (V, E)$, a source s and sink t, a target flow value d, a nonnegative capacity u_e for each edge, and a real-valued cost c_e for each edge. The goal is a flow f of value d, satisfying the usual conservation and capacity constraints, that minimizes

$$\sum_{e \in E} c_e f_e.$$

Thus c_e is a per-unit cost. If no flow of value d exists, an algorithm should report this, which one maximum flow computation detects. Equivalent variants ask for a maximum flow of minimum cost, or for a minimum-cost circulation, a flow satisfying conservation at every vertex. With nonnegative costs the all-zero circulation is optimal, but with negative cycles this is nontrivial.

The problem has three earlier problems as special cases.

1. *Shortest paths.* Set $d = 1$ and $u_e = 1$ for every edge. An integral flow of value 1 is an s–t path together with possibly some cycles, so an integral minimum-cost flow yields a shortest s–t

path (assuming no negative cycles, so that shortest paths are defined).

2. *Maximum flow.* Set all costs to 0 and binary search for the largest d for which a feasible flow exists.

3. *Minimum-cost perfect bipartite matching.* Use the network from the reduction of bipartite matching to maximum flow (Lecture 4), carry the edge costs over to the V–W edges, and set $d = n$. An integral flow of value n saturates every edge out of s and into t, so it corresponds to a perfect matching of the same cost.

Minimum-cost flows have optimality conditions like those of earlier problems. In the residual network, a reverse edge (w, v) corresponding to $(v, w) \in E$ gets cost $c_{wv} = -c_{vw}$, since it undoes flow. A flow of value d is minimum-cost if and only if its residual network has no negative cycle. This suggests a CYCLE-CANCELING algorithm analogous to Ford–Fulkerson, and the ideas used for maximum flow (blocking flows, push-relabel, scaling) give polynomial-time algorithms, some running in roughly $O(mn)$ time and fast in practice. A problem that reduces to minimum-cost flow is therefore in good shape.

6.2 Maximum-cardinality non-bipartite matching

In the general MATCHING problem the input is an undirected graph $G = (V, E)$, not necessarily bipartite, and the goal is a matching of maximum cardinality. The simplest non-bipartite graphs are odd cycles. A priori it is not obvious that this problem is polynomial-time solvable rather than NP-hard, and there is no natural reduction to maximum flow.

For example, the graph in Figure 16 has 16 vertices, so a matching has at most 8 edges, and a matching of size 6 is easy to find. No matching is larger. In each of the five triangles at most two vertices can be matched to each other, so at least five vertices, one per triangle, can be matched only to the center vertex 4. The center takes at most one of them, so every matching leaves at least four vertices unmatched, and hence has at most 6 edges.

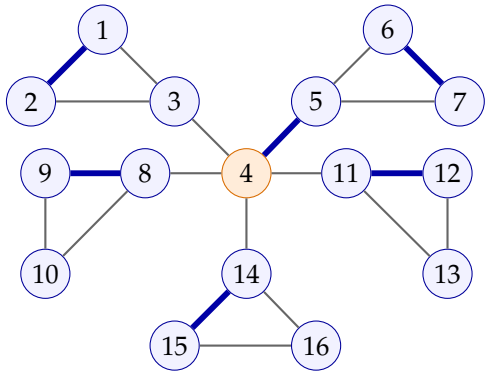


Figure 16: A 16-vertex graph: five triangles attached to the center vertex 4. The thick edges form a matching of size 6. Taking $S = \{4\}$ (orange), the graph $G \setminus S$ consists of five odd components, so $\frac{1}{2}(16 - (5 - 1)) = 6$ and the matching is maximum.

For $S \subseteq V$, let $\mathrm{oc}(S)$ be the number of odd-size connected components of $G \setminus S$, the graph left after deleting the vertices of S.

Lemma 6.1 [Odd-component bound]. *In every graph $G = (V, E)$, the maximum cardinality of a matching is at most*

$$\min_{S \subseteq V} \frac{1}{2}\big[\, |V| - (\mathrm{oc}(S) - |S|) \,\big]. \tag{6.1}$$

Proof. Fix $S \subseteq V$ and a matching. Every odd component C of $G \setminus S$ has a vertex not matched to another vertex of C. Such a vertex can be matched only to a vertex of S, since no edge joins two different components of $G \setminus S$. At most $|S|$ of these $\mathrm{oc}(S)$ vertices are matched, so at least $\mathrm{oc}(S) - |S|$ vertices are unmatched. Hence the matching covers at most $|V| - (\mathrm{oc}(S) - |S|)$ vertices and has at most half that many edges. Minimize over S. □

The lemma is the analog of the fact that every s–t cut bounds every flow: a matching of size equal to (6.1) is certainly maximum. In fact these parity obstructions are the only ones.

Theorem 6.2 [Tutte–Berge formula]. *In Lemma 6.1, equality always holds:*

$$\textit{maximum matching size} = \min_{S \subseteq V} \frac{1}{2}\big[\, |V| - (\mathrm{oc}(S) - |S|) \,\big].$$

The original proof is by induction and does not lead to an efficient algorithm. In 1965 Edmonds gave the first polynomial-time algorithm for maximum-cardinality matching. Since it always outputs a matching whose size equals (6.1), it also proves the Tutte–Berge formula.

The key difficulty is searching for a good path to enlarge the current matching. In the Hungarian algorithm (Lecture 5), bipartiteness ensures that the two endpoints of a matched edge never appear in the same level of the search tree. In a non-bipartite graph this can happen, already in a triangle. Edmonds called the resulting odd cycles blossoms, and his blossom algorithm shrinks (contracts) each blossom when it is found into a super-vertex of a smaller graph, and continues the search. At the end, the blossoms are expanded in reverse order.

6.3 Minimum-cost non-bipartite matching

Matching with both complications, a general graph and edge costs, is also polynomial-time solvable, again first shown by Edmonds. The idea is to combine blossom shrinking, which handles non-bipartiteness, with the vertex prices of the Hungarian algorithm, which handle costs. This takes care: it is unclear what price a new super-vertex should receive, and super-vertices may need to be expanded mid-algorithm.

Although in P, this problem is relatively hard: state-of-the-art algorithms handle graphs with hundreds of vertices, but thousands of vertices are already a challenge. Applications often involve

far larger graphs, which motivates fast heuristics that need not be exactly correct. For convenience, use the equivalent maximum-weight version, where edge e has weight w_e and the goal is a matching of maximum total weight.

Algorithm 6.1: Greedy matching

Input: Graph $G = (V, E)$ with edge weights w_e.
Output: A matching M.

```
Sort and rename the edges E = {1, 2, ..., m} so that w_1 ≥ w_2 ≥ ··· ≥ w_m
M ← ∅
for i ← 1 to m do
    if edge i shares no endpoint with the edges of M then
        Add edge i to M
return M
```

On the path of Figure 17, greedy picks only the middle edge, of weight $1 + \epsilon$, while the optimal matching has weight 2. So greedy can lose about half of the optimal weight, and no example is worse: on every graph, greedy achieves at least 50% of the maximum weight. More sophisticated algorithms compute a $(1-\epsilon)$-approximation in $O(m)$ time for every constant $\epsilon > 0$, with the constant depending on $1/\epsilon$.

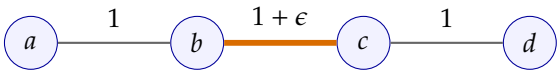


Figure 17: Greedy picks the thick edge (b, c), of weight $1 + \epsilon$; the maximum-weight matching is $\{(a, b), (c, d)\}$, of weight 2.

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 6: Generalizations of Maximum Flow and Bipartite Matching.

Lecture 7: Linear Programming: Introduction and Applications

Linear programming is a sweet spot between generality and computational efficiency. Every problem of the previous lectures is a special case, as are many others, yet linear programs can be solved efficiently both in theory (in polynomial time) and in practice (with millions of variables and constraints). Linear programs are also useful subroutines, for instance inside integer programming solvers and approximation algorithms. Conceptually, linear programming duality explains optimality conditions such as the max-flow/min-cut theorem and Hall's theorem, and is the general answer to the question of how to certify that an algorithm is done.

7.1 Linear programs

A system of linear equations $\sum_{j=1}^{n} a_{ij}x_j = b_i$, $i = 1, \ldots, m$, can be solved efficiently, for example by Gaussian elimination, meaning that one finds a feasible solution or correctly reports that none exists. Many problems also need inequalities: in maximum flow, conservation constraints are equations, but capacity and nonnegativity constraints are inequalities. Linear programming solves systems of linear equations and inequalities, and among multiple feasible solutions it finds the best.

A linear program consists of

1. decision variables $x_1, \ldots, x_n \in \mathbb{R}$;
2. linear constraints, each of the form $\sum_{j=1}^{n} a_{ij}x_j \ (*)\ b_i$, where $(*)$ is $\leq$, $\geq$, or $=$;
3. a linear objective function, $\max \sum_{j=1}^{n} c_j x_j$ or $\min \sum_{j=1}^{n} c_j x_j$.

The a_{ij}, b_i, c_j are constants that are part of the input; the x_j are free, and the algorithm chooses their values. Multiplying by -1 turns a $\geq$ constraint into a $\leq$ constraint and a minimization into a maximization, and an equality is equivalent to two inequalities, so these variants add no generality. Terms like x_j^2, $x_j x_k$, or $\log(1 + x_j)$ are not allowed: each variable appears alone, multiplied by a constant, in a sum of such terms.

7.2 Geometric intuition

Consider two variables, so that solutions are points (x_1, x_2) in the plane, and the linear program

$$\max\ x_1 + x_2 \quad \text{subject to} \quad x_1 \geq 0, \quad x_2 \geq 0, \quad 2x_1 + x_2 \leq 1, \quad x_1 + 2x_2 \leq 1.$$

The first two constraints restrict to the nonnegative quadrant, and the other two cut out the shaded region in Figure 18. The objective asks for the feasible point farthest in the direction $(1, 1)$. Its level

SETS, the sets of points with equal objective value, are parallel lines running northwest to southeast, and the optimum is the last point of the feasible region met by a level set swept from southwest to northeast: the vertex $(\frac{1}{3}, \frac{1}{3})$, with optimal value $\frac{2}{3}$.

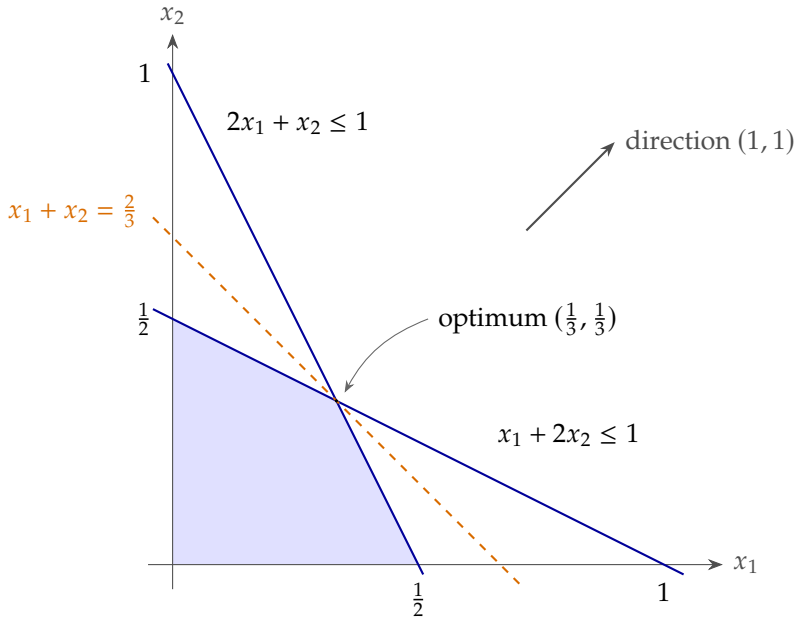


Figure 18: The feasible region of the toy linear program (shaded) and the objective direction $(1, 1)$. The dashed level set $x_1 + x_2 = \frac{2}{3}$ last touches the region at the optimal vertex $(\frac{1}{3}, \frac{1}{3})$.

The same picture holds in n dimensions.

1. A linear constraint defines a halfspace in $\mathbb{R}^n$, so the feasible region is an intersection of halfspaces, the analog of a polygon. A finite intersection of halfspaces is a POLYHEDRON; a bounded one is a POLYTOPE.
2. The level sets of the objective are parallel $(n-1)$-dimensional hyperplanes, each orthogonal to the objective vector c.
3. The optimal solution is the feasible point farthest in the direction of c (for maximization) or $-c$ (for minimization), the last point of the feasible region met by a level set.
4. A unique optimal solution is a vertex (corner) of the feasible region.

There are edge cases, which linear programming algorithms detect.

1. There may be no feasible solution, as when $x_1 + x_2 \ge 1$ is added to the toy example.
2. The optimal value may be unbounded ($+\infty$ for maximization, $-\infty$ for minimization), as when $2x_1 + x_2 \le 1$ and $x_1 + 2x_2 \le 1$ are dropped. An unbounded feasible region is necessary but not sufficient for this.

3. The optimal solution need not be unique, since a side of the feasible region may be parallel to the level sets. If the feasible region is bounded, some optimal solution is a vertex. (Without boundedness this can fail: the program $\max x_1 + x_2$ subject to $x_1 + x_2 = 1$ has no vertex.)

7.3 Flow problems as linear programs

For maximum flow, the decision variables are the edge flows $\{f_e\}_{e \in E}$. With $\delta^-(v)$ and $\delta^+(v)$ the edges into and out of v, the linear program is

$$\max \sum_{e \in \delta^+(s)} f_e \quad \text{subject to} \quad \begin{cases} \sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0 & \text{for all } v \neq s, t, \\ f_e \leq u_e & \text{for all } e \in E, \\ f_e \geq 0 & \text{for all } e \in E. \end{cases}$$

The nonnegativity constraints are needed because decision variables may be negative by default. All $2m + n - 2$ constraints are linear, since each f_e appears alone with coefficient ± 1.

For minimum-cost flow, change the objective to $\min \sum_{e \in E} c_e f_e$, which is linear because the c_e are constants, and add the constraint $\sum_{e \in \delta^+(s)} f_e = d$ for the target value d; the analogous constraint at t is implied by the others. Lower bounds ℓ_e on edge flows are equally easy: replace $f_e \geq 0$ by $f_e \geq \ell_e$. Specialized flow algorithms are generally faster, but linear programming gives these generalizations immediately.

7.4 Fitting a line

The input is m data points $\mathbf{p}^1, \ldots, \mathbf{p}^m \in \mathbb{R}^d$, each with d real-valued features, and a real label ℓ^i for each point $\mathbf{p}^i$. The goal is a linear function $h(\mathbf{z}) = \sum_{j=1}^{d} a_j z_j + b$ with $h(\mathbf{p}^i) \approx \ell^i$ for every i, used either to predict labels of new points or to understand the relation between features and labels. The decision variables are $a_1, \ldots, a_d, b$. The error on point i is

$$E_i(\mathbf{a}, b) = \left| \left(\sum_{j=1}^{d} a_j p_j^i + b \right) - \ell^i \right|,$$

the vertical distance from $(\mathbf{p}^i, \ell^i)$ to the line when $d = 1$. Here the objective is to minimize the total error

$$\min_{\mathbf{a}, b} \sum_{i=1}^{m} E_i(\mathbf{a}, b).$$

The more common objective is the squared error $\sum_i E_i(\mathbf{a}, b)^2$. The sum of errors is more robust to outliers, since squaring the error of a stray point, such as a faulty sensor reading, gives it more influence on the chosen line.

The absolute value makes this nonlinear, but it can be linearized. Since $|x| = \max\{x, -x\}$, introduce a variable e_i for each point, with the two constraints

$$e_i \geq \left(\sum_{j=1}^{d} a_j p_j^i + b\right) - \ell^i \qquad \text{and} \qquad e_i \geq -\left[\left(\sum_{j=1}^{d} a_j p_j^i + b\right) - \ell^i\right],$$

and minimize $\sum_{i=1}^{m} e_i$ over $e_1, \ldots, e_m, a_1, \ldots, a_d, b$. This is a linear program. Feasibility gives $e_i \geq E_i(\mathbf{a}, b)$, and at an optimal solution $e_i = E_i(\mathbf{a}, b)$ for every i, since otherwise decreasing e_i keeps both constraints satisfied and lowers the objective. So an optimal solution gives a line minimizing the total error.

7.5 Computing a linear classifier

Now the input is m POSITIVE points $\mathbf{p}^1, \ldots, \mathbf{p}^m \in \mathbb{R}^d$ and m' NEGATIVE points $\mathbf{q}^1, \ldots, \mathbf{q}^{m'} \in \mathbb{R}^d$ (labels 1 and 0), for example images with and without a cat. The goal is a LINEAR CLASSIFIER: a linear function $h(\mathbf{z}) = \sum_{j=1}^{d} a_j z_j + b$ with

$$h(\mathbf{p}^i) > 0 \text{ for all positive points} \qquad \text{and} \qquad h(\mathbf{q}^i) < 0 \text{ for all negative points.}$$

Geometrically, this is a hyperplane in $\mathbb{R}^d$ with normal vector $\mathbf{a}$ and shift b that has all positive points on one side and all negative points on the other (Figure 19, left). New points are classified by their side. If no such hyperplane exists, the algorithm should report this.

Strict inequalities are not allowed in linear programs, so add a decision variable δ for the margin and solve

$$\max\ \delta \quad \text{subject to} \quad \begin{cases} \sum_{j=1}^{d} a_j p_j^i + b - \delta \geq 0 & \text{for all positive points } \mathbf{p}^i, \\ \sum_{j=1}^{d} a_j q_j^i + b + \delta \leq 0 & \text{for all negative points } \mathbf{q}^i, \end{cases}$$

over $\delta, a_1, \ldots, a_d, b$. If the optimal value is strictly positive (it is then in fact unbounded, by scaling $\mathbf{a}$, b, and δ), the corresponding $\mathbf{a}, b$ give a separating hyperplane. Otherwise none exists.

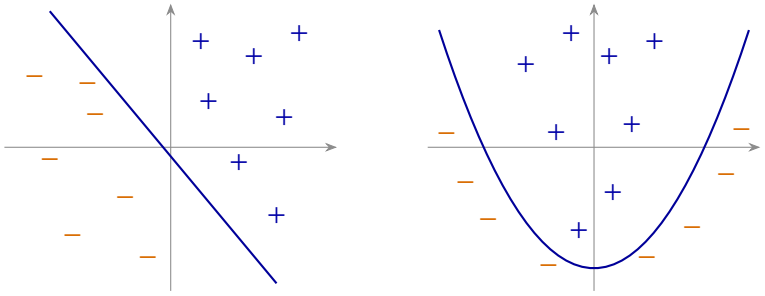


Figure 19: Schematic data, not real measurements. Left: positive (+) and negative (−) points separated by a line. Right: points that no line separates but a quadratic curve does.

7.6 Non-separable data

Real data is noisy and usually has no separating hyperplane. Two extensions handle this, and in practice they are often combined.

Minimizing hinge loss. Minimizing the number of misclassified points is NP-hard, so one uses a more tractable error. Ideally, $h(\mathbf{p}^i) \geq 1$ for positive points and $h(\mathbf{q}^i) \leq -1$ for negative points, the constant 1 fixing a normalization. The HINGE LOSS of h on a point is the amount by which its inequality fails: $\max\{1 - h(\mathbf{p}^i), 0\}$ for a positive point and $\max\{1 + h(\mathbf{q}^i), 0\}$ for a negative point. Taking the maximum with 0 gives no reward for classifying a point extra correctly. Each loss is a maximum of two linear functions, so the trick for fitting a line applies: with one variable e_i per point,

$$\min \sum_i e_i \quad \text{subject to} \quad \begin{cases} e_i \geq 1 - \left(\sum_{j=1}^d a_j p_j^i + b\right) & \text{for every positive point } \mathbf{p}^i, \\ e_i \geq 1 + \left(\sum_{j=1}^d a_j q_j^i + b\right) & \text{for every negative point } \mathbf{q}^i, \\ e_i \geq 0 & \text{for every point,} \end{cases}$$

over $a_1, \ldots, a_d, b$ and the $m + m'$ variables e_i.

Increasing the dimension. Alternatively, allow nonlinear boundaries (Figure 19, right) by generating extra features. For some $d' \geq d$ and map $\varphi : \mathbb{R}^d \to \mathbb{R}^{d'}$, replace each $\mathbf{p}^i$ by $\varphi(\mathbf{p}^i)$ and each $\mathbf{q}^i$ by $\varphi(\mathbf{q}^i)$, and separate the images by a hyperplane in $\mathbb{R}^{d'}$. An example adds all pairwise products of features,

$$(z_1, \ldots, z_d) \mapsto (z_1, \ldots, z_d,\ z_1^2, \ldots, z_d^2,\ z_1 z_2, z_1 z_3, \ldots, z_{d-1} z_d),$$

which is interesting even for $d = 1$, where it is $z \mapsto (z, z^2)$. The goal is now coefficients $a_1, \ldots, a_{d'}$ and b with $\sum_{j=1}^{d'} a_j\, \varphi(\mathbf{p}^i)_j + b > 0$ for positive points and $\sum_{j=1}^{d'} a_j\, \varphi(\mathbf{q}^i)_j + b < 0$ for negative points. If the new features include the original ones, every original hyperplane remains available (set $a_{d+1} = \cdots = a_{d'} = 0$), and there are many new options. For $d = 1$ and $z \mapsto (z, z^2)$, a linear function in the expanded space is $a_2 z^2 + a_1 z + b$, a quadratic in the original space. Applying φ as preprocessing reduces the problem to the classifier linear program above, now with $d' + 2$ variables $\delta, a_1, \ldots, a_{d'}, b$. This is the basic idea behind support vector machines.

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 7: Linear Programming: Introduction and Applications.

Lecture 8: Linear Programming Duality (Part 1)

8.1 A warm-up example

Consider the linear program

$$\max\ x_1 + x_2 \quad \text{subject to} \quad 4x_1 + x_2 \le 2, \quad x_1 + 2x_2 \le 1, \quad x_1 \ge 0, \quad x_2 \ge 0. \tag{8.1}$$

The most northeastern feasible point is the vertex $(\frac{3}{7}, \frac{2}{7})$, where both main constraints are tight, so the optimal value is $\frac{5}{7}$ (Figure 20). Beyond a few dimensions one cannot solve linear programs by inspection. Even given an allegedly optimal solution, how can one be convinced that it is optimal?

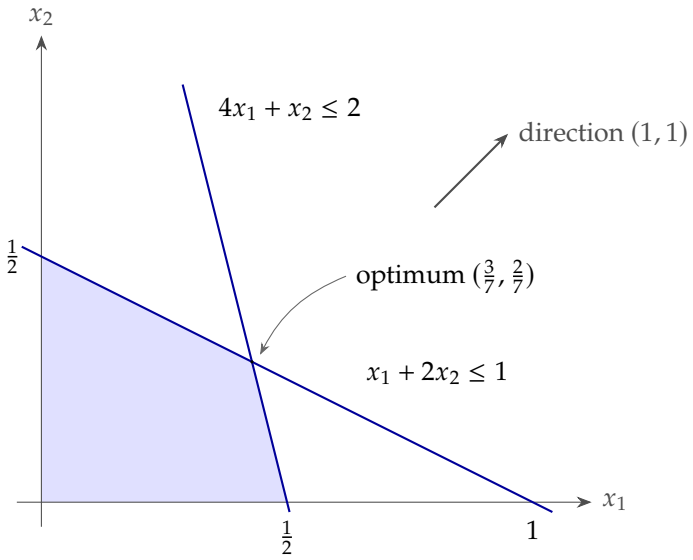


Figure 20: The feasible region of (8.1). The optimal vertex $(\frac{3}{7}, \frac{2}{7})$ lies on both lines $4x_1 + x_2 = 2$ and $x_1 + 2x_2 = 1$.

A feasible point proves that the optimum is at least $\frac{5}{7}$. For an upper bound, every feasible point satisfies

$$x_1 + x_2 \le 4x_1 + x_2 \le 2,$$

using $x_1 \ge 0$ and then the first constraint, so the optimum is at most 2. The second constraint does better: $x_1 + x_2 \le x_1 + 2x_2 \le 1$. Blending the constraints is allowed too, and the best blend takes $\frac{1}{7}$ of the first and $\frac{3}{7}$ of the second:

$$x_1 + x_2 \le \tfrac{1}{7}(4x_1 + x_2) + \tfrac{3}{7}(x_1 + 2x_2) \le \tfrac{1}{7} \cdot 2 + \tfrac{3}{7} \cdot 1 = \tfrac{5}{7}.$$

(The first inequality is in fact an equality.) Together with the feasible point $(\frac{3}{7}, \frac{2}{7})$, this proves that $\frac{5}{7}$ is the optimal value. Such matching bound proofs are the essence of linear programming duality.

8.2 The dual linear program

Consider a linear program (P) of the form

$$\max \sum_{j=1}^{n} c_j x_j \quad \text{subject to} \quad \sum_{j=1}^{n} a_{ij} x_j \le b_i \ \ (i = 1, \dots, m), \qquad x_1, \dots, x_n \ge 0,$$

or in matrix-vector notation, with $\mathbf{c}, \mathbf{x} \in \mathbb{R}^n$, $\mathbf{b} \in \mathbb{R}^m$, $\mathbf{A}$ the $m \times n$ matrix of the a_{ij}, and componentwise inequalities,

$$\max \ \mathbf{c}^\top \mathbf{x} \quad \text{subject to} \quad \mathbf{A}\mathbf{x} \le \mathbf{b}, \quad \mathbf{x} \ge 0.$$

Other forms reduce to this one: negate a minimization objective, replace an equality by two inequalities, negate a $\ge$ constraint, write a free variable as the difference of two nonnegative ones, and turn an inequality into an equality with an extra slack variable.

As in the example, take a nonnegative linear combination of the constraints, with multipliers $y_1, \dots, y_m \ge 0$, that dominates the objective componentwise:

$$\sum_{i=1}^{m} y_i a_{ij} \ge c_j \qquad \text{for } j = 1, \dots, n, \tag{8.2}$$

that is, $\mathbf{A}^\top \mathbf{y} \ge \mathbf{c}$. Every such $\mathbf{y}$ bounds the optimal value: for every feasible $\mathbf{x}$,

$$\sum_{j=1}^{n} c_j x_j \le \sum_{j=1}^{n} \Big(\sum_{i=1}^{m} y_i a_{ij} \Big) x_j = \sum_{i=1}^{m} y_i \Big(\sum_{j=1}^{n} a_{ij} x_j \Big) \le \sum_{i=1}^{m} y_i b_i .$$

The first inequality uses (8.2) and $\mathbf{x} \ge 0$, the equality reverses the order of summation, and the last inequality uses the feasibility of $\mathbf{x}$ and $\mathbf{y} \ge 0$. In matrix form, $\mathbf{c}^\top \mathbf{x} \le (\mathbf{A}^\top \mathbf{y})^\top \mathbf{x} = \mathbf{y}^\top (\mathbf{A}\mathbf{x}) \le \mathbf{y}^\top \mathbf{b}$.

The tightest such bound is itself the optimal value of a linear program, the DUAL (D) of (P):

$$\min \ \mathbf{b}^\top \mathbf{y} \quad \text{subject to} \quad \mathbf{A}^\top \mathbf{y} \ge \mathbf{c}, \quad \mathbf{y} \ge 0,$$

that is, $\min \sum_{i=1}^{m} b_i y_i$ subject to $\sum_{i=1}^{m} a_{ij} y_i \ge c_j$ for $j = 1, \dots, n$ and $y_1, \dots, y_m \ge 0$. To take the dual, swap the objective and the right-hand side and transpose the constraint matrix. For the warm-up example the dual is

$$\min \ 2y_1 + y_2 \quad \text{subject to} \quad 4y_1 + y_2 \ge 1, \quad y_1 + 2y_2 \ge 1, \quad y_1, y_2 \ge 0,$$

and its feasible solutions $(1, 0)$, $(0, 1)$, and $(\frac{1}{7}, \frac{3}{7})$, of values 2, 1, and $\frac{5}{7}$, are the three upper bounds found above.

Theorem 8.1 [WEAK DUALITY]. *For every linear program (P) of the above form and its dual (D),*

$$\textit{OPT value of (P)} \le \textit{OPT value of (D)}.$$

Proof. The derivation above shows $\mathbf{c}^\top \mathbf{x} \le \mathbf{b}^\top \mathbf{y}$ for every pair of feasible solutions $\mathbf{x}$ of (P) and $\mathbf{y}$ of (D), in particular for optimal ones. □

Strong duality, proved in the next lecture, says that equality holds whenever both (P) and (D) are feasible.

8.3 Max-flow/min-cut revisited

It is easiest to take the dual of a larger but simpler formulation of maximum flow, using paths instead of edge flows. Let $\mathcal{P}$ be the set of s–t paths, with a variable f_P for each $P \in \mathcal{P}$. No conservation constraints are needed:

$$\max \sum_{P \in \mathcal{P}} f_P \quad \text{subject to} \quad \sum_{P \in \mathcal{P}:\, e \in P} f_P \le u_e \ \text{ for all } e \in E, \qquad f_P \ge 0 \ \text{ for all } P \in \mathcal{P}.$$

Call this (P); the left side of each capacity constraint is the total flow on e. Its optimal value equals that of the edge formulation of Lecture 7: setting f_e to the total flow on e turns a path solution into an edge flow of the same value, and a path decomposition of an edge flow gives the reverse direction.

In matrix form (P) is $\max \mathbf{1}^\top \mathbf{f}$ subject to $\mathbf{A}\mathbf{f} \le \mathbf{u}$ and $\mathbf{f} \ge 0$, where $\mathbf{f}$ is indexed by $\mathcal{P}$, $\mathbf{u}$ by E, and $\mathbf{A}$ is the $E \times \mathcal{P}$ matrix with $a_{eP} = 1$ if $e \in P$ and 0 otherwise. The dual has a variable ℓ_e for each edge and is $\min \mathbf{u}^\top \boldsymbol{\ell}$ subject to $\mathbf{A}^\top \boldsymbol{\ell} \ge \mathbf{1}$ and $\boldsymbol{\ell} \ge 0$. The row of $\mathbf{A}^\top$ for a path P has a 1 exactly in the columns of the edges of P, so the dual (D) is

$$\min \sum_{e \in E} u_e \ell_e \quad \text{subject to} \quad \sum_{e \in P} \ell_e \ge 1 \ \text{ for all } P \in \mathcal{P}, \qquad \ell_e \ge 0 \ \text{ for all } e \in E. \tag{8.3}$$

Interpretation. Every s–t cut (A, B), with $s \in A$ and $t \in B$, gives a feasible dual solution: set $\ell_e = 1$ for $e \in \delta^+(A)$, the edges with tail in A and head in B, and $\ell_e = 0$ otherwise (Figure 21). Every s–t path starts in A and ends in B, so it uses an edge of $\delta^+(A)$ and the constraints hold. The objective value is $\sum_{e \in E} u_e \ell_e = \sum_{e \in \delta^+(A)} u_e$, the capacity of (A, B). Not every feasible solution is 0–1: any nonnegative edge lengths ℓ_e under which the shortest s–t path has length at least 1 are feasible. For example, in the graph $s \to v \to t$ the solution $\ell_{sv} = \ell_{vt} = \frac{1}{2}$ is feasible, and it is optimal if both edges have the same capacity.

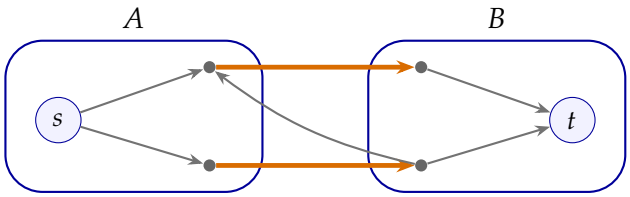


Figure 21: Schematic s–t cut (A, B). The two thick edges from A to B form $\delta^+(A)$ and receive $\ell_e = 1$; edges inside A or B and edges from B to A receive $\ell_e = 0$.

Since the feasible region of (D) contains all these cut solutions,

$$\text{max flow value} = \text{OPT of (P)} \le \text{OPT of (D)} \le \text{min cut value}, \tag{8.4}$$

where the first inequality is weak duality. By the max-flow/min-cut theorem (Lecture 2) the two ends are equal, so both inequalities are equalities. Thus OPT of (P) = OPT of (D), a special case of strong duality; and the dual (8.3), although it allows fractional solutions, always has an optimal solution with every variable 0 or 1.

8.4 A recipe for taking duals

One can take the dual of any linear program by converting it to the form of (P), but it is more convenient to dualize directly. Dual variables correspond to primal constraints, dual constraints to primal variables, maximization and minimization are exchanged, the objective and right-hand side are exchanged, and the constraint matrix is transposed. The constraint and sign types correspond as follows, for a maximization primal.

Primal (maximization)	Dual (minimization)
variables $x_1, \dots, x_n$	n constraints
m constraints	variables $y_1, \dots, y_m$
objective max $\mathbf{c}^\top \mathbf{x}$	right-hand side $\mathbf{c}$
right-hand side $\mathbf{b}$	objective min $\mathbf{b}^\top \mathbf{y}$
constraint matrix $\mathbf{A}$	constraint matrix $\mathbf{A}^\top$
ith constraint is $\le$	$y_i \ge 0$
ith constraint is $\ge$	$y_i \le 0$
ith constraint is $=$	$y_i \in \mathbb{R}$
$x_j \ge 0$	jth constraint is $\ge$
$x_j \le 0$	jth constraint is $\le$
$x_j \in \mathbb{R}$	jth constraint is $=$

For a minimization primal, the dual is defined by reading the table from right to left. By definition, the dual of the dual is the primal.

8.5 Weak duality

In every form, feasible dual solutions are bounds on the primal optimum obtained from linear combinations of the constraints, so weak duality holds in all cases.

Theorem 8.2 [Weak duality]. *For every maximization linear program (P) and its dual (D), OPT value of (P) ≤ OPT value of (D). For every minimization linear program (P) and its dual (D), OPT value of (P) ≥ OPT value of (D).*

Proof. For a maximization primal with feasible $\mathbf{x}$ and dual-feasible $\mathbf{y}$, write $\mathbf{a}_i$ for the ith row of $\mathbf{A}$. Then

$$\mathbf{b}^\top \mathbf{y} - \mathbf{c}^\top \mathbf{x} = \sum_{i=1}^{m} y_i \big(b_i - \mathbf{a}_i \mathbf{x}\big) + \sum_{j=1}^{n} \big((\mathbf{A}^\top \mathbf{y})_j - c_j\big) x_j.$$

By the table, each term is nonnegative: y_i has the sign of $b_i - \mathbf{a}_i \mathbf{x}$ (both ≥ 0 for a $\leq$ constraint, both ≤ 0 for a $\geq$ constraint, and $b_i - \mathbf{a}_i \mathbf{x} = 0$ for an equality), and similarly x_j has the sign of $(\mathbf{A}^\top \mathbf{y})_j - c_j$. The minimization case is the same statement with the roles of (P) and (D) exchanged. □

Picture the objective values of primal feasible solutions and of dual feasible solutions on a line (Figure 22): for a maximization primal, every primal value lies to the left of every dual value.



Figure 22: Weak duality for a maximization primal: objective values of primal feasible solutions (crosses) never exceed those of dual feasible solutions (circles). Schematic.

Corollary 8.3 [Consequences of weak duality]. *Let (P), (D) be a primal-dual pair of linear programs.*

(a) If the optimal value of (P) is unbounded, then (D) is infeasible.

(b) If the optimal value of (D) is unbounded, then (P) is infeasible.

(c) If $\mathbf{x}$, $\mathbf{y}$ are feasible for (P), (D) and $\mathbf{c}^\top \mathbf{x} = \mathbf{b}^\top \mathbf{y}$, then $\mathbf{x}$ and $\mathbf{y}$ are both optimal.

Proof. For (a) and (b), any feasible solution of one program bounds the optimal value of the other, so if no bound exists there is no feasible solution. For (c), every primal value is at most $\mathbf{b}^\top \mathbf{y} = \mathbf{c}^\top \mathbf{x}$ and every dual value is at least $\mathbf{c}^\top \mathbf{x} = \mathbf{b}^\top \mathbf{y}$, so neither solution can be improved. □

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 8: Linear Programming Duality (Part 1).

Lecture 9: Linear Programming Duality (Part 2)

9.1 Recap: primal, dual, and weak duality

Throughout, the primal linear program (P) and its dual (D) are

$$\text{(P)} \quad \begin{aligned} &\max \mathbf{c}^\top \mathbf{x} \\ &\text{s.t. } \mathbf{A}\mathbf{x} \le \mathbf{b}, \;\; \mathbf{x} \ge 0, \end{aligned} \qquad\qquad \text{(D)} \quad \begin{aligned} &\min \mathbf{y}^\top \mathbf{b} \\ &\text{s.t. } \mathbf{A}^\top \mathbf{y} \ge \mathbf{c}, \;\; \mathbf{y} \ge 0, \end{aligned}$$

where $\mathbf{c}, \mathbf{x} \in \mathbb{R}^n$, $\mathbf{b}, \mathbf{y} \in \mathbb{R}^m$, $\mathbf{A} = (a_{ij})$ is $m \times n$, and inequalities are componentwise. Expanded, (P) maximizes $\sum_{j=1}^n c_j x_j$ subject to $\sum_{j=1}^n a_{ij} x_j \le b_i$ for $i = 1, \dots, m$, and (D) minimizes $\sum_{i=1}^m b_i y_i$ subject to $\sum_{i=1}^m a_{ij} y_i \ge c_j$ for $j = 1, \dots, n$, with all variables nonnegative.

A nonnegative combination $\mathbf{y}$ of the primal constraints whose left-hand side dominates the objective $\mathbf{c}$ proves the upper bound $\mathbf{y}^\top \mathbf{b}$ on the primal optimum, and the dual is the problem of finding the tightest such bound. The recipe for any form of LP: one dual variable per primal constraint and one dual constraint per primal variable, max and min interchanged, objective and right-hand side interchanged, and the constraint matrix transposed (with the usual rules for equality constraints and unrestricted variables). Weak duality then holds by construction. For feasible $\mathbf{x}, \mathbf{y}$,

$$\sum_{j=1}^n c_j x_j \le \sum_{j=1}^n \Big(\sum_{i=1}^m y_i a_{ij} \Big) x_j \tag{9.1}$$

$$= \sum_{i=1}^m y_i \Big(\sum_{j=1}^n a_{ij} x_j \Big) \tag{9.2}$$

$$\le \sum_{i=1}^m y_i b_i, \tag{9.3}$$

that is, $\mathbf{c}^\top \mathbf{x} \le (\mathbf{A}^\top \mathbf{y})^\top \mathbf{x} = \mathbf{y}^\top (\mathbf{A}\mathbf{x}) \le \mathbf{y}^\top \mathbf{b}$. The first inequality uses $\mathbf{x} \ge 0$ and $\mathbf{A}^\top \mathbf{y} \ge \mathbf{c}$, the second $\mathbf{y} \ge 0$ and $\mathbf{A}\mathbf{x} \le \mathbf{b}$. In particular, if (P) is unbounded then (D) is infeasible, and vice versa.

Corollary 9.1 [Sufficient condition for optimality]. *If $\mathbf{x}, \mathbf{y}$ are feasible for (P),(D) and $\mathbf{c}^\top \mathbf{x} = \mathbf{y}^\top \mathbf{b}$, then both are optimal.*

Every primal objective value lies to the left of every dual objective value (Figure 23), so a value attained by both must be the rightmost primal value and the leftmost dual value. Similarly, a flow and an s-t cut with the same value are a maximum flow and a minimum cut.

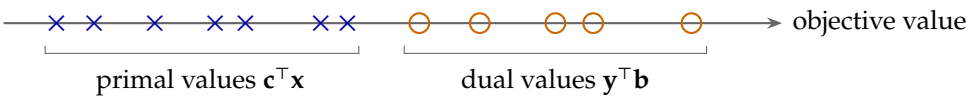


Figure 23: Weak duality, schematically: objective values of feasible primal solutions (×) all lie to the left of those of feasible dual solutions (○).

9.2 Complementary slackness

Corollary 9.2 [Complementary slackness conditions]. *Let* $\mathbf{x}, \mathbf{y}$ *be feasible for (P),(D). If*

(1) whenever $x_j \neq 0$, $\mathbf{y}$ *satisfies the* j*th constraint of (D) with equality, and*

(2) whenever $y_i \neq 0$, $\mathbf{x}$ *satisfies the* i*th constraint of (P) with equality,*

then both $\mathbf{x}$ *and* $\mathbf{y}$ *are optimal.*

The conditions forbid a decision variable and its corresponding constraint from being simultaneously slack (variable nonzero and constraint not tight).

Proof. Condition (1) gives $c_j x_j = \left(\sum_i y_i a_{ij}\right) x_j$ for every j, since either $x_j = 0$ or $c_j = \sum_i y_i a_{ij}$, so (9.1) holds with equality. Condition (2) gives $y_i\left(\sum_j a_{ij} x_j\right) = y_i b_i$ for every i, so (9.3) holds with equality. Hence $\mathbf{c}^\top\mathbf{x} = \mathbf{y}^\top\mathbf{b}$, and Corollary 9.1 applies. Other forms of primal-dual pairs are handled the same way. □

Physical interpretation. Think of the objective of (P) as a force in direction $\mathbf{c}$ pushing a particle within the feasible region until it comes to rest at $\mathbf{x}^*$, where the forces acting on it sum to zero. The walls also push: the wall of the constraint $\sum_j a_{ij} x_j \leq b_i$ exerts force in direction $-\mathbf{a}_i$, where $\mathbf{a}_i$ is the ith row of $\mathbf{A}$, and y_i is its magnitude. Balance of forces is $\mathbf{c} = \sum_i y_i \mathbf{a}_i$, and complementary slackness says $y_i > 0$ only when $\mathbf{a}_i^\top \mathbf{x}^* = b_i$: only walls that touch the particle push on it.

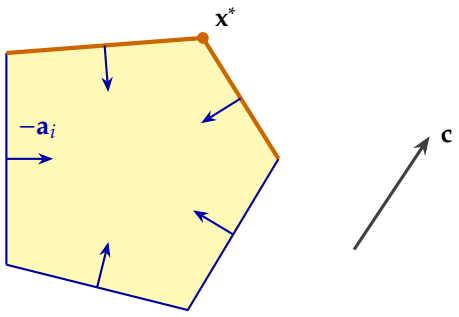


Figure 24: Physical interpretation of complementary slackness. The objective pushes the particle in direction $\mathbf{c}$ until it rests at $\mathbf{x}^*$. Each wall pushes inward along its normal $-\mathbf{a}_i$; only the two walls touching $\mathbf{x}^*$ (highlighted) may exert force, and the forces sum to zero.

A general algorithm design paradigm. Consider three conditions:

1. $\mathbf{x}$ is feasible for (P);
2. $\mathbf{y}$ is feasible for (D);
3. $\mathbf{x}, \mathbf{y}$ satisfy the complementary slackness conditions.

By Corollary 9.2, achieving all three proves both solutions optimal. Pick two of the three to maintain at all times, and work toward achieving the third. Each choice gives a disciplined way of working toward optimality, and many algorithms for linear programs and their special cases are instances of this paradigm.

9.3 The Hungarian algorithm revisited

Lecture 8 reinterpreted the max-flow/min-cut theorem through LP duality (Example 1): s-t cuts correspond to 0-1 solutions of the dual of the flow LP, with objective value equal to the cut capacity. The max-flow/min-cut theorem then shows that this primal-dual pair has no gap between the optimal values and that its dual always has an integral optimal solution. Bipartite matching (Example 2) behaves the same way.

The primal. Given a bipartite graph $G = (V \cup W, E)$ with edge costs c_e, let $\delta(v)$ denote the edges incident to v. The program (P-BM) is

$$\min \sum_{e \in E} c_e x_e \quad \text{s.t.} \quad \sum_{e \in \delta(v)} x_e = 1 \ \text{ for all } v \in V \cup W, \qquad x_e \geq 0 \ \text{ for all } e \in E.$$

The intended semantics is $x_e = 1$ if e is in the chosen perfect matching and $x_e = 0$ otherwise, but the LP may use fractional values. (The bounds $x_e \leq 1$ are implied, since $x_e \leq \sum_{f \in \delta(v)} x_f = 1$ for an endpoint v of e.) In matrix form this is $\min \mathbf{c}^\top \mathbf{x}$ subject to $\mathbf{A}\mathbf{x} = \mathbf{1}$ and $\mathbf{x} \geq 0$, where $\mathbf{A}$ is the $(V \cup W) \times E$ incidence matrix with $a_{ve} = 1$ if $e \in \delta(v)$ and $a_{ve} = 0$ otherwise.

The dual. The column of $\mathbf{A}$ for edge e, a row of $\mathbf{A}^\top$, has 1s exactly in the rows of the two endpoints of e. Since the primal constraints are equalities, the dual variables p_v, one per vertex, are unrestricted in sign. The recipe gives (D-BM): $\max \mathbf{p}^\top \mathbf{1}$ subject to $\mathbf{A}^\top \mathbf{p} \leq \mathbf{c}$ and $\mathbf{p} \in \mathbb{R}^{V \cup W}$, that is,

$$\max \sum_{v \in V \cup W} p_v \quad \text{s.t.} \quad p_v + p_w \leq c_{vw} \ \text{ for all } (v, w) \in E, \qquad p_v \in \mathbb{R} \ \text{ for all } v \in V \cup W.$$

The vertex prices of the Hungarian algorithm (Lecture 5) are exactly the decision variables of

(D-BM). Rewriting the dual constraints as

$$\underbrace{c_{vw} - p_v - p_w}_{\text{reduced cost}} \geq 0 \qquad \text{for every } (v, w) \in E \tag{9.4}$$

shows that the algorithm's first invariant, that every edge has nonnegative reduced cost, is precisely dual feasibility of $\mathbf{p}$. As for complementary slackness, condition (2) is trivial because every constraint of (P-BM) is an equation. Condition (1) says that $x_e > 0$ forces equality in (9.4), so every edge used has zero reduced cost: this is the second invariant, that edges of the current matching are tight. Thus the Hungarian algorithm maintains conditions 2 and 3 of the paradigm and works toward condition 1, a perfect matching. Algorithms of this type are called PRIMAL-DUAL ALGORITHMS, and the Hungarian algorithm is a canonical example.

Consequences. We have

$$\text{OPT of (D-BM)} \leq \text{OPT of (P-BM)} \leq \text{min-cost perfect matching.} \tag{9.5}$$

The first inequality is weak duality for a minimization primal. The second holds because every perfect matching is a 0-1 feasible solution of (P-BM), and the LP minimizes over a superset of these. The Hungarian algorithm terminates with a perfect matching (when one exists) together with feasible prices satisfying complementary slackness, so, as in the proof of Corollary 9.2, the matching's cost equals the dual objective value of the final prices. Both inequalities in (9.5) are therefore equalities. The first is another special case of strong duality; the second shows that linear programs of the form (P-BM) always have 0-1 optimal solutions.

9.4 Strong LP duality

Theorem 9.3 [STRONG LP DUALITY**].** *If a primal-dual pair (P),(D) are both feasible, then* OPT *of (P)* = OPT *of (D).*

So the simple method of bounding the primal optimum by linear combinations of constraints always yields the tightest possible bound. Strong duality generalizes the max-flow/min-cut theorem and Hall's theorem. When at least one of (P),(D) is infeasible, three cases can occur: (P) unbounded and (D) infeasible, (P) infeasible and (D) unbounded, or both infeasible.

Corollary 9.4 [LP OPTIMALITY CONDITIONS**].** *Let* $\mathbf{x}, \mathbf{y}$ *be feasible for a primal-dual pair (P),(D). Then* $\mathbf{x}, \mathbf{y}$ *are both optimal if and only if* $\mathbf{c}^\top \mathbf{x} = \mathbf{y}^\top \mathbf{b}$*, if and only if the complementary slackness conditions hold.*

Proof. For the first equivalence, Theorem 9.3 provides feasible $\mathbf{x}^*, \mathbf{y}^*$ with $\mathbf{c}^\top \mathbf{x}^* = (\mathbf{y}^*)^\top \mathbf{b}$. If $\mathbf{c}^\top \mathbf{x} \neq \mathbf{y}^\top \mathbf{b}$, then $\mathbf{c}^\top \mathbf{x} < \mathbf{c}^\top \mathbf{x}^*$ or $\mathbf{y}^\top \mathbf{b} > (\mathbf{y}^*)^\top \mathbf{b}$, so one of them is not optimal; the converse is

Corollary 9.1. The second equivalence does not need strong duality. In (9.1)–(9.3), the two gaps are $\sum_j x_j\big(\sum_i y_i a_{ij} - c_j\big)$ and $\sum_i y_i\big(b_i - \sum_j a_{ij} x_j\big)$, sums of nonnegative terms. So $\mathbf{c}^\top \mathbf{x} = \mathbf{y}^\top \mathbf{b}$ exactly when every term vanishes, which is complementary slackness. □

The proof sketch of Theorem 9.3 follows the road map

$$\underbrace{\text{separating hyperplane theorem}}_{\text{assumed}} \implies \underbrace{\text{Farkas's lemma}}_{\text{proved below}} \implies \text{strong LP duality.}$$

9.5 The separating hyperplane theorem

A set $C \subseteq \mathbb{R}^n$ is CONVEX if it contains all its chords: $\lambda \mathbf{x} + (1-\lambda)\mathbf{y} \in C$ for all $\mathbf{x}, \mathbf{y} \in C$ and $\lambda \in [0,1]$. A filled-in polygon is convex; an annulus is not. A set is CLOSED if it contains its boundary: whenever a sequence of points of C converges to $\mathbf{x}^*$, also $\mathbf{x}^* \in C$. The unit disc is closed; the open unit disc is not.

Theorem 9.5 [SEPARATING HYPERPLANE]. *Let $C \subseteq \mathbb{R}^n$ be closed and convex, and let $\mathbf{z} \in \mathbb{R}^n$ with $\mathbf{z} \notin C$. Then there is a* SEPARATING HYPERPLANE*: a coefficient vector $\boldsymbol{\alpha} \in \mathbb{R}^n$ and an intercept $\beta \in \mathbb{R}$ with*

(1) $\boldsymbol{\alpha}^\top \mathbf{x} \geq \beta$ for all $\mathbf{x} \in C$ (all of C on one side), and

(2) $\boldsymbol{\alpha}^\top \mathbf{z} < \beta$ ($\mathbf{z}$ on the other side).

The set C need not be bounded. The theorem is taken on faith here; for a formal proof, let $\mathbf{y}$ be the point of C nearest to $\mathbf{z}$ (it exists because C is closed) and take the hyperplane perpendicular to the segment from $\mathbf{y}$ to $\mathbf{z}$ through its midpoint. Convexity of C and the choice of $\mathbf{y}$ put all of C on the side opposite $\mathbf{z}$.

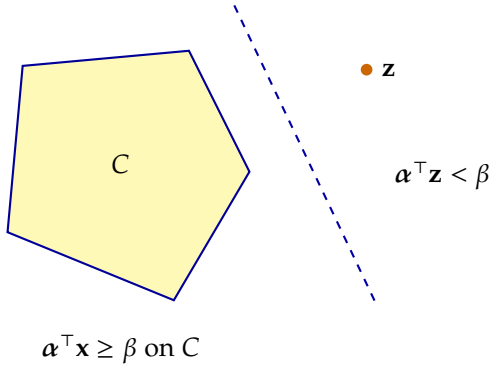


Figure 25: A closed convex set C, a point $\mathbf{z} \notin C$, and a separating hyperplane (dashed).

9.6 Farkas's lemma

Exhibiting a solution convinces anyone that a system of linear inequalities is feasible. Farkas's lemma, a feasibility version of strong duality, says how to convince someone that a system has no solution.

Theorem 9.6 [Farkas's lemma]. *Given* $\mathbf{A} \in \mathbb{R}^{m \times n}$ *and* $\mathbf{b} \in \mathbb{R}^m$*, exactly one of the following holds:*

(i) there exists $\mathbf{x} \in \mathbb{R}^n$ *with* $\mathbf{x} \geq 0$ *and* $\mathbf{A}\mathbf{x} = \mathbf{b}$*;*

(ii) there exists $\mathbf{y} \in \mathbb{R}^m$ *with* $\mathbf{y}^\top \mathbf{A} \geq 0$ *and* $\mathbf{y}^\top \mathbf{b} < 0$*.*

A solution of (ii) is a proof that the system (i) is infeasible. Analogous statements for other linear systems (for instance with inequality and nonnegativity constraints) follow from this one.

Proof. Not both. Solutions $\mathbf{x}$ and $\mathbf{y}$ would give $(\mathbf{y}^\top \mathbf{A})\mathbf{x} \geq 0$, as a product of nonnegative vectors, and yet $\mathbf{y}^\top(\mathbf{A}\mathbf{x}) = \mathbf{y}^\top \mathbf{b} < 0$.
At least one. Suppose (i) is infeasible, and let

$$Q = \{\mathbf{d} \in \mathbb{R}^m : \ \mathbf{A}\mathbf{x} = \mathbf{d} \text{ for some } \mathbf{x} \geq 0\},$$

the set of right-hand sides that are feasible with constraint matrix $\mathbf{A}$. Equivalently, Q is the set of nonnegative linear combinations of the columns of $\mathbf{A}$ (the cone they generate). It is convex, since an average of two nonnegative combinations is another, and closed, since a limit of nonnegative combinations is another. By assumption $\mathbf{b} \notin Q$, so Theorem 9.5 gives $\boldsymbol{\alpha} \in \mathbb{R}^m$ and β with $\boldsymbol{\alpha}^\top \mathbf{d} \geq \beta$ for all $\mathbf{d} \in Q$ and $\boldsymbol{\alpha}^\top \mathbf{b} < \beta$. Because Q is a cone, we may take $\beta = 0$: since $\mathbf{0} \in Q$ we have $\beta \leq 0$, so $\boldsymbol{\alpha}^\top \mathbf{b} < 0$; and if some $\mathbf{d} \in Q$ had $\boldsymbol{\alpha}^\top \mathbf{d} < 0$, then the multiples $\lambda \mathbf{d} \in Q$ would violate $\boldsymbol{\alpha}^\top(\lambda \mathbf{d}) \geq \beta$ for large λ. Thus

$$\boldsymbol{\alpha}^\top \mathbf{d} \geq 0 \ \text{ for all } \mathbf{d} \in Q, \qquad \boldsymbol{\alpha}^\top \mathbf{b} < 0. \tag{9.6}$$

Take $\mathbf{y} = \boldsymbol{\alpha}$. The second part of (9.6) is $\mathbf{y}^\top \mathbf{b} < 0$. The jth coordinate of $\boldsymbol{\alpha}^\top \mathbf{A}$ is the inner product of $\boldsymbol{\alpha}$ with the jth column $\mathbf{a}^j$ of $\mathbf{A}$, which lies in Q, so it is nonnegative by the first part. Hence $\mathbf{y}$ solves (ii). □

To finish the proof of strong duality, let $\mathrm{OPT}_{(D)}$ be the optimal value of the dual, add to the primal the constraint that its objective value is at least $\mathrm{OPT}_{(D)}$, and use Farkas's lemma to show that the augmented linear program is feasible. Strong LP duality thus reduces to the intuitive existence of a hyperplane separating a closed convex set from a point outside it.

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 9: Linear Programming Duality (Part 2).

Lecture 10: The Minimax Theorem and Algorithms for Linear Programming

10.1 Zero-sum games

In rock-paper-scissors, a player forced to move first loses to a best response. But if she only has to commit to a probability distribution over rock, paper, and scissors (after which the opponent responds and coins are flipped on her behalf), she can protect herself by randomizing uniformly: whatever the opponent does, each player is equally likely to win, lose, or tie. The minimax theorem says that in every game of pure competition, a player moving first can protect herself by randomizing appropriately.

Definition 10.1 [Zero-sum game]. A zero-sum game is specified by a real $m \times n$ matrix $\mathbf{A}$. The row player picks a row i and the column player picks a column j (rows and columns are strategies). The row player's payoff in outcome (i, j) is a_{ij} and the column player's is $-a_{ij}$: the column player pays a_{ij} to the row player (possibly negative). The row player prefers larger entries and the column player smaller ones.

	Rock	Paper	Scissors
Rock	0	−1	1
Paper	1	0	−1
Scissors	−1	1	0

Table 1: Row player's payoffs in rock-paper-scissors.

If the row player uses a mixed strategy $\mathbf{x}$ (a distribution over rows) and the column player independently uses $\mathbf{y}$, the row player's expected payoff is

$$\sum_{i=1}^{m}\sum_{j=1}^{n} \Pr[\text{outcome } (i,j)]\, a_{ij} = \sum_{i=1}^{m}\sum_{j=1}^{n} x_i y_j\, a_{ij} = \mathbf{x}^\top \mathbf{A}\mathbf{y},$$

and the column player's is $-\mathbf{x}^\top\mathbf{A}\mathbf{y}$. Intuitively there is only a first-mover disadvantage, since the second player can adapt to the first player's strategy. The minimax theorem says that under optimal play it does not matter.

Theorem 10.2 [Minimax theorem]. *For every two-player zero-sum game* $\mathbf{A}$*,*

$$\max_{\mathbf{x}}\Big(\min_{\mathbf{y}} \mathbf{x}^\top\mathbf{A}\mathbf{y}\Big) = \min_{\mathbf{y}}\Big(\max_{\mathbf{x}} \mathbf{x}^\top\mathbf{A}\mathbf{y}\Big), \tag{10.1}$$

where $\mathbf{x}$ *and* $\mathbf{y}$ *range over distributions on the rows and columns.*

On the left the row player moves first and the column player responds optimally, and the row player plays optimally anticipating this response; on the right the roles are reversed. The common value of both sides is the VALUE of the game. In rock-paper-scissors both sides are 0, with the first player playing uniformly and the second responding arbitrarily. In a game skewed toward one player, both sides are nonzero but still equal.

10.2 From LP duality to minimax

Two obstacles keep the left side of (10.1) from being a linear program: the nested max/min, and the quadratic dependence of $\mathbf{x}^\top \mathbf{A}\mathbf{y}$ on $(\mathbf{x}, \mathbf{y})$. The second goes away because the player moving second never needs to randomize: given $\mathbf{x}$, the column player can compute the expected payoff of each column and choose a best one deterministically (randomizing among tied best columns is also optimal, but unnecessary). With $\mathbf{e}_j$ the jth standard basis vector,

$$\max_{\mathbf{x}} \Big(\min_{\mathbf{y}} \mathbf{x}^\top \mathbf{A}\mathbf{y} \Big) = \max_{\mathbf{x}} \Big(\min_{j=1}^{n} \mathbf{x}^\top \mathbf{A}\mathbf{e}_j \Big) = \max_{\mathbf{x}} \Big(\min_{j=1}^{n} \sum_{i=1}^{m} a_{ij} x_i \Big). \tag{10.2}$$

The inner minimum is simulated by a new variable v, intended to equal it, just as in Lecture 7:

$$\max\ v \quad \text{s.t.} \quad v - \sum_{i=1}^{m} a_{ij} x_i \le 0 \ \ (j = 1, \dots, n), \qquad \sum_{i=1}^{m} x_i = 1, \qquad x_1, \dots, x_m \ge 0, \ \ v \in \mathbb{R}. \tag{10.3}$$

The constraints say $v \le \sum_i a_{ij} x_i$ for every j, so $v \le \min_j \sum_i a_{ij} x_i$. At an optimal solution $(v^*, \mathbf{x}^*)$ this holds with equality, since otherwise v^* could be increased slightly while remaining feasible. As the LP maximizes over all distributions $\mathbf{x}$, its optimal value is

$$v^* = \max_{\mathbf{x}} \Big(\min_{j=1}^{n} \mathbf{x}^\top \mathbf{A}\mathbf{e}_j \Big) = \max_{\mathbf{x}} \Big(\min_{\mathbf{y}} \mathbf{x}^\top \mathbf{A}\mathbf{y} \Big), \tag{10.4}$$

and $\mathbf{x}^*$ is an optimal strategy for the row player moving first. Repeating this for the column player gives

$$\min\ w \quad \text{s.t.} \quad w - \sum_{j=1}^{n} a_{ij} y_j \ge 0 \ \ (i = 1, \dots, m), \qquad \sum_{j=1}^{n} y_j = 1, \qquad y_1, \dots, y_n \ge 0, \ \ w \in \mathbb{R}, \tag{10.5}$$

with optimal value

$$w^* = \min_{\mathbf{y}} \Big(\max_{i=1}^{m} \mathbf{e}_i^\top \mathbf{A}\mathbf{y} \Big) = \min_{\mathbf{y}} \Big(\max_{\mathbf{x}} \mathbf{x}^\top \mathbf{A}\mathbf{y} \Big). \tag{10.6}$$

Proof of Theorem 10.2. The linear programs (10.3) and (10.5) are duals. Give the jth constraint of (10.3) the dual variable $y_j \geq 0$ and the equality $\sum_i x_i = 1$ the unrestricted dual variable w. The dual objective is $\min w$. The unrestricted primal variable v has objective coefficient 1 and column $(1, \ldots, 1; 0)$, giving the dual equality constraint $\sum_j y_j = 1$. Each $x_i \geq 0$ has coefficient 0 and column $(-a_{i1}, \ldots, -a_{in}; 1)$, giving $w - \sum_j a_{ij} y_j \geq 0$. This is (10.5). Both programs are feasible (any distribution, with v small or w large), so strong duality gives $v^* = w^*$, which is (10.1) by (10.4) and (10.6). □

10.3 The simplex method

Linear programs can be solved efficiently, in both theory and practice. In theory, they can be solved in worst-case polynomial time; in practice, commercial solvers routinely handle inputs with millions of variables and constraints. The algorithms used in the two cases are not necessarily the same.

The SIMPLEX METHOD, developed by Dantzig in 1947 together with the general formalism of linear programming, remains the dominant paradigm for solving linear programs.

Geometry. Ignoring edge cases (empty or unbounded feasible regions), some optimal solution lies at a VERTEX (corner) of the feasible region (Figure 26). This already gives a finite, if bad, algorithm, even though there are infinitely many feasible solutions. With n decision variables, every vertex satisfies at least n constraints with equality. Conversely, if a feasible $\mathbf{x}$ satisfies at most $n - 1$ constraints with equality, there is a line through $\mathbf{x}$ along which these constraints stay tight, and moving a little in either direction gives two feasible points whose midpoint is $\mathbf{x}$; a vertex cannot be a nontrivial convex combination of other feasible points. So: enumerate the finitely many sets of n linearly independent constraints, check whether the unique point satisfying them with equality is feasible, and remember the best feasible point found.

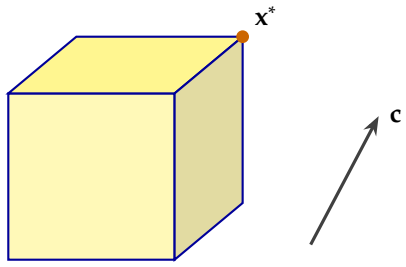


Figure 26: A feasible region and an optimal solution $\mathbf{x}^*$ at a vertex, extreme in the direction $\mathbf{c}$ of the objective function.

The simplex method also searches through vertices, but by local search: move to a better neighboring vertex if there is one, and halt otherwise. Two vertices are NEIGHBORS if they share $n - 1$ constraints satisfied with equality, the two endpoints of an edge of the feasible region. Moving to a neighbor

swaps one tight constraint for another; such a swap is a PIVOT. When several neighbors are better, a PIVOT RULE chooses among them. (At a DEGENERATE vertex, one satisfying more than n constraints with equality, a swap can leave the point unchanged; simple rules prevent the resulting cycling.)

Correctness. If the linear program is feasible with a finite optimum, the simplex method terminates at an optimal solution; otherwise it detects which case holds. Intuitively, since the objective is linear and the feasible region convex, a vertex with no improving local move has no improving direction at all. Formally, at termination the simplex method exhibits a feasible dual solution satisfying complementary slackness with the final vertex (Lecture 9), so the proof of its correctness gives another proof of strong LP duality. In terms of the design paradigm of Lecture 9, the simplex method maintains primal feasibility and complementary slackness and works toward dual feasibility. (An initial feasible point is sometimes obvious, such as $\mathbf{0}$; in general one adds a variable, heavily penalized in the objective, that makes finding one trivial.)

Worst-case running time. The simplex method is very fast in practice, yet its worst-case running time is exponential in the input size. A feasible region can have exponentially many vertices (the n-dimensional hypercube has 2^n), and Klee and Minty (early 1970s) constructed a squashed n-dimensional hypercube on which the simplex method visits all of them. Such exponential lower bounds are known for all natural deterministic pivot rules. The number of iterations is also tied to the HIRSCH CONJECTURE, that the diameter of a polytope (of the graph formed by its vertices and edges) is at most linear in the number of variables and constraints. The best known upper bound is quasi-polynomial, roughly $n^{\log n}$ (Kalai and Kleitman, early 1990s). Since the simplex method walks along edges, its worst-case number of iterations is at least the diameter, so a good enough iteration bound for some pivot rule would yield progress on the conjecture.

Average-case and smoothed running time. In the 1980s, Borgwardt, Smale, Adler and Karp, and others showed that the simplex method (with a suitable pivot rule) runs in polynomial time on average, for various distributions over linear programs, although defining a natural random linear program is not easy. Spielman and Teng later proved that the simplex method has polynomial SMOOTHED COMPLEXITY: for every initial linear program, the expected running time on a small random perturbation of it is polynomial in the input size. Bad examples are thus both rare and isolated.

10.4 The ellipsoid method

The ELLIPSOID METHOD, proposed by Shor and others in the 1970s for nonlinear programming, was shown by Khachiyan in 1979 to solve linear programs in polynomial time, the first algorithm to do so. It is very slow in practice, often by orders of magnitude: the polynomial has large degree (around 4 or 5, depending on implementation details), and its typical performance is close to its worst case, whereas the simplex method usually runs far faster than its worst-case bound.

Separation oracles. The ellipsoid method is uniquely useful for proving that problems are polynomial-time solvable in principle, because it can solve some linear programs with n variables and exponentially many (in n) constraints in time polynomial in n. It needs no explicit list of constraints, only a SEPARATION ORACLE: a polynomial-time subroutine that takes a candidate solution $\mathbf{x}$ and either verifies that it is feasible or returns a constraint that it violates.

For example, the dual of the path-based linear program for maximum flow (Lecture 8) is

$$\min \sum_{e \in E} u_e \ell_e \quad \text{s.t.} \quad \sum_{e \in P} \ell_e \geq 1 \ \text{ for all } P \in \mathcal{P}, \qquad \ell_e \geq 0 \ \text{ for all } e \in E, \tag{10.7}$$

where $\mathcal{P}$ is the set of s-t paths and u_e are the edge capacities. A graph can have exponentially many s-t paths (two parallel edges from v_i to v_{i+1} along $s = v_1, \ldots, v_n = t$ give 2^{n-1}), so there can be exponentially many constraints. But some constraint is violated if and only if $\min_{P \in \mathcal{P}} \sum_{e \in P} \ell_e < 1$. The separation oracle first checks $\ell_e \geq 0$ for each e, returning a violated nonnegativity constraint if one exists. Otherwise it runs Dijkstra's algorithm with edge lengths ℓ_e. If the shortest s-t path has length at least 1, all constraints hold; if a shortest path P^* has length less than 1, it returns the violated constraint $\sum_{e \in P^*} \ell_e \geq 1$. So the ellipsoid method solves (10.7) in polynomial time. (Minimum cuts already do this, but for some problems the only known proof of polynomial-time solvability goes through the ellipsoid method.)

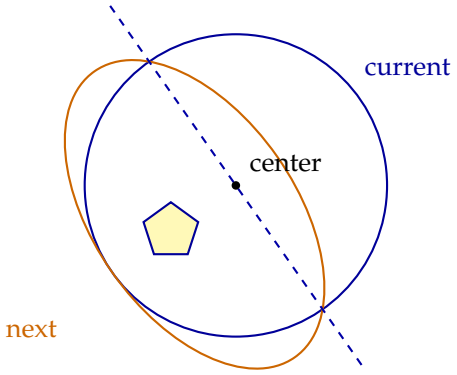


Figure 27: One step of the ellipsoid method. The current ellipsoid (blue) contains the feasible region (yellow). Its center is infeasible, and the violated constraint (dashed) leaves the feasible region in one half-ellipsoid, which the next ellipsoid (orange) contains. The new ellipsoid shown is the smallest one containing the half-disk.

How it works. First reduce optimization to feasibility: replace the objective $\max \mathbf{c}^\top \mathbf{x}$ by the constraint $\mathbf{c}^\top \mathbf{x} \geq M$ for a target M, and binary search on M. The method maintains an ellipsoid guaranteed to contain the entire feasible region, starting from a huge ball. It calls the separation oracle on the center of the current ellipsoid. If the center is feasible, it is done. Otherwise the oracle returns a constraint satisfied by all feasible points and violated by the center, so the feasible region lies on the opposite side of its boundary from the center, inside a half-ellipsoid. The method then draws a new ellipsoid containing that half-ellipsoid and recurses (Figure 27). Elementary

but tedious calculations show that the volume shrinks by a guaranteed factor in each iteration, which gives a polynomial bound on the number of iterations; the method stops when the ellipsoid is too small to contain a feasible point given the precision of the input data. Because constraints are generated only as needed and there are polynomially many iterations, only polynomially many constraints are ever generated.

10.5 Interior-point methods

The simplex method works along the boundary of the feasible region and the ellipsoid method from the outside in; INTERIOR-POINT METHODS work from the inside out. They began with Karmarkar's algorithm in 1984. In the popular CENTRAL PATH methods, instead of maximizing $\mathbf{c}^\top \mathbf{x}$ one maximizes

$$\mathbf{c}^\top \mathbf{x} - \lambda \cdot \underbrace{f(\text{distance between } \mathbf{x} \text{ and the boundary})}_{\text{barrier function}},$$

where $\lambda \geq 0$ is a parameter and the barrier f blows up as its argument tends to 0, for example $f(z) = \log \frac{1}{z}$. For very large λ the problem is easy: with $f(z) = \log \frac{1}{z}$ the solution is the ANALYTIC CENTER of the feasible region, computable for example by Newton's method. One then decreases λ gradually, tracking the optimal point; the CENTRAL PATH is the set of these optimal points as λ goes from ∞ to 0, and at $\lambda = 0$ the point is optimal for the linear program. Many interior-point algorithms run in worst-case polynomial time and are also competitive with the simplex method in practice. Each beats the other on some linear programs (interior-point methods especially on larger ones), with no good understanding of which will win when, and the simplex method remains the most commonly used linear programming algorithm in practice.

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 10: The Minimax Theorem and Algorithms for Linear Programming.

Lecture 11: Online Learning and the Multiplicative Weights Algorithm

11.1 Online algorithms

An online problem has two features:

1. the input arrives one piece at a time, and
2. an algorithm makes an irrevocable decision each time it receives a new piece of the input.

For example, jobs may arrive one by one and each must be scheduled on some machine immediately, or the vertices of a graph may arrive one by one with their edges to previously arrived vertices. Many real-world problems are inherently online.

11.2 Online decision-making

Fix a set A of $n \geq 2$ actions and a time horizon $T \geq 1$.

Algorithm 11.1: Online decision-making

```
for each time step t = 1, 2, ..., T do
    the decision-maker picks a probability distribution p^t over A
    an adversary picks a reward vector r^t : A → [−1, 1]
    an action a^t is drawn from p^t, and the decision-maker receives reward r^t(a^t)
    the decision-maker learns r^t, the entire reward vector
```

An online decision-making algorithm specifies each $\mathbf{p}^t$ as a function of $\mathbf{r}^1, \ldots, \mathbf{r}^{t-1}$ and the realized actions $a^1, \ldots, a^{t-1}$. An adversary for an algorithm $\mathcal{A}$ specifies each $\mathbf{r}^t$ as a function of the distributions $\mathbf{p}^1, \ldots, \mathbf{p}^t$ used by $\mathcal{A}$ and the realized actions $a^1, \ldots, a^{t-1}$. The actions might be investment strategies, driving routes between home and work, or strategies in a zero-sum game.

The adversary chooses $\mathbf{r}^t$ after the decision-maker commits to $\mathbf{p}^t$, so expectations must be modest. Three examples delimit what is possible.

Example: *The best action sequence is too strong a benchmark.* Let $A = \{1, 2\}$. Each day, if $\mathbf{p}^t$ puts probability at least $\frac{1}{2}$ on action 1, the adversary sets $\mathbf{r}^t = (-1, 1)$, and otherwise $\mathbf{r}^t = (1, -1)$. The algorithm's expected reward is $1 - 2p^t(1) \leq 0$ or $2p^t(1) - 1 < 0$ each day, so its expected total reward is nonpositive, while the best action sequence in hindsight, $\sum_{t=1}^{T} \max_{a \in A} r^t(a)$, earns T.

Definition 11.1 [Regret]. The regret of actions $a^1, \ldots, a^T$ for rewards $\mathbf{r}^1, \ldots, \mathbf{r}^T$ is

$$\underbrace{\max_{a \in A} \sum_{t=1}^{T} r^t(a)}_{\text{best fixed action}} - \underbrace{\sum_{t=1}^{T} r^t(a^t)}_{\text{our algorithm}} . \tag{11.1}$$

The goal is regret close to 0 (negative is even better). Since rewards lie in $[-1, 1]$, regret is at most $2T$, and regret $\Omega(T)$ counts as an epic fail. The best fixed action in hindsight is a good benchmark: simple learning algorithms can compete with it, doing so requires some ingenuity, and it already suffices for many applications. Comparing with the best fixed action is also called combining expert advice, viewing each action as an expert. A natural algorithm is follow-the-leader, which at time t chooses an action with maximum cumulative reward $\sum_{u=1}^{t-1} r^u(a)$ so far.

Example: *Randomization is necessary for no regret.* A deterministic algorithm, such as follow-the-leader, commits to a single action a^t at each step. The adversary sets $r^t(a^t) = 0$ and $r^t(a) = 1$ for every other action. The algorithm earns 0, while the actions earn $T(n-1)$ in total, so the best fixed action earns at least $T(1 - \frac{1}{n})$. With only 2 actions and arbitrarily large T, the worst-case regret is at least $\frac{T}{2}$.

Example: *A $\sqrt{(\ln n)/T}$ lower bound on time-averaged regret.* Let $n = 2$ and choose each $\mathbf{r}^t$ independently and uniformly from $\{(1, -1), (-1, 1)\}$. Every algorithm has expected reward exactly 0 at each step, hence expected total reward 0. The best fixed action in hindsight earns $|S|$, where S is the sum of T independent uniform signs; since a fair coin flipped T times has $\frac{T}{2}$ heads in expectation with standard deviation $\frac{1}{2}\sqrt{T}$, its expectation is $b\sqrt{T}$ for a constant $b > 0$ independent of T. So every algorithm $\mathcal{A}$ has expected regret at least $b\sqrt{T}$ over the random rewards and its own random actions, and therefore some fixed choice of reward vectors (an adversary) forces expected regret at least $b\sqrt{T}$ over the actions alone. A similar argument with n actions shows that expected regret cannot grow more slowly than $b\sqrt{T \ln n}$, for a constant $b > 0$ independent of n and T.

11.3 The multiplicative weights algorithm

The multiplicative weights (MW) algorithm matches this lower bound up to a constant factor.

Theorem 11.2 [Regret of multiplicative weights]. *There is an online decision-making algorithm that, for every adversary and every $T \geq 4 \ln n$, has expected regret at most $2\sqrt{T \ln n}$.*

Dividing by T gives the time-averaged regret. The number of steps needed to make it small is only logarithmic in the number of actions, and this is the form used in applications.

Corollary 11.3. *There is an online decision-making algorithm that, for every adversary and every $\epsilon \in (0, 1]$, has expected time-averaged regret at most ϵ after at most $(4 \ln n)/\epsilon^2$ time steps.*

Indeed, $2\sqrt{T \ln n}/T \le \epsilon$ exactly when $T \ge (4 \ln n)/\epsilon^2$, and this T is at least $4 \ln n$. The design follows two principles: past performance guides the choice of action, with the probability of an action increasing in its cumulative reward (and randomization is necessary); and the probability of choosing a poorly performing action decreases at an exponential rate. The first is essential for regret sublinear in T, the second for optimal regret bounds. MW keeps a weight, intuitively a credibility, for each action and picks actions with probability proportional to their current weights.

Algorithm 11.2: Multiplicative weights (MW)

Input: Actions A, horizon T, learning rate $\eta \in (0, \frac{1}{2}]$.

$w^1(a) \leftarrow 1$ for every $a \in A$

for each time step $t = 1, 2, \ldots, T$ **do**

  use the distribution $\mathbf{p}^t \leftarrow \mathbf{w}^t / \Gamma^t$, where $\Gamma^t = \sum_{a \in A} w^t(a)$

  given $\mathbf{r}^t$, set $w^{t+1}(a) \leftarrow w^t(a) \cdot (1 + \eta\, r^t(a))$ for every $a \in A$

With rewards ± 1, each weight goes up by a factor $1 + \eta$ or down by a factor $1 - \eta$. As $\eta \to 0$, the distributions stay close to uniform, so small η encourages exploration; large η behaves like follow-the-leader and encourages exploitation. The parameter η interpolates between the two and is chosen at the end of the proof.

11.4 Proof of the regret bound

Fix reward vectors $\mathbf{r}^1, \ldots, \mathbf{r}^T$. (Because MW's distributions do not depend on the realized actions, the worst adaptive adversary for it is in fact oblivious, one that fixes all reward vectors in advance.) The expected reward of MW and the reward of the best fixed action seem unrelated; the idea is to relate both to the final total weight $\Gamma^{T+1} = \sum_{a \in A} w^{T+1}(a)$.

Step 1: total weight tracks MW's expected reward. Let

$$v^t = \sum_{a \in A} p^t(a)\, r^t(a) = \sum_{a \in A} \frac{w^t(a)}{\Gamma^t}\, r^t(a)$$

be MW's expected reward at step t. Then

$$\Gamma^{t+1} = \sum_{a \in A} w^t(a)\big(1 + \eta\, r^t(a)\big) = \Gamma^t(1 + \eta v^t) \le \Gamma^t e^{\eta v^t},$$

using $1 + x \le e^x$ for all real x (Figure 28). Since $\Gamma^1 = n$,

$$\Gamma^{T+1} \le n \prod_{t=1}^{T} e^{\eta v^t} = n \cdot e^{\eta \sum_{t=1}^{T} v^t}. \tag{11.2}$$

Step 2: a good fixed action makes the total weight large. Let $\mathrm{OPT} = \sum_{t=1}^{T} r^t(a^*)$ be the cumulative reward

of the best fixed action a^*. Then

$$\Gamma^{T+1} \geq w^{T+1}(a^*) = w^1(a^*) \prod_{t=1}^{T} \bigl(1 + \eta\, r^t(a^*)\bigr), \qquad w^1(a^*) = 1. \tag{11.3}$$

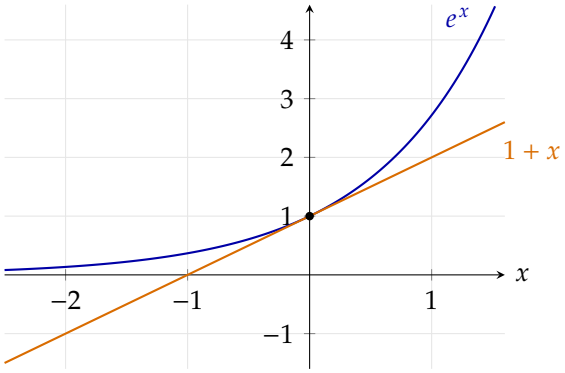


Figure 28: The line $1 + x$ lies below e^x everywhere and is tangent to it at $x = 0$, so the two are close for x near 0.

To turn the product into a sum we need a lower bound on $1 + x$ this time. From the Taylor expansion $\ln(1 + x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots$, for $|x| \leq \frac{1}{2}$ each term $\frac{|x|^k}{k}$ with $k \geq 3$ is at most $\frac{x^2}{2}\bigl(\frac{1}{2}\bigr)^{k-2}$, so the terms after the second sum to at most $\frac{x^2}{2}$ in absolute value and

$$\ln(1 + x) \geq x - x^2 \qquad \text{for } |x| \leq \tfrac{1}{2}.$$

Since $\eta \leq \frac{1}{2}$ and $|r^t(a^*)| \leq 1$, plugging into (11.3) and using $(r^t(a^*))^2 \leq 1$ gives

$$\Gamma^{T+1} \geq \prod_{t=1}^{T} e^{\eta r^t(a^*) - \eta^2 (r^t(a^*))^2} \geq e^{\eta\, \mathrm{OPT} - \eta^2 T}.$$

Step 3: combine. By (11.2) and Step 2, $n \cdot e^{\eta \sum_t v^t} \geq \Gamma^{T+1} \geq e^{\eta\, \mathrm{OPT} - \eta^2 T}$. Taking logarithms and dividing by η,

$$\sum_{t=1}^{T} v^t \geq \mathrm{OPT} - \eta T - \frac{\ln n}{\eta}. \tag{11.4}$$

The term ηT is the cost of inaccurate learning (larger for larger learning rates), and $\frac{\ln n}{\eta}$ the overhead before converging (larger for smaller rates). Equalizing them with $\eta = \sqrt{(\ln n)/T}$, which is at most $\frac{1}{2}$ because $T \geq 4 \ln n$, makes MW's expected cumulative reward at most $2\sqrt{T \ln n}$ less than that of the best fixed action. This proves Theorem 11.2.

The choice of η assumes the horizon T is known. Minor modifications extend MW and its guarantee to an unknown horizon, with the constant 2 replaced by a modestly larger one.

11.5 Minimax revisited

In a zero-sum game $\mathbf{A}$, the minimax theorem (proved via strong LP duality in Lecture 10) states that

$$\max_{\mathbf{p}}\Big(\min_{\mathbf{q}} \mathbf{p}^\top \mathbf{A}\mathbf{q}\Big) = \min_{\mathbf{q}}\Big(\max_{\mathbf{p}} \mathbf{p}^\top \mathbf{A}\mathbf{q}\Big). \tag{11.5}$$

The easy direction "$\le$" holds because for any $\mathbf{p}'$ and $\mathbf{q}'$, $\min_{\mathbf{q}} \mathbf{p}'^\top \mathbf{A}\mathbf{q} \le \mathbf{p}'^\top \mathbf{A}\mathbf{q}' \le \max_{\mathbf{p}} \mathbf{p}^\top \mathbf{A}\mathbf{q}'$. The MW guarantee gives the other direction directly.

Let $\mathbf{A}$ have payoffs in $[-1, 1]$, fix $\epsilon \in (0, 1]$, and let n be the larger of the numbers of rows and columns. For $t = 1, \dots, T = (4 \ln n)/\epsilon^2$:

- the row and column players choose mixed strategies $\mathbf{p}^t$ and $\mathbf{q}^t$ using their own copies of MW, with the rows or the columns as actions;
- the row player feeds MW the reward vector $\mathbf{r}^t = \mathbf{A}\mathbf{q}^t$, the expected payoff of each row against $\mathbf{q}^t$;
- the column player feeds MW the reward vector $\mathbf{r}^t = -(\mathbf{p}^t)^\top \mathbf{A}$.

Let $v = \frac{1}{T}\sum_{t=1}^{T} (\mathbf{p}^t)^\top \mathbf{A}\mathbf{q}^t$ be the row player's time-averaged payoff, and let $\hat{\mathbf{p}} = \frac{1}{T}\sum_t \mathbf{p}^t$ and $\hat{\mathbf{q}} = \frac{1}{T}\sum_t \mathbf{q}^t$ be the time-averaged strategies. MW's expected reward at step t is exactly $(\mathbf{p}^t)^\top \mathbf{r}^t$. By Corollary 11.3 for the row player, compared with the best fixed row (equivalently, the best fixed distribution over rows, since $\mathbf{p}^\top \mathbf{A}\hat{\mathbf{q}}$ is linear in $\mathbf{p}$),

$$v \ge \max_{\mathbf{p}} \frac{1}{T}\sum_{t=1}^{T} \mathbf{p}^\top \mathbf{A}\mathbf{q}^t - \epsilon = \max_{\mathbf{p}} \mathbf{p}^\top \mathbf{A}\hat{\mathbf{q}} - \epsilon,$$

and symmetrically for the column player, $v \le \min_{\mathbf{q}} \hat{\mathbf{p}}^\top \mathbf{A}\mathbf{q} + \epsilon$. Therefore

$$\max_{\mathbf{p}}\Big(\min_{\mathbf{q}} \mathbf{p}^\top \mathbf{A}\mathbf{q}\Big) \ge \min_{\mathbf{q}} \hat{\mathbf{p}}^\top \mathbf{A}\mathbf{q} \ge v - \epsilon \ge \max_{\mathbf{p}} \mathbf{p}^\top \mathbf{A}\hat{\mathbf{q}} - 2\epsilon \ge \min_{\mathbf{q}}\Big(\max_{\mathbf{p}} \mathbf{p}^\top \mathbf{A}\mathbf{q}\Big) - 2\epsilon.$$

Letting $\epsilon \to 0$ and combining with the easy direction proves (11.5).

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 11: Online Learning and the Multiplicative Weights Algorithm.

Lecture 12: Applications of Multiplicative Weights to Games and Linear Programs

12.1 Extensions of the multiplicative weights guarantee

The multiplicative weights algorithm (Lecture 11) is used here as a black box, through its regret guarantee.

Theorem 12.1 [Multiplicative weights**].** *With n actions, time horizon $T \geq 4 \ln n$, and rewards in $[-1, 1]$, the expected regret of the multiplicative weights algorithm is always at most $2\sqrt{T \ln n}$.*

Here the regret is $\max_{a \in A} \sum_{t=1}^{T} r^t(a) - \sum_{t=1}^{T} r^t(a^t)$, the expectation is over the random choice of action at each step, and the reward vectors $\mathbf{r}^1, \ldots, \mathbf{r}^T$ are arbitrary.

Fixed distributions. The guarantee also holds against the best fixed probability distribution in hindsight, because the best fixed action is as good as the best fixed distribution: for every distribution $\mathbf{p}$ over A,

$$\sum_{t=1}^{T} \sum_{a \in A} p_a \, r^t(a) = \sum_{a \in A} p_a \Big(\sum_{t=1}^{T} r^t(a) \Big) \leq \max_{b \in A} \sum_{t=1}^{T} r^t(b),$$

since the p_a sum to 1.

Corollary 12.2. *For $\epsilon \in (0, 1]$, the expected time-averaged regret of the multiplicative weights algorithm, with respect to any fixed distribution over the actions, is at most ϵ after at most $(4 \ln n)/\epsilon^2$ time steps.*

Rewards in $[-M, M]$. Scaling reduces this case to the previous one: scale all rewards down by M, run multiplicative weights until the time-averaged expected regret is at most $\frac{\epsilon}{M}$, and scale back up. Equivalently, use the update rule $w^{t+1}(a) = w^t(a)\big(1 + \frac{\eta}{M} r^t(a)\big)$. Either way, after $T = \frac{4M^2 \ln n}{\epsilon^2}$ iterations the time-averaged expected regret is at most ϵ.

12.2 Minimax revisited (again)

In Lecture 11 both players of a zero-sum game ran multiplicative weights simultaneously. In the following alternative, the players move sequentially at each step, and only the row player uses multiplicative weights while the column player best responds. This version translates more directly into algorithms. Fix a zero-sum game $\mathbf{A}$ with payoffs in $[-M, M]$ and m rows, and fix $\epsilon \in (0, M]$.

Algorithm 12.1: Thought experiment

for each time step $t = 1, 2, \ldots, T = \frac{4M^2 \ln m}{\epsilon^2}$ **do**
  the row player chooses a mixed strategy $\mathbf{p}^t$ using multiplicative weights, with the rows as actions
  the column player responds optimally with a deterministic strategy $\mathbf{q}^t$ *// the second mover need not randomize*
  if the column player chose column j, set $r^t(i) = a_{ij}$ for every row i and feed $\mathbf{r}^t$ to multiplicative weights

The reward vector is the payoff of each row in hindsight, given the column player's choice at time t. Each claim below uses the benchmark that is more favorable to the player in question, the one where that player moves second.

Lemma 12.3 [Column player's payoff]. *The column player's negative time-averaged expected payoff is at most* $\max_{\mathbf{p}}\left(\min_{\mathbf{q}} \mathbf{p}^\top \mathbf{A}\mathbf{q}\right)$*:*

$$\frac{1}{T}\sum_{t=1}^{T} (\mathbf{p}^t)^\top \mathbf{A}\mathbf{q}^t \le \max_{\mathbf{p}}\left(\min_{\mathbf{q}} \mathbf{p}^\top \mathbf{A}\mathbf{q}\right).$$

Proof. The column player not only moves second but tailors its best response each day to that day's strategy. Let $\hat{\mathbf{p}} = \frac{1}{T}\sum_{t=1}^{T} \mathbf{p}^t$ and let $\mathbf{q}^*$ be an optimal response to $\hat{\mathbf{p}}$. Then

$$\max_{\mathbf{p}}\left(\min_{\mathbf{q}} \mathbf{p}^\top \mathbf{A}\mathbf{q}\right) \ge \min_{\mathbf{q}} \hat{\mathbf{p}}^\top \mathbf{A}\mathbf{q} = \hat{\mathbf{p}}^\top \mathbf{A}\mathbf{q}^* = \frac{1}{T}\sum_{t=1}^{T} (\mathbf{p}^t)^\top \mathbf{A}\mathbf{q}^* \ge \frac{1}{T}\sum_{t=1}^{T} (\mathbf{p}^t)^\top \mathbf{A}\mathbf{q}^t,$$

where the last step holds because each $\mathbf{q}^t$ is an optimal response to $\mathbf{p}^t$, minimizing $(\mathbf{p}^t)^\top \mathbf{A}\mathbf{q}$. □

Lemma 12.4 [Row player's payoff]. *The row player's time-averaged expected payoff satisfies*

$$\frac{1}{T}\sum_{t=1}^{T} (\mathbf{p}^t)^\top \mathbf{A}\mathbf{q}^t \ge \min_{\mathbf{q}}\left(\max_{\mathbf{p}} \mathbf{p}^\top \mathbf{A}\mathbf{q}\right) - \epsilon.$$

Proof. Let $\hat{\mathbf{q}} = \frac{1}{T}\sum_{t=1}^{T} \mathbf{q}^t$. By Corollary 12.2 with the $[-M, M]$ extension, the row player's time-averaged expected payoff is within ϵ of that of every fixed mixed strategy $\mathbf{p}$:

$$\frac{1}{T}\sum_{t=1}^{T} (\mathbf{p}^t)^\top \mathbf{A}\mathbf{q}^t \ge \max_{\mathbf{p}}\left(\frac{1}{T}\sum_{t=1}^{T} \mathbf{p}^\top \mathbf{A}\mathbf{q}^t\right) - \epsilon = \max_{\mathbf{p}} \mathbf{p}^\top \mathbf{A}\hat{\mathbf{q}} - \epsilon \ge \min_{\mathbf{q}}\left(\max_{\mathbf{p}} \mathbf{p}^\top \mathbf{A}\mathbf{q}\right) - \epsilon.$$ □

Letting $\epsilon \to 0$, the two lemmas with the easy direction $\max_{\mathbf{p}} \min_{\mathbf{q}} \mathbf{p}^\top \mathbf{A}\mathbf{q} \le \min_{\mathbf{q}} \max_{\mathbf{p}} \mathbf{p}^\top \mathbf{A}\mathbf{q}$ give another proof of the minimax theorem. Next, the thought experiment becomes a fast algorithm for approximately solving linear programs.

12.3 Linear classifiers revisited

The input consists of m positive points $\mathbf{p}^1, \ldots, \mathbf{p}^m \in \mathbb{R}^d$ (label +1) and m' negative points $\mathbf{q}^1, \ldots, \mathbf{q}^{m'} \in \mathbb{R}^d$ (label −1). The goal is a linear function $h(\mathbf{z}) = \sum_{j=1}^{d} a_j z_j + b$ with $h(\mathbf{p}^i) > 0$ for all positive points and $h(\mathbf{q}^i) < 0$ for all negative points (Figure 29). Lecture 7 solved this by linear programming, using an extra variable to turn the strict inequalities into weak ones. For data sets too large for that, multiplicative weights computes a classifier faster, under an assumption.

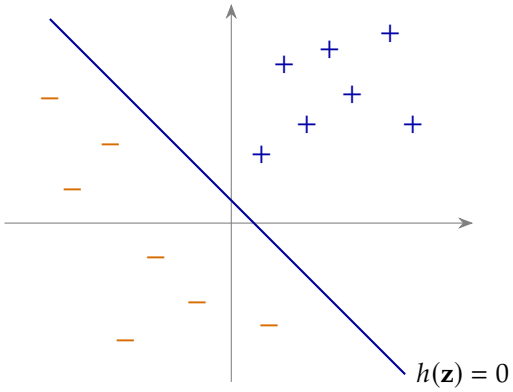


Figure 29: A linear classifier separating positive points (+) from negative points (−), schematically.

Preprocessing.

1. *Zero intercept.* Give each point a new $(d+1)$th coordinate equal to 1; the new coefficient a_{d+1} plays the role of b. Now we seek a separating hyperplane through the origin.
2. *One class.* Multiply every coordinate of each (extended) negative point by −1. Writing $\mathbf{x}^i$ for all the resulting points, the constraints become $h(\mathbf{x}^i) > 0$ for every i: all points on the same side of a hyperplane through the origin.
3. *Nonnegative coefficients.* Replace each coordinate x_j^i by the two entries x_j^i and $-x_j^i$, and interpret the two corresponding coefficients $a_j', a_j'' \geq 0$ as $a_j = a_j' - a_j''$. The normal vector now lies in the nonnegative orthant.

From now on d denotes the dimension after preprocessing (formerly $2d+2$), and there are m points $\mathbf{x}^1, \ldots, \mathbf{x}^m \in \mathbb{R}^d$.

Assumption. There is a coefficient vector $\mathbf{a}^* \in \mathbb{R}_+^d$ with

$$\sum_{j=1}^{d} a_j^* = 1 \qquad \text{and} \qquad \sum_{j=1}^{d} a_j^* x_j^i > \epsilon \ \text{ for every data point } \mathbf{x}^i.$$

Any solution can be scaled so that its coefficients sum to 1; the second condition asks for a MARGIN ϵ after this normalization. Let M satisfy $|x_j^i| \le M$ for all i, j.

Algorithm 12.2: Linear classifier via multiplicative weights

Input: Points $\mathbf{x}^1, \dots, \mathbf{x}^m \in \mathbb{R}^d$ with $|x_j^i| \le M$ and margin ϵ.

Output: Coefficients $\mathbf{a} \in \mathbb{R}_+^d$ with $\sum_j a_j x_j^i > 0$ for every i.

Use the action set $A = \{1, \dots, d\}$, one action per coordinate

for $t = 1, 2, \dots, T = \frac{4M^2 \ln d}{\epsilon^2}$ **do**

  use multiplicative weights to generate a distribution $\mathbf{a}^t$ over the coordinates

  if $\sum_{j=1}^d a_j^t x_j^i > 0$ for every i **then** halt and **return** $\mathbf{a}^t$

  choose a data point $\mathbf{x}^i$ with $\sum_j a_j^t x_j^i \le 0$, and set $r^t(j) \leftarrow x_j^i$ for each j

  feed $\mathbf{r}^t$ to multiplicative weights

The reward $\mathbf{r}^t = \mathbf{x}^i$ nudges the coefficients toward the violated point: under the update $w^{t+1}(j) = w^t(j)(1 + \eta r^t(j))$, coordinates with $x_j^i > 0$ gain weight and those with $x_j^i < 0$ lose it, so the inner product with $\mathbf{x}^i$ tends to go up.

Theorem 12.5 [CLASSIFIER IN T ITERATIONS]. *Under the assumption, the algorithm halts with a feasible classifier within $T = \frac{4M^2 \ln d}{\epsilon^2}$ iterations.*

Proof. Viewing $\mathbf{a}^t$ as a distribution over actions, the expected reward of multiplicative weights at step t is $\sum_j a_j^t r^t(j) = \sum_j a_j^t x_j^i \le 0$, so as long as the algorithm has not halted, its time-averaged expected reward is at most 0. The fixed distribution $\mathbf{a}^*$ would have earned $\sum_j a_j^* r^t(j) \ge \min_i \sum_j a_j^* x_j^i > \epsilon$ at every step. So while no feasible solution has been found, the time-averaged regret with respect to $\mathbf{a}^*$ exceeds ϵ. Rewards lie in $[-M, M]$, so by Corollary 12.2 (against a fixed distribution, with the $[-M, M]$ extension) this regret is at most ϵ after T steps. Hence the algorithm halts by then, necessarily with a feasible solution. □

Interpretation as a zero-sum game. The classifier algorithm is a special case of the thought experiment. Give the row player one row per coordinate j and the column player one column per data point $\mathbf{x}^i$, with payoff matrix $\mathbf{A}$ whose (j, i) entry is x_j^i. The row player's strategies $\mathbf{a}^1, \dots, \mathbf{a}^T$ come from multiplicative weights. The column player's best response to $\mathbf{a}^t$ is a data point minimizing $\sum_j a_j^t x_j^i$, which is a violated point if one exists, and the reward vector for that column is the payoff of each row, namely that data point. The assumption makes the value of the game larger than ϵ, since the row player can play $\mathbf{a}^*$. By Lemma 12.4 (with d rows), once T reaches $\frac{4M^2 \ln d}{\epsilon^2}$ the row player's time-averaged payoff is at least the value minus ϵ, which is positive. Every step at which no feasible solution is found gives the row player payoff at most 0, so the algorithm must halt with a feasible solution within T iterations.

12.4 Maximum flow revisited

A linear program can be approximately solved by multiplicative weights by modeling it as a zero-sum game, with one player for the decision variables and the other for the constraints, and running the thought experiment. For the classifier, this is very fast unless the margin ϵ is tiny or the radius M of the ℓ_∞ ball containing the data is large. The same idea applies to maximum flow and extends to multicommodity flow, for which no exact algorithms significantly faster than linear programming are known.

The game. Consider unit capacities ($u_e = 1$ for all e); general capacities are handled by the same ideas, with running time quadratic in the ratio of the largest to the smallest capacity. With $\mathcal{P}$ the set of s-t paths, the maximum flow problem and its dual (Lecture 8) are

$$\max \sum_{P \in \mathcal{P}} f_P \ \text{ s.t. } \underbrace{\sum_{P \in \mathcal{P}:\, e \in P} f_P}_{\text{total flow on } e} \le 1 \ (e \in E), \ \ f_P \ge 0, \qquad \min \sum_{e \in E} \ell_e \ \text{ s.t. } \sum_{e \in P} \ell_e \ge 1 \ (P \in \mathcal{P}), \ \ \ell_e \ge 0.$$

Let OPT be their common optimal value (strong duality). The row player is associated with edges (dual variables) and the column player with paths (primal variables), with payoff matrix $\mathbf{A}$ having entries $a_{eP} = 1$ if $e \in P$ and $a_{eP} = 0$ otherwise. The matrix is huge but is never written down. The value of the game is the row player's expected payoff under optimal play, $\max_{\mathbf{x}} \min_{\mathbf{y}} \mathbf{x}^\top \mathbf{A}\mathbf{y} = \min_{\mathbf{y}} \max_{\mathbf{x}} \mathbf{x}^\top \mathbf{A}\mathbf{y}$.

Algorithm. Assume for simplicity that OPT is known (in general, guess OPT $= 1, 2, 4, 8, \ldots$ by successive doubling until the algorithm succeeds). The thought experiment becomes the following.

Algorithm 12.3: Approximate maximum flow via multiplicative weights

Input: Unit-capacity graph $G = (V, E)$ with source s, sink t, and OPT.
Output: Paths $P^1, \ldots, P^T$.

Associate an action with each edge $e \in E$
for $t = 1, 2, \ldots, T = \frac{4\,\mathrm{OPT}^2 \ln |E|}{\epsilon^2}$ **do**
  use multiplicative weights to generate a distribution $\mathbf{x}^t$ over the edges
  let $P^t \in \arg\min_{P \in \mathcal{P}} \sum_{e \in P} x_e^t$ *// column minimizing the row player's payoff*
  set $r^t(e) \leftarrow 1$ for $e \in P^t$ and $r^t(e) \leftarrow 0$ otherwise (the P^t column of $\mathbf{A}$), and feed $\mathbf{r}^t$ to multiplicative weights

Running time. The algorithm keeps one weight per edge. Computing P^t is a shortest s-t path computation with edge lengths x_e^t, which Dijkstra's algorithm solves; this is exactly the separation oracle for the dual linear program from Lecture 10. With Dijkstra in $O(m \log n)$ time, where $m = |E|$ and $n = |V|$, the total running time is $O\big(\frac{\mathrm{OPT}^2}{\epsilon^2}\, m \log m \log n\big)$. With unit capacities, OPT $\le m$, and OPT $\le n - 1$ if there are no parallel edges.

Lemma 12.6 [Value of the flow game]. *The value of this zero-sum game is* $\frac{1}{\text{OPT}}$.

Proof. Let $\{\ell_e^*\}$ be an optimal dual solution, with $\sum_e \ell_e^* = \text{OPT}$, and let $x_e = \ell_e^*/\text{OPT}$, a probability distribution. Against this row strategy, each path P gives expected payoff $\sum_{e \in P} x_e = \frac{1}{\text{OPT}} \sum_{e \in P} \ell_e^* \geq \frac{1}{\text{OPT}}$ by dual feasibility, so the value is at least $\frac{1}{\text{OPT}}$. Conversely, let $\mathbf{x}$ be an optimal row strategy, guaranteeing the value v against every column: $\sum_{e \in P} x_e \geq v$ for every $P \in \mathcal{P}$. Then $\{x_e/v\}$ is dual feasible with objective value $\sum_e x_e/v = 1/v$, which is at least OPT. So $v \leq \frac{1}{\text{OPT}}$. □

Approximate correctness. Let f^t route OPT units of flow on the path P^t (possibly violating capacities), and let $f^* = \frac{1}{T} \sum_{t=1}^{T} f^t$ be the time-averaged flow. It routes OPT units from s to t.

Lemma 12.7 [Approximate feasibility]. *The flow f^* routes at most $1 + \epsilon$ units on every edge.*

Proof. Suppose f^* routes more than $1 + \epsilon$ units on edge e. The flow of f^* on e is $\text{OPT} \cdot |\{t : e \in P^t\}|/T$, so more than $(1 + \epsilon)T/\text{OPT}$ of the paths $P^1, \ldots, P^T$ contain e. Let $\mathbf{z}$ be the row strategy that deterministically plays e, and let $\mathbf{y}^t$ denote column P^t. In hindsight, $\mathbf{z}$ would have earned the time-averaged payoff

$$\frac{1}{T} \sum_{t=1}^{T} \mathbf{z}^\top \mathbf{A} \mathbf{y}^t = \frac{1}{T} \sum_{t:\, e \in P^t} 1 > \frac{1+\epsilon}{\text{OPT}}.$$

Payoffs lie in $[0, 1]$ and $T = \frac{4 \ln |E|}{(\epsilon/\text{OPT})^2}$, so the multiplicative weights guarantee (as in Lemma 12.4) gives time-averaged regret at most $\frac{\epsilon}{\text{OPT}}$ with respect to $\mathbf{z}$, and

$$\frac{1}{T} \sum_{t=1}^{T} (\mathbf{x}^t)^\top \mathbf{A} \mathbf{y}^t \geq \frac{1}{T} \sum_{t=1}^{T} \mathbf{z}^\top \mathbf{A} \mathbf{y}^t - \frac{\epsilon}{\text{OPT}} > \frac{1+\epsilon}{\text{OPT}} - \frac{\epsilon}{\text{OPT}} = \frac{1}{\text{OPT}}.$$

But each $\mathbf{y}^t$ is a best response to $\mathbf{x}^t$, so $(\mathbf{x}^t)^\top \mathbf{A} \mathbf{y}^t = \min_{\mathbf{y}} (\mathbf{x}^t)^\top \mathbf{A} \mathbf{y} \leq \frac{1}{\text{OPT}}$, the value of the game by Lemma 12.6; this is the column player's guarantee of Lemma 12.3. Averaging over t gives a contradiction. □

Scaling f^* down by a factor of $1 + \epsilon$ yields a feasible flow with value at least $\text{OPT}/(1 + \epsilon)$.

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 12: Applications of Multiplicative Weights to Games and Linear Programs.

Lecture 13: Online Scheduling and Online Steiner Tree

In an ONLINE PROBLEM the input arrives one piece at a time, and the algorithm makes an irrevocable decision each time it receives a new piece. An online algorithm is compared with the strongest benchmark, the optimal solution in hindsight. For a minimization problem it is α-COMPETITIVE, or has COMPETITIVE RATIO at most α, if on every input its cost is at most α times the optimal cost in hindsight.

The analyses below do not use optimality conditions. Instead they use simple lower bounds on the optimal cost, quantities that are "only better than optimal": an algorithm whose cost is close to such a bound is also close to optimal.

13.1 Online makespan scheduling

There are m identical machines, known up front. Jobs arrive online, one at a time, and job j has a known processing time p_j. Each job must be assigned to a machine immediately upon its arrival. A SCHEDULE assigns each job to one machine; the LOAD of a machine is the sum of the processing times of its jobs, and the MAKESPAN is the maximum load. The objective is to minimize the makespan, which is the completion time of the last job when the jobs are pieces of one task processed in parallel.

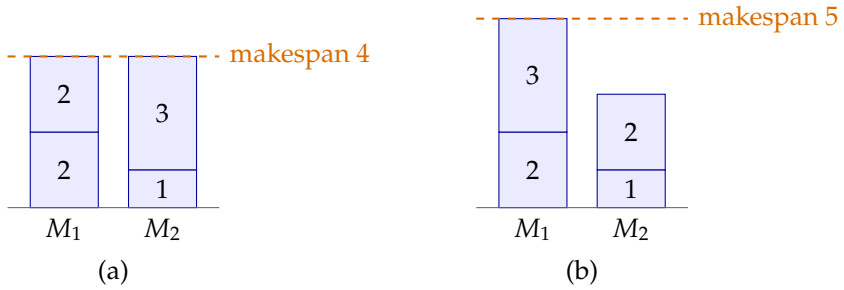


Figure 30: Two schedules of jobs with processing times 2, 2, 3, 1 on two machines, with makespans 4 and 5.

Algorithm 13.1: Graham's scheduling algorithm

Input: m machines; jobs with processing times $p_1, p_2, \ldots$ arriving online.
Output: An assignment of each job to a machine.

```
for each arriving job j do
    Assign j to a machine with the smallest current load (ties broken arbitrarily)
```

The benchmark OPT is the minimum makespan in hindsight. Computing it is NP-hard, so the benchmark is both omniscient about the future and computationally unbounded.

Lemma 13.1 [LOWER BOUND #1]. $OPT \geq \max_{j=1}^{n} p_j$.

The biggest job is assigned somewhere, and that machine's load is at least its size.

Lemma 13.2 [Lower bound #2]. $OPT \geq \frac{1}{m} \sum_{j=1}^{n} p_j$.

Proof. In every schedule the maximum load of a machine is at least the average load of a machine, which is $\frac{1}{m} \sum_{j=1}^{n} p_j$. □

Theorem 13.3 [Graham]. *The makespan of the schedule produced by Graham's algorithm is always at most twice the minimum makespan in hindsight. That is, the algorithm is 2-competitive.*

Proof. Let machine i have the largest final load, and let j be the last job assigned to it. When j arrived, machine i had the smallest load, so just before j was assigned,

$$\text{load of } i = \text{minimum load} \leq \text{average load} = \frac{1}{m} \sum_{k=1}^{j-1} p_k.$$

Hence

$$\text{final load of } i \leq \underbrace{\frac{1}{m} \sum_{k=1}^{j-1} p_k}_{\leq OPT} + \underbrace{p_j}_{\leq OPT} \leq 2\,OPT,$$

by Lemmas 13.2 and 13.1. □

The bound 2 is tight in the worst case as $m \to \infty$, though better bounds hold when all jobs are relatively small.

13.2 The online Steiner tree problem

In the online Steiner tree problem, an algorithm is given in advance a connected undirected graph $G = (V, E)$ with a nonnegative cost $c_e \geq 0$ on each edge. Terminals $t_1, \ldots, t_k \in V$ then arrive online. The algorithm must maintain at all times a subgraph of G spanning all terminals that have arrived so far, so each new terminal must be connected to the subgraph built so far. The benchmark is the minimum-cost subgraph spanning all terminals, the Steiner tree (since costs are nonnegative, it may be taken to be a tree). Think of a cable company extending its infrastructure to new markets.

In the metric case,

1. G is the complete graph, and
2. the costs satisfy the triangle inequality: $c_{uw} \leq c_{uv} + c_{vw}$ for all $u, v, w \in V$.

Then a one-hop path is at least as cheap as a two-hop path, and by induction the direct edge is a

shortest path between its endpoints. Distances in a normed space such as Euclidean space are an example; airline fares, where an intermediate stop can lower the price, are a non-example. The first assumption alone is harmless, since missing edges can be added with very high cost. The metric case is no less general than the general case.

Lemma 13.4. *Every α-competitive online algorithm for the metric case of the online Steiner tree problem can be transformed into an α-competitive online algorithm for the general case.*

From now on the instance is metric.

Algorithm 13.2: Greedy online Steiner tree

Input: Metric costs c on the complete graph; terminals $t_1, \dots, t_k$ arriving online.
Output: A tree T spanning the terminals that have arrived.

```
T ← ∅
for each arrival t_i, i = 2, ..., k do
    Add to T a cheapest edge of the form (t_i, t_j) with j < i
```

So in the 11th iteration the algorithm examines the 10 edges from the new terminal to the terminals that have already arrived, and adds the cheapest. Unlike Prim's algorithm, which chooses the next vertex greedily, the online algorithm must process the terminals in the order given.

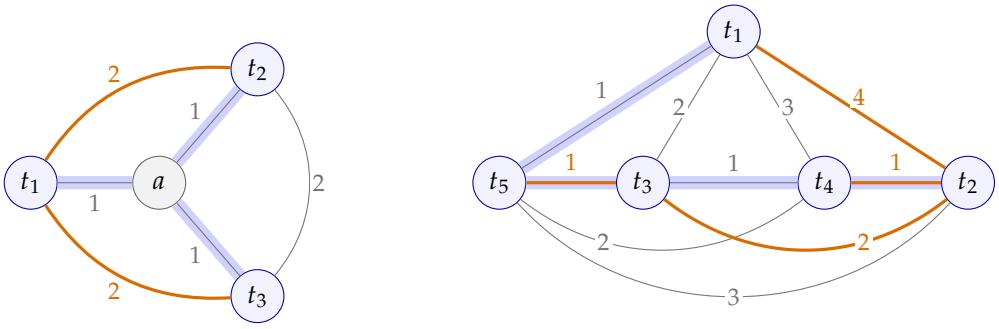


Figure 31: Two metric instances with terminals arriving in index order. Orange edges form one possible greedy solution; edges with a pale blue band form the optimal Steiner tree. Left: greedy pays $2 + 2 = 4$ and the optimum (the three spokes through the non-terminal a) costs 3. Right: greedy pays $4 + 2 + 1 + 1 = 8$ and the optimal path t_1–t_5–t_3–t_4–t_2 costs 4.

In the left instance of Figure 31, t_1 needs no action, t_2 is connected by the edge (t_1, t_2) of cost 2, and t_3 by an edge of cost 2 to t_1 or t_2. In the right instance, t_2 is connected by (t_1, t_2) of cost 4; for t_3 the edges (t_1, t_3) and (t_2, t_3) tie at cost 2; and t_4 and t_5 each have two unit-cost options. Hence greedy is not better than 2-competitive. In fact it is not constant-competitive.

Proposition 13.5. *The worst-case competitive ratio of the greedy online Steiner tree algorithm is $\Omega(\log k)$, where k is the number of terminals; extending the right instance above gives the bad examples.*

Proposition 13.6. *The worst-case competitive ratio of every online Steiner tree algorithm, deterministic or randomized, is $\Omega(\log k)$.*

13.3 Analysis of the greedy algorithm

Let $H_{k-1} = \sum_{i=1}^{k-1} \frac{1}{i}$ be the $(k-1)$th harmonic number.

Theorem 13.7. *The greedy online Steiner tree algorithm is $2H_{k-1}$-competitive. Since $H_{k-1} \le 1 + \ln k$, it is $O(\log k)$-competitive.*

By Proposition 13.6, greedy is therefore an optimal online algorithm in the worst case, up to a constant factor. The theorem follows from a key lemma relating the greedy costs to the optimal cost OPT in hindsight.

Lemma 13.8 [Key lemma]. *For every $i = 1, 2, \dots, k-1$, the ith most expensive edge of the greedy solution T has cost at most $2\,OPT/i$.*

So the most expensive greedy edge costs at most $2\,OPT$, the second most at most OPT, the third most at most $2\,OPT/3$, and so on.

Proof of Theorem 13.7. Greedy adds exactly one edge in each of the $k-1$ iterations after the first. Sorting these edges from most to least expensive and applying Lemma 13.8 to each gives

$$\text{greedy cost} \le \sum_{i=1}^{k-1} \frac{2\,OPT}{i} = 2H_{k-1} \cdot OPT.$$

Comparing the sum with an integral, $H_{k-1} \le 1 + \int_1^k \frac{dx}{x} = 1 + \ln k$. □

The proof of the key lemma uses two tricks, *tree doubling* and *shortcutting*. Recall that a connected multigraph (parallel copies of an edge allowed) in which every vertex has even degree is Eulerian, and has an Euler tour: a closed walk, not necessarily simple, that uses every edge exactly once. Even degrees are clearly necessary, since a tour visiting a vertex r times uses $2r$ of its edges; sufficiency follows by peeling off cycles one by one until the graph is empty.

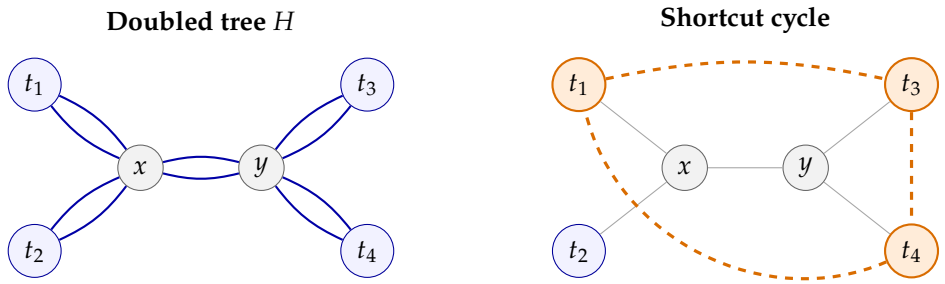


Figure 32: Tree doubling and shortcutting, schematic. Left: an optimal Steiner tree with non-terminals x, y, every edge doubled; one Euler tour is $t_1, x, t_2, x, y, t_3, y, t_4, y, x, t_1$. Right: shortcutting the tour to the chosen terminals t_1, t_3, t_4 (in order of first occurrence) gives the dashed cycle, which costs at most the tour.

Proof of Lemma 13.8. Let T^* be an optimal Steiner tree, of cost $OPT = \sum_{e \in T^*} c_e$. Obtain H from T^* by adding a second copy of every edge. Every degree doubles, so H is Eulerian, and $\sum_{e \in H} c_e = 2\,OPT$. An Euler tour C of H visits every terminal at least once, possibly several times, possibly together with non-terminals, and its cost is $2\,OPT$.

The CONNECTION COST of a terminal t_j with $j > 1$ is the cost of the edge added by greedy when t_j arrived. These edges are exactly the edges of T, so the ith most expensive edge of T is the cheapest connection cost among the i terminals $s_1, \ldots, s_i$ with the largest connection costs. It suffices to show that some s_j has connection cost at most $2\,OPT/i$.

For $i = 1$, the edge (t_1, s_1) was available when s_1 arrived, and by the triangle inequality it costs at most the t_1–s_1 path in T^*, hence at most OPT. Now let $i \geq 2$. Shortcut C to a cycle C_i on $\{s_1, \ldots, s_i\}$ that visits these terminals in the order of their first occurrences on C (for $i = 2$, C_i uses the edge (s_1, s_2) twice). Each edge of C_i replaces a subwalk of C between its endpoints, so by the triangle inequality $\sum_{e \in C_i} c_e \leq \sum_{e \in C} c_e = 2\,OPT$. Since C_i has i edges,

$$\underbrace{\min_{e \in C_i} c_e}_{\text{cheapest edge}} \quad \leq \quad \underbrace{\frac{1}{i} \sum_{e \in C_i} c_e}_{\text{average edge}} \quad \leq \frac{2\,OPT}{i}.$$

Let (s_h, s_j) be such a cheapest edge, with s_j arriving after s_h. When s_j arrived, (s_h, s_j) was one option for connecting it to a previous terminal, so greedy connected s_j at cost at most $c_{s_h s_j} \leq 2\,OPT/i$. □

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 13: Online Scheduling and Online Steiner Tree.

Lecture 14: Online Bipartite Matching

In the online bipartite matching problem,

- the left-hand vertices L are known up front;
- the right-hand vertices R arrive online, one at a time, and a vertex $w \in R$ arrives together with all its incident edges (all its neighbors are in L);
- a new vertex $w \in R$ can be matched only immediately upon arrival, and the decision is irrevocable.

The goal is a matching of maximum cardinality. The benchmark OPT is the maximum matching in hindsight, which could be computed (for example, by max flow) if all of R were known. For this maximization problem, an online algorithm is c-competitive if on every input its matching has size at least $c \cdot OPT$. In the motivating application, Web advertising, L consists of the ad impressions advertisers have purchased (an advertiser who bought 5000 views contributes 5000 vertices), each arriving search query or page view is a vertex of R, and its edges go to the advertisers whose targeting criteria it meets.

The algorithms below never mention linear programs. Their near-optimality is nevertheless proved by exhibiting a feasible solution to the dual of the maximum matching problem.

14.1 Deterministic algorithms

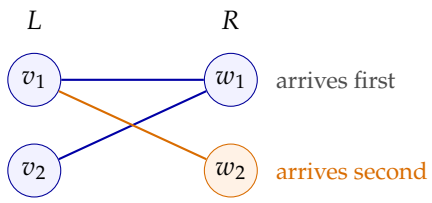


Figure 33: No deterministic online algorithm has competitive ratio better than $\frac{1}{2}$. The vertex w_1 arrives adjacent to v_1 and v_2; if w_1 is matched to v_1, the next vertex w_2 is adjacent only to v_1 (shown), and symmetrically otherwise.

In Figure 33, every deterministic algorithm matches w_1 to v_1 or to v_2 (matching it to neither has ratio 0 if nothing else arrives). The adversary then sends w_2 adjacent only to the vertex that is already matched. The algorithm's matching has one edge while the best matching in hindsight has two, so no deterministic algorithm is better than $\frac{1}{2}$-competitive.

The obvious greedy algorithm: when $w \in R$ arrives, match it to an arbitrary unmatched neighbor,

or to no one if it has none.

The analysis uses the primal-dual pair for maximum-cardinality matching,

$$\text{(P)} \qquad \max \sum_{e \in E} x_e \qquad \text{s.t.} \sum_{e \in \delta(v)} x_e \le 1 \;\; \forall v \in L \cup R, \quad x_e \ge 0 \;\; \forall e \in E,$$

$$\text{(D)} \qquad \min \sum_{v \in L \cup R} p_v \quad \text{s.t.}\; p_v + p_w \ge 1 \;\; \forall (v,w) \in E, \quad p_v \ge 0 \;\; \forall v \in L \cup R.$$

Matchings need not be perfect, so the primal constraints are inequalities and the dual variables are nonnegative. By weak duality, every feasible solution of (D) has objective value at least OPT.

Proposition 14.1. *The deterministic greedy algorithm is $\frac{1}{2}$-competitive.*

Proof. For the analysis only, define a PRE-DUAL solution: $q_v = \frac{1}{2}$ if greedy matches v and $q_v = 0$ otherwise. Equivalently, start with $q \equiv 0$ and, whenever greedy adds an edge (v, w), set q_v and q_w to $\frac{1}{2}$; each vertex is set at most once since the edges form a matching. Hence, for the output matching M,

$$|M| = \sum_{v \in L \cup R} q_v. \tag{14.1}$$

For every edge (v, w) of the final graph, $q_v = \frac{1}{2}$ or $q_w = \frac{1}{2}$. Indeed, if $q_v = 0$ then v was never matched, so w had an unmatched neighbor when it arrived, and greedy matched w. Thus $p = 2q$ is feasible for (D), and by (14.1) and weak duality

$$|M| = \frac{1}{2} \sum_{v \in L \cup R} p_v \ge \frac{1}{2} \cdot OPT. \qquad \square$$

14.2 Online fractional bipartite matching

In the fractional version, the algorithm maintains a feasible solution x of (P); when $w \in R$ arrives it irrevocably chooses the values x_e of the edges incident to w. Offline, fractions do not help, since the fractional and integral maximum matchings of a bipartite graph have the same size. Online, they let the algorithm *hedge*. In Figure 33, a fractional algorithm can match w_1 half to v_1 and half to v_2; then w_2, adjacent to only one of them, can still be matched with value $\frac{1}{2}$, for a total of $\frac{3}{2}$ against $OPT = 2$. Since w_2 can always be made adjacent only to the more loaded of v_1, v_2, which has received at least $\frac{1}{2}$, this example shows only that no fractional online algorithm is better than $\frac{3}{4}$-competitive, leaving room to beat $\frac{1}{2}$.

The WATER LEVEL (WL) algorithm is a natural way to hedge. Think of each $v \in L$ as a water container of capacity 1, with current WATER LEVEL $y_v = \sum_{e \in \delta(v)} x_e$, and of each $w \in R$ as a source of one unit of water.

Algorithm 14.1: Water level (WL) algorithm

Input: Vertices L up front; vertices $w \in R$ arriving online with their edges.
Output: A fractional matching x.

```
x ← 0
for each arriving w ∈ R do
    while some neighbor of w is not full and w has not sent its whole unit do
        Send water from w, at equal rates, to the neighbors of w with the lowest current water level
    // stops when (i) all neighbors of w are full, or (ii) w is empty
```

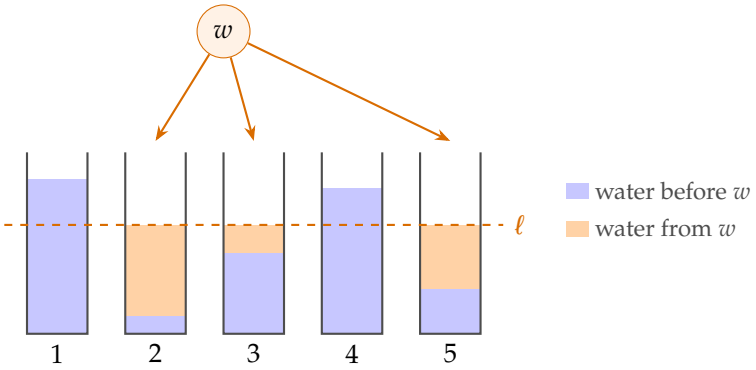


Figure 34: Schematic of an arrival in the WL algorithm. The new vertex w first fills the lowest container (2), then fills containers 2 and 5 at equal rates once their levels meet, then 2, 5, and 3 together. Here w sends its whole unit (case (ii)) and stops at the common level ℓ; containers 1 and 4, already above ℓ, receive nothing.

On the instance of Figure 33, WL reproduces the hedge: w_1 splits its water equally between v_1 and v_2 (case (ii)), and w_2 sends $\frac{1}{2}$ unit to its unique neighbor (case (i)). The analysis needs only one property of WL.

Lemma 14.2 [Key property of WL]. *Let $(v, w) \in E$ be an edge of the final graph with $v \in L$, and let $y_v = \sum_{e \in \delta(v)} x_e$ be the final water level of v. Then w sent water only to containers whose current water level was at most y_v.*

Proof. This is trivial if $y_v = 1$, so suppose $y_v < 1$. Container v is a neighbor of w that never became full, so case (i) was not triggered when w arrived, and w routed all its water. When this transfer was complete, all containers that received water from w had a common level ℓ, and all other neighbors of w had level at least ℓ (Figure 34). Water levels only increase, so every neighbor of w, including v, ends with level at least ℓ; thus $\ell \le y_v$. Since w sent water only to containers at level at most ℓ, the lemma follows. □

14.3 Analysis: a false start

Mimic the integral analysis: start with $q \equiv 0$, and whenever the water x_{vw} sent from w to v increases by Δ, increase both q_v and q_w by $\Delta/2$. At all times the value of the fractional matching equals $\sum_{v \in L \cup R} q_v$. If $p = \frac{1}{c} q$ were feasible for (D) for some $c > \frac{1}{2}$, then WL would be c-competitive, because its value would be $c \sum_v p_v \geq c \cdot OPT$.

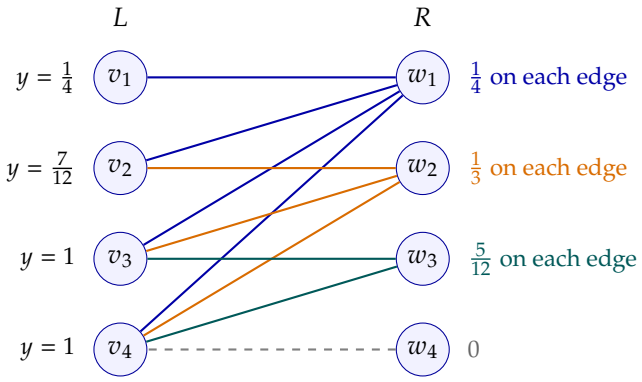


Figure 35: A bad example for the equal-split analysis. The vertices $w_1, \ldots, w_4$ arrive in order; each sends equal amounts on all its edges, as labeled, and the final water levels are shown on the left. The vertex w_3 fills v_3 and v_4 (case (i)) and w_4 sends nothing.

This fails on the example of Figure 35. With four vertices in L, w_1 is adjacent to all of them and gives each $\frac{1}{4}$. Then w_2, adjacent to v_2, v_3, v_4, raises their levels to $\frac{1}{4} + \frac{1}{3} = \frac{7}{12}$. Next w_3, adjacent to v_3, v_4, sends $\frac{5}{12}$ to each, filling both, and cannot send the rest. Finally w_4, adjacent only to the full v_4, sends nothing. On the edge (v_4, w_4), $q_{v_4} = \frac{1}{2}$ (as large as this scheme allows) and $q_{w_4} = 0$, so q must be scaled up by 2 to become feasible, which gives no improvement over $\frac{1}{2}$. Yet WL itself is not bad here: its value is $1 + 1 + \frac{5}{6} + 0 = \frac{17}{6}$, against $OPT = 4$. The weakness is in the analysis.

14.4 Analysis with a splitting function

Since w_4 never receives any q-value, beating $\frac{1}{2}$ requires v_4 to end with q-value above $\frac{1}{2}$. Crediting only vertices of L fails if w_1 is the only arrival (left q-values $\frac{1}{4}$, right ones 0). The compromise is to split each increase unequally, giving v a larger share as v becomes fuller. The algorithm is still WL; only the analysis changes.

Fix a nondecreasing SPLITTING FUNCTION $g : [0,1] \to [0,1]$. Start with $q \equiv 0$. Whenever x_{vw} increases by an infinitesimal amount dz while the current level of v is y_v,

- increase q_v by $g(y_v)\, dz$, and
- increase q_w by $(1 - g(y_v))\, dz$.

The constant functions $g \equiv 0$, $g \equiv 1$, and $g \equiv \frac{1}{2}$ credit only R, only L, and both equally (the false

start). For every g, the value of the WL fractional matching equals $\sum_{v \in L \cup R} q_v$ at all times. Fix an edge (v, w) of the final graph; we bound $q_v + q_w$ from below.

Case 1: v ends full, $y_v = 1$. While its level was z, v accrued q-value at rate $g(z)$, so

$$q_v + q_w \geq q_v = \int_0^1 g(z)\, dz. \tag{14.2}$$

Discarding q_w is not wasteful: in Figure 35, v_4 is full and its neighbor w_4 has q-value 0.

Case 2: $y_v < 1$. Then w routed its entire unit of water, since otherwise WL would have sent more water to the non-full container v. By Lemma 14.2, every container that received water from w had level at most y_v at that time, so, g being nondecreasing, w accrued q-value at rate at least $1 - g(y_v)$ throughout. Hence $q_w \geq 1 - g(y_v)$, and with $q_v = \int_0^{y_v} g(z)\, dz$,

$$q_v + q_w \geq \int_0^{y_v} g(z)\, dz + 1 - g(y_v). \tag{14.3}$$

Larger g helps (14.2) and smaller g helps (14.3). For $g \equiv 0$ the bounds are 0 and 1; for $g \equiv 1$ the first is 1 and the second can be 0; for $g \equiv \frac{1}{2}$ both are at least $\frac{1}{2}$. If both right-hand sides are at least c for every $y_v \in [0, 1]$, then $p = \frac{1}{c} q$ is feasible for (D) and WL is c-competitive.

14.5 The optimal splitting function

Guess that an optimal g leaves no slack: the bound (14.3) is the same for every $y_v \in [0, 1]$, and equals the bound (14.2). The first guess makes the right-hand side of (14.3) constant in $y = y_v$, so its derivative vanishes:

$$g(y) - g'(y) = 0, \qquad \text{so} \qquad g(z) = ke^z \ \text{ for a constant } k > 0.$$

With $g(z) = ke^z$, the two bounds become

$$\int_0^1 ke^z\, dz = k(e - 1) \qquad \text{and} \qquad \int_0^y ke^z\, dz + 1 - ke^y = k(e^y - 1) + 1 - ke^y = 1 - k.$$

The first increases and the second decreases in k. Equating them gives $k(e - 1) = 1 - k$, so $k = \frac{1}{e}$ and $g(y) = e^{y-1}$. This g is nondecreasing with values in $[\frac{1}{e}, 1]$: an empty container gets a $\frac{1}{e}$ share of each increase, and its share approaches 100% as it fills. Both bounds then equal $1 - \frac{1}{e}$.

Theorem 14.3. *The WL algorithm is $\left(1 - \frac{1}{e}\right)$-competitive for online fractional bipartite matching; note $1 - \frac{1}{e} \approx 63.2\%$.*

Proof. With $g(y) = e^{y-1}$, both (14.2) and (14.3) give $q_v + q_w \geq 1 - \frac{1}{e}$ for every edge (v, w). So $p = q/(1 - \frac{1}{e})$ is feasible for (D), and the value of the WL matching is $\sum_v q_v = (1 - \frac{1}{e}) \sum_v p_v \geq (1 - \frac{1}{e})\, OPT$ by weak duality. □

The same ideas give a randomized online algorithm for the original integral problem whose matching has expected size at least $(1 - \frac{1}{e})\, OPT$, the expectation being over the algorithm's coin flips. Roughly, the probability that an edge is in the matching plays the role of its fractional value in WL. This is best possible: no online algorithm, deterministic or randomized, has competitive ratio better than $1 - \frac{1}{e}$ for maximum bipartite matching.

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 14: Online Bipartite Matching.

Lecture 15: Introduction to Approximation Algorithms

Many important problems are NP-hard, so they do not seem to be solvable in polynomial time. NP-hardness is not a death sentence, but it forces compromises, and more computational and human effort generally buys better solutions. Three compromises are common.

1. *Restrict to a relevant special case.* It may be polynomial-time solvable (vertex cover is NP-hard in general graphs but reduces to max flow/min cut in bipartite graphs), or still NP-hard but easier than the general case (metric TSP, Lecture 16). This requires identifying and exploiting the special structure of the application.
2. *Spend super-polynomial time,* but beat brute-force search. Many NP-complete problems have exponential-time algorithms much faster than brute force, such as $(4/3)^n$ rather than 2^n for 3SAT and 2^n rather than $n!$ for TSP.
3. *Relax correctness:* settle for a feasible solution that is approximately optimal.

15.1 Approximation algorithms

An approximation algorithm must run in polynomial time on every input; assuming P $\neq$ NP, it is then necessarily suboptimal on some inputs of an NP-hard problem, and the goal is to be as close to optimal as possible on every input. For a minimization problem, an α-approximation algorithm always returns a feasible solution of cost at most α times the minimum; for a maximization problem, value at least α times the maximum. The factor α is the approximation ratio, playing the role of the competitive ratio in online algorithms. Greedy algorithms, divide and conquer, dynamic programming, and linear programming all remain useful, as does local search.

Example: *Minimum-makespan scheduling.* Given m identical machines and n jobs with processing times $p_1, \ldots, p_n$, all known up front, schedule the jobs to minimize the makespan (the maximum machine load). This offline problem is NP-hard. Graham's algorithm (Lecture 13), which assigns each job in turn to the currently least-loaded machine, runs in polynomial time and has makespan at most twice optimal, so it is a 2-approximation algorithm; processing jobs online is now just a bonus. An offline algorithm can also sort the jobs: running Graham's algorithm on the jobs from largest to smallest improves the approximation ratio to $\frac{4}{3}$.

Example: *Knapsack.* Given n items with values and weights and a capacity W, find a subset of maximum total value with total weight at most W. With integer weights, dynamic programming solves it in $O(nW)$ time. This is not polynomial, since the input size is only $O(n \log W)$, and indeed the problem is NP-hard; the dynamic program beats brute-force search unless W is exponential in n. In polynomial time, the greedy algorithm that processes items in order of value per unit weight, and returns the better of its solution and the single most valuable item, is a $\frac{1}{2}$-approximation. Rounding

the data (discarding lower-order bits) and then running dynamic programming on the instance with small numbers gives a $(1-\epsilon)$-approximation in time polynomial in n and $\frac{1}{\epsilon}$, for any $\epsilon > 0$: arbitrarily close approximation in polynomial time.

15.2 Metric Steiner tree

In the offline STEINER TREE PROBLEM, the input is a complete graph $G = (V, E)$ with nonnegative edge costs satisfying the triangle inequality $c_{uw} \leq c_{uv} + c_{vw}$ (without loss of generality, as in Lecture 13), and a set $R = \{t_1, \ldots, t_k\}$ of terminals, all known up front. The goal is a minimum-cost subgraph spanning the terminals. The online greedy algorithm of Lecture 13 runs in polynomial time, so its guarantee of $2H_{k-1} = O(\log k)$ times optimal makes it an $O(\log k)$-approximation algorithm.

Knowing all terminals allows a favorable order. The most natural one always processes next the terminal that is cheapest to connect to a previous terminal; this is exactly Prim's algorithm on the subgraph induced by the terminals.

Definition: The MST HEURISTIC for metric Steiner tree outputs a minimum spanning tree of the subgraph induced by the terminals.

It runs in polynomial time, so it is suboptimal on some instances.

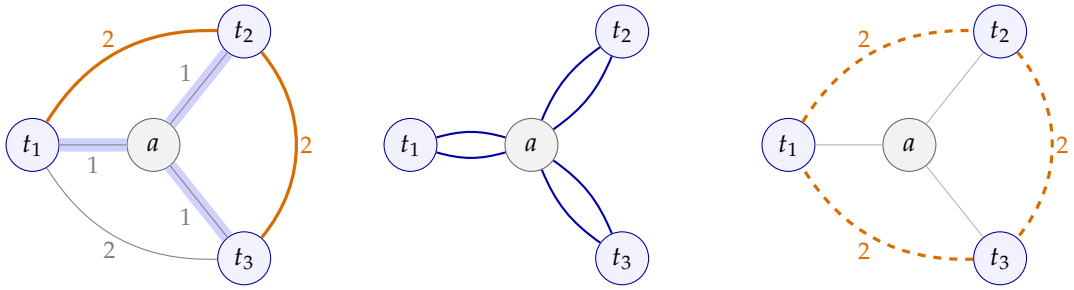


Figure 36: (a) The MST of the terminals t_1, t_2, t_3 (orange) costs 4, while the optimal Steiner tree (pale blue band) uses the non-terminal a and costs 3. (b) Doubling every edge of the optimal tree gives an Eulerian graph H of cost 6. (c) Shortcutting an Euler tour of H gives the cycle t_1, t_2, t_3 of cost 6; deleting one of its edges leaves a spanning tree of the terminals of cost $4 \leq 2 \cdot 3$.

In Figure 36(a), the cost can be decreased by spanning an additional vertex, which is what makes the problem hard. Larger "wheel" graphs of the same type show that the MST heuristic can be off by a factor arbitrarily close to 2. There are no worse examples.

Theorem 15.1. *In the metric Steiner tree problem, the minimum spanning tree of the terminals costs at most twice an optimal Steiner tree.*

Proof. This is the analysis of Lecture 13 done in one shot instead of edge by edge. Let T^* be an optimal Steiner tree, of cost OPT, and obtain H by adding a second copy of every edge. Every degree doubles, so H is Eulerian, with an Euler tour C (a closed walk using every edge of H exactly once) of cost $2\,OPT$. The tour visits each of $t_1, \ldots, t_k$ at least once. Shortcutting it to a simple cycle $\widehat{C}$ on $\{t_1, \ldots, t_k\}$ only decreases the cost, by the triangle inequality. Then $\widehat{C}$ minus an edge is a spanning tree of the subgraph induced by R with cost at most $2\,OPT$, and the MST can only be cheaper. □

15.3 Set coverage

Definition 15.2 [Set coverage]. The input is a collection $S_1, \ldots, S_m$ of subsets of a ground set U, each given as a list of its elements, and a budget k. The goal is to pick k of the subsets to maximize the size of their union.

Bigger sets are better, all else being equal, but some sets are largely redundant while others are uniquely useful.

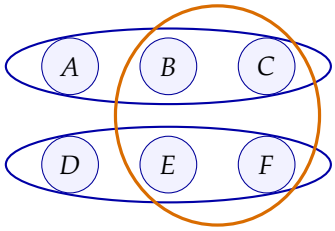


Figure 37: A set coverage instance with $k = 2$. The two horizontal sets (blue) cover all six elements. The largest set (orange) has four elements but is redundant with either of them.

Examples include hiring k people, each a set of skills, to maximize the number of distinct skills; choosing k locations for factories, fire stations, or Web caches to cover as many neighborhoods as possible; and choosing a few features to explain as much of the data as possible. The problem is NP-hard. The natural greedy algorithm increases the size of the union as much as possible at each step.

Algorithm 15.1: Greedy algorithm for set coverage

Input: Subsets $S_1, \ldots, S_m$ of U and a budget k.
Output: k of the subsets.

for $i = 1, 2, \ldots, k$ **do**
  $A_i \leftarrow$ a set maximizing the number of new elements covered, relative to $\bigcup_{j=1}^{i-1} A_j$
return $\{A_1, \ldots, A_k\}$

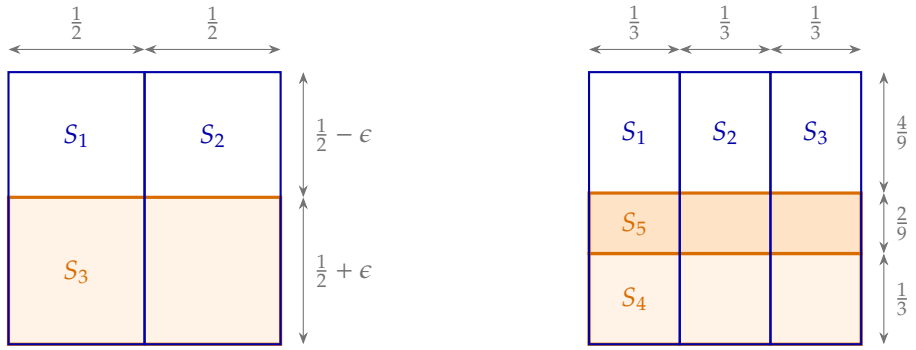


Figure 38: Bad examples for greedy, with U drawn as a unit square and sizes as fractions of $|U|$. (a) $k = 2$: the columns S_1, S_2 are optimal, and the row S_3 covers slightly more than half. (b) $k = 3$: the columns S_1, S_2, S_3 are optimal; the row S_4 covers slightly more than $\frac{1}{3}$ and the row S_5 slightly more than $\frac{2}{9}$.

In Figure 38(a), S_1 and S_2 partition U in half, so $OPT = |U|$. Greedy first picks S_3, which covers slightly more than half, and then either S_1 or S_2, for a total of about $\frac{3}{4}|U|$. In (b), S_1, S_2, S_3 partition U into thirds. The set S_4 covers slightly more than $\frac{1}{3}$ and meets each of S_1, S_2, S_3 evenly, so greedy picks it first. Each of S_1, S_2, S_3 would then add $\frac{1}{3} \cdot \frac{2}{3}|U| = \frac{2}{9}|U|$, so a set S_5 disjoint from S_4 and covering slightly more than $\frac{2}{9}$ is picked second. Third, greedy picks one of S_1, S_2, S_3, for a total of about

$$|U|\left(\frac{1}{3} + \frac{2}{9} + \frac{1}{3} \cdot \frac{4}{9}\right) = \frac{19}{27}|U|,$$

roughly 70% of optimal. Extending this family, for every k the greedy solution can be as small as $1 - (1 - \frac{1}{k})^k$ times optimal; this equals $\frac{3}{4}$ and $\frac{19}{27}$ for $k = 2, 3$, decreases with k, and tends to $1 - \frac{1}{e} \approx 63.2\%$, since $1 - x \approx e^{-x}$ for small x.

Theorem 15.3. *For every $k \geq 1$, the greedy algorithm is a $\left(1 - (1 - \frac{1}{k})^k\right)$-approximation algorithm for set coverage with budget k.*

So there are no worse examples than those above. Moreover, under standard complexity assumptions, no polynomial-time algorithm has a better approximation ratio as k grows (for constant k, brute force is polynomial), so greedy is an optimal approximation algorithm for set coverage. The analysis shows that greedy makes healthy progress at every step.

Lemma 15.4. *Suppose the first $i - 1$ sets $A_1, \ldots, A_{i-1}$ chosen by greedy cover ℓ elements. Then A_i covers at least $\frac{1}{k}(OPT - \ell)$ new elements, where OPT is the optimal value.*

Proof. If greedy could pick k new sets in this iteration, the k sets of an optimal solution would cover at least $OPT - \ell$ new elements. So one of them covers at least $\frac{1}{k}(OPT - \ell)$ new elements, and greedy's choice A_i covers at least as many. □

Proof of Theorem 15.3. Let $g_i = |\bigcup_{j=1}^{i} A_j|$, with $g_0 = 0$. By Lemma 15.4,

$$g_i = (g_i - g_{i-1}) + g_{i-1} \geq \frac{1}{k}(OPT - g_{i-1}) + g_{i-1} = \frac{OPT}{k} + \left(1 - \frac{1}{k}\right) g_{i-1}.$$

Applying this for $i = k, k-1, \ldots, 1$ gives one term per iteration,

$$g_k \geq \frac{OPT}{k}\left[1 + \left(1 - \frac{1}{k}\right) + \left(1 - \frac{1}{k}\right)^2 + \cdots + \left(1 - \frac{1}{k}\right)^{k-1}\right].$$

With the geometric sum $1 + z + \cdots + z^{k-1} = \frac{1-z^k}{1-z}$ for $z = 1 - \frac{1}{k}$, so that $1 - z = \frac{1}{k}$,

$$g_k \geq \frac{OPT}{k} \cdot \frac{1 - (1 - \frac{1}{k})^k}{1/k} = OPT\left[1 - \left(1 - \frac{1}{k}\right)^k\right]. \qquad \square$$

15.4 Influence maximization

In a social network, a directed graph $G = (V, E)$, an edge (v, w) means that v influences w (for example, w follows v). In the CASCADE MODEL of how an idea goes viral, with a parameter p:

- initially the vertices of a set S are active, all others inactive, and every edge is undetermined;
- while there is an active vertex v with an undetermined edge (v, w), mark the edge active with probability p and inactive otherwise; if it is active and w is inactive, activate w.

An activated vertex stays active and gets a chance to activate each vertex it influences, and a vertex gets one chance per activated influencer. A vertex ends up active exactly when an active vertex reaches it by a path of active edges.

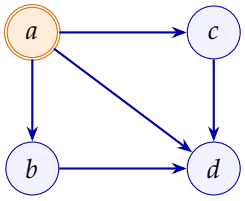


Figure 39: A cascade started from a (double circle). Each of b and c is activated by a with probability p; d gets a chance from each of a, b, and c that becomes active.

The INFLUENCE MAXIMIZATION PROBLEM: given G and a budget k, find $S \subseteq V$ with $|S| = k$ maximizing $f(S)$, the expected number of active vertices at the end of the cascade (over the coin flips for the edges) when exactly S is active initially.

Algorithm 15.2: Greedy algorithm for influence maximization

Input: Directed graph $G = (V, E)$, a budget k, and the function f.
Output: A set $S \subseteq V$ with $|S| = k$.

```
S ← ∅
for i = 1, 2, ..., k do
    Add to S a vertex v maximizing f(S ∪ {v})
return S
```

The set coverage analysis shows that this is also a $\left(1 - (1 - \frac{1}{k})^k\right)$-approximation algorithm. The guarantee holds for every MONOTONE and SUBMODULAR set function f (adding elements never decreases f, and the gain from adding an element can only shrink as the set grows), and the influence function is one, being essentially a convex combination of set coverage functions.

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 15: Introduction to Approximation Algorithms.

Lecture 16: The Traveling Salesman Problem

The input to the traveling salesman problem (TSP) is a complete undirected graph $G = (V, E)$ with a nonnegative cost $c_e \geq 0$ on each edge. A TSP tour is a simple cycle visiting each vertex exactly once (not to be confused with an Euler tour, which uses each edge exactly once). The goal is a TSP tour of minimum total cost. Applications include ordering tasks to minimize total setup cost between consecutive tasks, and ordering outstanding disk read requests.

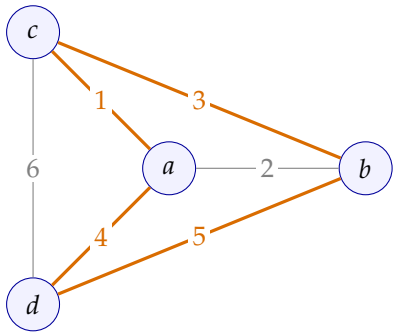


Figure 40: A TSP instance on four vertices. The optimal tour a–c–b–d–a (orange) costs $1 + 3 + 5 + 4 = 13$; the other two tours cost 15 and 14.

16.1 Inapproximability of general TSP

Recall that an α-approximation algorithm for a minimization problem runs in polynomial time and always returns a feasible solution of cost at most α times the minimum.

Theorem 16.1. *If* P $\neq$ NP*, then there is no α-approximation algorithm for the TSP, for any α.*

The proof reduces from the NP-complete Hamiltonian cycle problem: given an undirected graph, does it contain a simple cycle visiting every vertex exactly once?

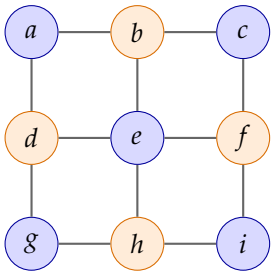


Figure 41: The 3×3 grid has no Hamiltonian cycle. Every edge joins a blue vertex (a corner or the center) to an orange one, so a cycle alternates colors and has equally many of each; a Hamiltonian cycle would contain all 5 blue and 4 orange vertices.

Proof. Given an instance $G = (V, E)$ of Hamiltonian cycle with $n = |V|$, build a TSP instance G' on the complete graph with vertex set V and costs

$$c_e = \begin{cases} 1 & \text{if } e \in E, \\ > \alpha n & \text{if } e \notin E. \end{cases}$$

The Hamiltonian cycles of G correspond one-to-one to the TSP tours of G' using only unit-cost edges. Hence

(i) if G has a Hamiltonian cycle, some TSP tour costs n;

(ii) if G has no Hamiltonian cycle, every TSP tour costs more than αn.

An α-approximation algorithm run on G' would return a tour of cost at most αn in case (i), and can only return one of cost more than αn in case (ii). So it would decide in polynomial time whether G is Hamiltonian. □

16.2 Metric TSP and the MST heuristic

In the METRIC TSP the edge costs satisfy the triangle inequality, $c_{uw} \le c_{uv} + c_{vw}$ for all $u, v, w \in V$. For Steiner tree this assumption was without loss of generality; for TSP it makes the problem significantly easier. Metric TSP is still NP-hard: in the reduction above, give the non-edges cost 2 instead. The triangle inequality then holds because every edge costs at most 2 and every two-hop path at least 2. The optimal tour costs n if G is Hamiltonian and at least $n + 1$ otherwise, so there is no exact polynomial-time algorithm unless P = NP, but this does not rule out good approximation.

The analysis needs a bound that is "only better than optimal."

Lemma 16.2. *For every instance $G = (V, E, c)$, the minimum cost of a TSP tour is at least the cost of a minimum spanning tree (MST).*

Proof. Removing an edge from an optimal tour leaves a spanning tree of no greater cost, and the MST is no more expensive than that. □

To turn the MST into a tour, reuse doubling and shortcutting from the Steiner tree analyses (Lectures 13 and 15). There they appeared only in the analysis; here they are steps of the algorithm, since the output must be a tour rather than a tree.

Algorithm 16.1: MST heuristic for metric TSP

Input: Complete graph $G = (V, E)$ with metric costs c.
Output: A TSP tour.

```
Compute an MST T of G
Construct H by doubling every edge of T
Compute an Euler tour C of H                    // visits every v ∈ V at least once
Shortcut repeated occurrences of vertices in C to obtain a TSP tour
```

Every step runs in polynomial (even near-linear) time.

Theorem 16.3. *The MST heuristic is a 2-approximation algorithm for metric TSP.*

Proof.

$$\text{cost of the tour} \le \text{cost of } C = \sum_{e \in H} c_e = 2 \sum_{e \in T} c_e \le 2 \cdot \text{cost of an optimal tour.}$$

The first inequality is the triangle inequality (each shortcut replaces a subwalk by a direct edge), the equalities hold because C uses each edge of H exactly once and H doubles T, and the last inequality is Lemma 16.2. □

This analysis is tight: for every constant $c < 2$ there is a metric instance on which the MST heuristic's tour costs more than c times optimal.

16.3 Christofides's algorithm

The factor 2 comes from doubling every edge of T to make it Eulerian. A cheaper way to fix the parities rests on a second lower bound.

Lemma 16.4. *Let $G = (V, E)$ be a metric TSP instance, let $S \subseteq V$ have even size, and let M be a minimum-cost perfect matching of the complete graph induced by S. Then*

$$\sum_{e \in M} c_e \le \frac{1}{2} \cdot OPT,$$

where OPT is the cost of an optimal TSP tour.

Proof. Shortcut an optimal tour C^* to a tour C_S of S; by the triangle inequality, C_S costs at most OPT. Since $|S|$ is even, C_S is a simple cycle of even length, so coloring its edges alternately splits it into two disjoint perfect matchings of S (Figure 42). Their costs sum to at most OPT, so the cheaper one costs at most $OPT/2$, and M costs no more. □

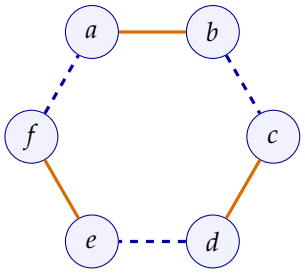


Figure 42: An even cycle C_S on $S = \{a, \dots, f\}$ is the union of two disjoint perfect matchings (solid orange and dashed blue).

Algorithm 16.2: Christofides's algorithm

Input: Complete graph $G = (V, E)$ with metric costs c.
Output: A TSP tour.

1 Compute an MST T of G
2 Compute the set W of vertices with odd degree in T
3 Compute a minimum-cost perfect matching M of W
4 Construct H by adding M to T
5 Compute an Euler tour C of H *// visits every $v \in V$ at least once*
6 Shortcut repeated occurrences of vertices in C to obtain a TSP tour

The set W has even size, since the degrees of a graph sum to twice the number of edges, so the number of odd-degree vertices is even. The matching instance is the complete graph on W, which is not bipartite when $|W| \geq 4$; this nonbipartite matching problem is still solvable in polynomial time (extending the ideas of the Hungarian algorithm), and it dominates the running time. An edge in both T and M appears twice in H, which is fine. Adding M raises the degree of each vertex of W by exactly one and leaves the others unchanged, so all degrees of H are even; H is connected because T is. So H is Eulerian.

Theorem 16.5. *Christofides's algorithm is a $\frac{3}{2}$-approximation algorithm for metric TSP.*

Proof.

$$\text{cost of the tour} \leq \text{cost of } C = \sum_{e \in H} c_e = \underbrace{\sum_{e \in T} c_e}_{\leq OPT} + \underbrace{\sum_{e \in M} c_e}_{\leq OPT/2} \leq \frac{3}{2} \cdot OPT,$$

by the triangle inequality, the definitions of C and H, and Lemmas 16.2 and 16.4. □

This analysis is also tight: for every constant $c < \frac{3}{2}$ there is a metric instance on which Christofides's algorithm outputs a tour costing more than c times optimal.

16.4 Asymmetric TSP

In the ASYMMETRIC TSP (ATSP), the input is a complete directed graph, with an edge in each direction between each pair of vertices and a nonnegative cost $c_e \geq 0$ on each edge; opposite edges may have different costs, as when the setup time from task i to task j differs from that from j to i. The goal is a minimum-cost DIRECTED TSP TOUR, a simple directed cycle visiting each vertex exactly once. The ordinary TSP is the special case where opposite edges have equal costs, so ATSP is at least as hard. We again assume the triangle inequality, $c_{uw} \leq c_{uv} + c_{vw}$ for all $u, v, w \in V$, which makes sense for directed graphs too.

The strategy mirrors the metric TSP algorithms:

1. construct a not-too-expensive Eulerian directed graph H;
2. shortcut an Euler tour of H to a directed TSP tour, which by the triangle inequality costs at most $\sum_{e \in H} c_e$.

A directed graph H is EULERIAN if (i) it is strongly connected and (ii) every vertex has equal in-degree and out-degree. Every Eulerian directed graph has a DIRECTED EULER TOUR, a directed closed walk using every edge exactly once. Both conditions are clearly necessary, since a tour enters and exits each vertex equally often; sufficiency is proved as in the undirected case.

In place of the MST, the starting point is a cycle cover. A CYCLE COVER of a directed graph is a collection $C_1, \ldots, C_k$ of directed cycles, each with at least two vertices, such that every vertex lies in exactly one of them; the cycles partition the vertex set.

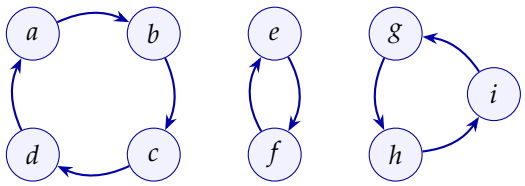


Figure 43: A cycle cover with three cycles: $a \to b \to c \to d \to a$, $e \to f \to e$, and $g \to h \to i \to g$.

Directed TSP tours are exactly the cycle covers with $k = 1$, which gives a lower bound.

Lemma 16.6. *For every instance $G = (V, E, c)$ of ATSP, the minimum cost of a directed TSP tour is at least the minimum cost of a cycle cover.*

A minimum-cost cycle cover can be computed in polynomial time, by a reduction to minimum-cost bipartite perfect matching.

Algorithm 16.3: Approximation algorithm for ATSP

Input: Complete directed graph $G = (V, E)$ with costs c satisfying the triangle inequality.
Output: A directed TSP tour.

```
F ← ∅; G′ ← G
while G′ has at least 2 vertices do
    Compute a minimum-cost cycle cover C_1, ..., C_k of G′
    Add the edges of C_1, ..., C_k to F
    for i = 1, 2, ..., k do
        Delete from G′ all but one vertex of C_i
Compute a directed Euler tour C of H = (V, F)
Shortcut repeated occurrences of vertices on C to obtain a TSP tour
```

Here G' always denotes the complete directed graph on the remaining vertices, with the original costs.

Proposition 16.7. *The graph $H = (V, F)$ constructed by the algorithm is Eulerian.*

Proof. H is the union of the cycle covers computed in all iterations. Two invariants hold throughout.
Degrees. In (V, F), every vertex has equal in-degree and out-degree. This holds for $F = \emptyset$. Each iteration adds a cycle cover of the remaining vertices, which raises the in- and out-degree of each of them by exactly 1; deleted vertices receive no further edges. At the end, both degrees of v equal the number of iterations in which v participated.
Reachability. Every deleted vertex can reach, and be reached from, some undeleted vertex in (V, F). A vertex v deleted in an iteration lies on a cycle C_i of that iteration's cover, $C_i \subseteq F$, and some vertex w of C_i stays undeleted; C_i contains directed paths from v to w and from w to v. A vertex u deleted earlier has, by induction, paths to and from a vertex v that remained at that time; if v is deleted now, stitching the paths through v and w gives paths from u to w and from w to u.
In the final iteration the cycle cover is a single cycle, since otherwise at least two vertices would remain and the loop would continue. That cycle lets the vertices remaining in the final iteration all reach one another, and by the second invariant every deleted vertex reaches and is reached by one of them. So (V, F) is strongly connected, and with the first invariant it is Eulerian. □

The proposition makes the last two steps well defined; H spans V because every vertex lies on the first cycle cover.

Lemma 16.8. *In every iteration of the main loop, the current graph G' has a directed TSP tour of cost at most OPT, the minimum cost of a TSP tour of the original graph.*

Proof. Shortcut an optimal tour of the original graph to the vertices of G'; by the triangle inequality the cost does not increase. □

Corollary 16.9. *In every iteration of the main loop, the edges added to F cost at most OPT.*

This follows from Lemma 16.6 applied to G' and Lemma 16.8.

Lemma 16.10. *The main loop runs at most $\log_2 n$ times.*

Proof. Each cycle of a cycle cover has at least two vertices, and all but one of them are deleted, so at least one vertex is deleted for each vertex that remains. The number of remaining vertices therefore at least halves in each iteration, and the loop needs at least 2 vertices to run, so it runs at most $\log_2 n$ times. □

Theorem 16.11. *The algorithm above is a $\log_2 n$-approximation algorithm for ATSP with the triangle inequality.*

Proof. By the triangle inequality, the tour costs at most $\sum_{e \in F} c_e$, which by Corollary 16.9 and Lemma 16.10 is at most $\log_2 n \cdot OPT$. □

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 16: The Traveling Salesman Problem.

Lecture 17: Linear Programming and Approximation Algorithms

Approximation guarantees need a bound on the optimal objective value that is "only better than optimal": an algorithm within a factor α of such a bound is within α of optimal. Linear programs and their duals generate these bounds systematically. Some approximation algorithms solve an LP explicitly, some use LP duality only to guide their design, and some use duality only in the analysis of a natural algorithm.

17.1 Greedy set cover without costs

Definition 17.1 [Set cover]. The input is a ground set U with $|U| = n$ and sets $S_1, \ldots, S_m \subseteq U$. A set cover is a collection of the S_i whose union is U. The goal is a set cover with as few sets as possible.

This reverses set coverage (Lecture 15): there, at most k sets could be chosen and the number of covered elements was maximized; here covering everything is the hard constraint. The problem is NP-hard. The greedy algorithm for set coverage makes sense with a new stopping rule.

Algorithm 17.1: Greedy set cover (no costs)

```
C ← ∅
while C is not a set cover do
    Add to C the set S_i covering the most elements not covered by C
return C
```

The bad examples of Lecture 15 show that greedy is not optimal in general.

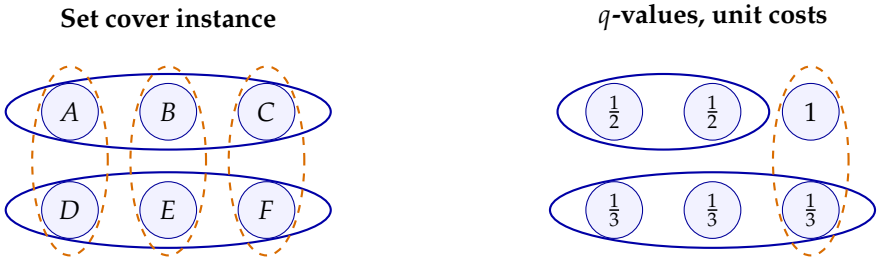


Figure 44: Left: a set cover instance on six elements with two horizontal sets of size three (solid) and three vertical sets of size two (dashed); the two horizontal sets form an optimal cover. Right: an instance with unit costs, labeled by the q-values of the greedy algorithm, which picks the bottom set (ratio $\frac{1}{3}$), then the top-left set (ratio $\frac{1}{2}$), then the vertical set (ratio 1).

Theorem 17.2. *Let OPT be the minimum size of a set cover. The greedy algorithm uses at most* $OPT \cdot \ln n + 1$ *sets; it is a* $\approx \ln n$*-approximation algorithm.*

Proof. The first OPT iterations of the set cover greedy algorithm are exactly the set coverage greedy algorithm with budget $k = OPT$. The key step of its analysis applies at every iteration: the elements still uncovered are covered by the OPT sets of an optimal cover, so some set covers at least a $1/OPT$ fraction of them, and greedy covers at least that many. After j iterations at most $n(1-1/OPT)^j \le ne^{-j/OPT}$ elements remain uncovered; in particular, every OPT iterations shrink the uncovered set by a factor of e, matching the $(1-\frac{1}{e})$ coverage guarantee. Once $j > OPT \ln n$ fewer than one element remains, so greedy stops after at most $OPT \ln n + 1$ iterations. □

17.2 Greedy set cover with costs

Now each set S_i also has a cost $c_i \ge 0$, and the goal is a set cover of minimum total cost. A set costing twice as much should cover twice as many new elements to compete, which suggests minimizing the average cost per newly covered element.

Algorithm 17.2: Greedy set cover (with costs)

```
C ← ∅
while C is not a set cover do
    Add to C the set S_i minimizing the ratio r_i = c_i / (# elements of S_i not covered by C)
return C
```

Only sets that cover at least one new element are considered. With equal costs this is the previous algorithm.

Lemma 17.3. *If the current greedy solution covers $\ell < |S_i|$ elements of S_i, the next set chosen has ratio at most $c_i/(|S_i| - \ell)$.*

Proof. Choosing S_i itself would attain this ratio, and greedy picks the minimum ratio. □

For each $e \in U$ let q_e be the ratio of the first set chosen by the greedy algorithm that covers e (and $q_e = 0$ while e is uncovered). Figure 44 (right) shows an example.

Corollary 17.4. *For every set S_i, the jth element e of S_i to be covered by the greedy algorithm satisfies $q_e \le c_i/(|S_i| - (j-1))$.*

Proof. When e is covered, at most $j-1$ elements of S_i were covered before the current step; elements covered in the same step as e only reduce this number. Apply Lemma 17.3 with $\ell \le j-1$. □

In Figure 44 (right) the corollary only guarantees q-values of at most $\frac{1}{3}, \frac{1}{2}, 1$ in the bottom set, while all three are $\frac{1}{3}$; in the vertical set it guarantees at most $\frac{1}{2}$ and 1, while the values are $\frac{1}{3}$ and 1.

Summing the corollary over S_i, with $H_s = \sum_{j=1}^{s} 1/j$,

$$\sum_{e \in S_i} q_e \le \frac{c_i}{|S_i|} + \frac{c_i}{|S_i| - 1} + \cdots + \frac{c_i}{1} = c_i H_{|S_i|} \le c_i H_n, \qquad H_n \le 1 + \ln n \approx \ln n. \tag{17.1}$$

Moreover, at all times,

$$\sum_{e \in U} q_e = \text{cost of the greedy solution so far.} \tag{17.2}$$

Both sides start at 0. When greedy adds S_i, the right side grows by c_i, and each newly covered element receives $q_e = r_i$, so the left side grows by $r_i \cdot (\#\text{ newly covered elements}) = c_i$.

Theorem 17.5. *The greedy algorithm is an $H_n \approx \ln n$-approximation algorithm for set cover with costs.*

Proof. Let $S_1^*, \ldots, S_k^*$ be an optimal set cover of cost OPT. Then

$$\text{greedy cost} = \sum_{e \in U} q_e \le \sum_{i=1}^{k} \sum_{e \in S_i^*} q_e \le \sum_{i=1}^{k} c(S_i^*)\, H_n = H_n \cdot OPT,$$

using (17.2), then that the S_i^* cover every element at least once and $q \ge 0$, then (17.1). □

The analysis is tight. Let $U = \{1, \ldots, n\}$, $S_0 = U$ with $c_0 = 1 + \epsilon$, and $S_i = \{i\}$ with $c_i = 1/i$ for $i = 1, \ldots, n$. The optimum is S_0 with cost $1 + \epsilon$. When j elements remain uncovered, S_0 has ratio $(1 + \epsilon)/j$ while S_j has ratio $1/j$, so greedy picks $S_n, S_{n-1}, \ldots, S_1$ for a total of $H_n \approx \ln n$. More generally, no polynomial-time algorithm achieves an approximation factor significantly better than $\ln n$ (under standard complexity assumptions), so greedy is optimal in this sense.

17.3 Interpretation via LP duality

The analysis above becomes systematic through LP duality, following the same template as the WaterLevel analysis for online bipartite matching. The LP RELAXATION (P) of set cover and its dual (D) are

$$\text{(P)} \quad \begin{aligned} \min \ & \sum_{i=1}^{m} c_i x_i \\ \text{s.t.} \ & \sum_{i:\, e \in S_i} x_i \ge 1 \quad \text{for all } e \in U, \\ & x_i \ge 0 \quad \text{for all } S_i; \end{aligned} \qquad \text{(D)} \quad \begin{aligned} \max \ & \sum_{e \in U} p_e \\ \text{s.t.} \ & \sum_{e \in S_i} p_e \le c_i \quad \text{for every set } S_i, \\ & p_e \ge 0 \quad \text{for all } e \in U. \end{aligned}$$

The intended meaning is $x_i = 1$ if S_i is chosen and 0 otherwise. Set covers correspond exactly, with the same cost, to the 0-1 feasible solutions of (P). (Constraints $x_i \le 1$ are unnecessary; they hold at

an optimal solution anyway.) Since (P) minimizes over a superset of the set covers,

$$\text{fractional } OPT \le OPT.$$

Unlike the relaxations for minimum s-t cut and bipartite matching, this one need not have an integral optimal solution, as expected since set cover is NP-hard and LPs are solvable in polynomial time. For example, let U have three elements and let the sets be the three 2-element subsets, each of cost 1. Every set cover has cost 2, but $x_i = \frac{1}{2}$ for every set is feasible with value $\frac{3}{2}$.

Lemma 17.6. *If p is feasible for (D), then $\sum_{e \in U} p_e \le$ fractional $OPT \le OPT$.*

Proof. The first inequality is weak duality for a minimization primal; the second holds because (P) is a relaxation. □

The purpose of (17.1) is now transparent: it says exactly that $p = q/H_n$ satisfies every dual constraint, so p is feasible for (D). By (17.2) and Lemma 17.6,

$$\text{greedy cost} = H_n \sum_{e \in U} p_e \le H_n \cdot OPT.$$

As in online bipartite matching, the analysis defines a vector q whose sum equals the algorithm's objective value and scales it into a feasible dual solution, which bounds the optimum.

17.4 LP rounding for vertex cover

Definition 17.7 [Vertex cover]. Given an undirected graph $G = (V, E)$ with costs $c_v \ge 0$, find a minimum-cost $S \subseteq V$ containing at least one endpoint of every edge.

Vertex cover is NP-hard in general graphs (in bipartite graphs it reduces to minimum cut). It is the special case of set cover in which the elements are the edges and each vertex v gives the set of edges incident to it, with cost c_v. The greedy algorithm has no better approximation ratio on this special case, so a different algorithm is needed. Specialized to vertex cover, the relaxation (P) is

$$\min \sum_{v \in V} c_v x_v \quad \text{s.t.} \quad x_v + x_w \ge 1 \ \text{ for all } e = (v, w) \in E, \qquad x_v \ge 0 \ \text{ for all } v \in V,$$

and its 0-1 solutions are exactly the vertex covers. On a triangle with unit costs, the example above becomes: the minimum vertex cover has size 2, but $x_v = \frac{1}{2}$ for all v has value $\frac{3}{2}$.

Algorithm 17.3: LP rounding for vertex cover

```
1 Compute an optimal solution x* of the LP relaxation (P)
2 return S = {v ∈ V : x*_v ≥ 1/2}
```

Turning a fractional solution into an integral one with little loss in objective is called ROUNDING. Rounding to the nearest integer is the simplest approach, and for vertex cover it already works well.

Lemma 17.8. *The rounding algorithm outputs a vertex cover.*

Proof. For every $(v, w) \in E$, feasibility gives $x_v^* + x_w^* \geq 1$, so at least one of x_v^*, x_w^* is at least $\frac{1}{2}$ and that endpoint is in S. □

Theorem 17.9. *The LP rounding algorithm is a 2-approximation algorithm for vertex cover.*

Proof. Since $v \in S$ only if $2x_v^* \geq 1$,

$$\sum_{v \in S} c_v \leq \sum_{v \in V} c_v (2x_v^*) = 2 \cdot \text{fractional } OPT \leq 2 \cdot OPT.$$ □

Assuming the Unique Games Conjecture, no polynomial-time algorithm achieves a $(2 - \epsilon)$-approximation for any constant $\epsilon > 0$.

17.5 A primal-dual algorithm for vertex cover

A 2-approximation is also possible without solving the LP, using (P) and its dual only to guide the algorithm. The dual (D) specializes to

$$\max \sum_{e \in E} p_e \quad \text{s.t.} \quad \sum_{e \in \delta(v)} p_e \leq c_v \ \text{ for every } v \in V, \qquad p_e \geq 0 \ \text{ for every } e \in E,$$

where $\delta(v)$ is the set of edges incident to v. A dual constraint is TIGHT if it holds with equality.

Algorithm 17.4: Primal-dual vertex cover

```
p_e ← 0 for every e ∈ E; S ← ∅
while S is not a vertex cover do
    Pick an edge e = (v, w) with v, w ∉ S
    Increase p_e until the dual constraint of v or of w becomes tight
    Add the vertex with the tight constraint to S
return S
```

Such an edge exists whenever S is not a vertex cover. A single pass over the edges, raising p_e by the smaller of the two remaining slacks, implements the algorithm in linear time. The algorithm maintains:

(P1) p is feasible for (D): true initially since $c \geq 0$, and no dual constraint is ever violated;

(P2) if $v \in S$ then $\sum_{e \in \delta(v)} p_e = c_v$: a vertex enters S only when its constraint is tight, and its edges are never raised again;

(P3) if $p_e > 0$ for $e = (v, w)$, then $|S \cap \{v, w\}| \leq 2$, trivially;

and at termination (P4) S is a vertex cover. So the algorithm keeps dual feasibility and works toward primal feasibility. (P2) is the first set of complementary slackness conditions (a primal variable is positive only if its dual constraint is tight). The second set would require exactly one endpoint of every edge with $p_e > 0$ in S; (P3) is this condition relaxed by a factor of 2. This makes it a PRIMAL-DUAL ALGORITHM like the Hungarian algorithm, except that it maintains only approximate complementary slackness and so ends with an approximately optimal solution.

Theorem 17.10. *The primal-dual algorithm is a* 2*-approximation algorithm for vertex cover.*

Proof. Repeat the derivation of weak duality:

$$\sum_{v \in S} c_v = \sum_{v \in S} \sum_{e \in \delta(v)} p_e = \sum_{e=(v,w) \in E} p_e \, |S \cap \{v, w\}| \leq 2 \sum_{e \in E} p_e \leq 2 \cdot \text{fractional } OPT \leq 2 \cdot OPT.$$

The steps use (P2), a reversal of the order of summation, (P3), dual feasibility (P1) with weak duality, and the fact that (P) is a relaxation. □

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 17: Linear Programming and Approximation Algorithms.

Lecture 18: Five Essential Tools for the Analysis of Randomized Algorithms

The setup is a finite state space Ω (for example, the outcomes of an algorithm's coin flips) with a probability distribution. A random variable is a function $X : \Omega \to \mathbb{R}$, with expectation $\mathbb{E}[X] = \sum_{\omega \in \Omega} \mathbb{P}[\omega]\, X(\omega)$. An event is a subset of Ω, and its indicator random variable is 1 on the event and 0 off it. Events E_1, E_2 are independent if $\mathbb{P}[E_1 \wedge E_2] = \mathbb{P}[E_1]\mathbb{P}[E_2]$; random variables X_1, X_2 are independent if the events $\{X_1 = x_1\}$ and $\{X_2 = x_2\}$ are independent for all x_1, x_2, and then $\mathbb{E}[X_1 X_2] = \mathbb{E}[X_1]\mathbb{E}[X_2]$. For three or more events or variables, the probabilities must factor for every subset. Without independence nothing factors in general: complementary events E_1, E_2 have $\mathbb{P}[E_1 \wedge E_2] = 0$.

18.1 Linearity of expectation and MAX 3SAT

Theorem 18.1 [Tool 1: linearity of expectation]**.** *For random variables $X_1, \ldots, X_n$ on a common state space, not necessarily independent, $\mathbb{E}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \mathbb{E}[X_i]$.*

Proof. Expand both sides as sums over Ω and reverse the order of summation. □

The analogous statement for products fails without independence (take indicators of complementary events). The typical use is to write a complicated random variable as a sum of simple ones, often indicators, analyze those directly, and add.

Definition 18.2 [MAX 3SAT]**.** There are n Boolean variables $x_1, \ldots, x_n$ and m clauses, each the disjunction of three literals (a literal is a variable or its negation), such as $x_3 \vee \neg x_6 \vee \neg x_{10}$. Assume the three literals of each clause involve distinct variables. The goal is a truth assignment satisfying as many clauses as possible.

MAX 3SAT is NP-hard, since 3SAT asks whether the optimum equals m.

Theorem 18.3. *A truth assignment chosen uniformly at random from all 2^n satisfies $\frac{7}{8}m$ clauses in expectation. Hence choosing a random assignment is a $\frac{7}{8}$-approximation algorithm in expectation.*

Proof. Let X_j indicate that clause j is satisfied, so the number of satisfied clauses is $X = \sum_{j=1}^m X_j$ and $\mathbb{E}[X_j] = \mathbb{P}[\text{clause } j \text{ satisfied}]$. A clause on three distinct variables is violated by exactly one of the 8 equally likely assignments to those variables (for $x_1 \vee x_2 \vee x_3$, all three false), so $\mathbb{E}[X_j] = \frac{7}{8}$ and $\mathbb{E}[X] = \sum_j \mathbb{E}[X_j] = \frac{7}{8}m$. No assignment satisfies more than m clauses. □

Corollary 18.4. *For every 3SAT formula, some truth assignment satisfies at least* 87.5% *of the clauses.*

Proof. Some assignment does at least as well as the average. □

This is best possible: assuming P ≠ NP, there is no $(\frac{7}{8} + \epsilon)$-approximation algorithm for MAX 3SAT for any constant $\epsilon > 0$.

18.2 Tail inequalities, hashing, and Markov's inequality

A TAIL INEQUALITY shows that a random variable concentrates near its expectation, as needed to prove that an algorithm is good "with high probability." The more one assumes about the random variable, the sharper the concentration. Hashing serves as the running example.

A family $\mathcal{H}$ of hash functions maps a large universe U to buckets $\{1, \ldots, n\}$. An adversary picks a data set $S \subseteq U$, then h is drawn uniformly from $\mathcal{H}$ and all elements of S are hashed. The quality measure is the maximum load of a bucket (the number of elements hashing to it), which governs, for example, worst-case search time with chaining. Throughout, $|S| = n$ unless stated otherwise.

Assume first only

(P1) for every $x \in U$ and bucket i, $\mathbb{P}_{h \in \mathcal{H}}[h(x) = i] = \frac{1}{n}$.

By linearity of expectation over the indicators of $h(x) = i$, every bucket i has expected load $\sum_{x \in S} \mathbb{P}[h(x) = i] = |S|/n = 1$.

Theorem 18.5 [TOOL 2: MARKOV'S INEQUALITY**].** *If X is a nonnegative random variable with finite expectation, then for every $c \geq 1$, $\mathbb{P}[X \geq c \cdot \mathbb{E}[X]] \leq \frac{1}{c}$.*

Proof. Pointwise $X \geq c\,\mathbb{E}[X] \cdot \mathbf{1}\{X \geq c\,\mathbb{E}[X]\}$, since $X \geq 0$; take expectations. □

Both hypotheses are needed: a variable equally likely to be M or $-M$ shows no concentration as $M \to \infty$. Markov's inequality suffices when a constant-probability guarantee is enough. With $c = n$ it gives $\mathbb{P}[\text{load of } i \geq n] \leq \frac{1}{n}$, although load n means that *every* element hashes to i. Under (P1) alone this cannot be improved: the family of the n constant functions satisfies (P1), and all elements land in bucket i with probability exactly $\frac{1}{n}$.

18.3 Chebyshev's inequality

Now assume the family is PAIRWISE (or 2-WISE) INDEPENDENT:

(P2) for all distinct $x, y \in U$ and all buckets i, j, $\mathbb{P}_{h \in \mathcal{H}}[h(x) = i \text{ and } h(y) = j] = \frac{1}{n^2}$.

Looking at any two elements, h behaves like a totally random function. This is essentially universal hashing; constant functions fail (P2). Recall $\mathrm{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2]$ and $\mathrm{StdDev}[X] = \sqrt{\mathrm{Var}[X]}$.

Theorem 18.6 [Tool 3: Chebyshev's inequality**].** *If X has finite expectation and variance, then for every $t \geq 1$, $\mathbb{P}\big[|X - \mathbb{E}[X]| > t \cdot \mathrm{StdDev}[X]\big] \leq \frac{1}{t^2}$.*

Chebyshev's inequality follows from Markov's inequality applied to $(X - \mathbb{E}[X])^2$. Let $X = \sum_{y \in S} X_y$ be the load of bucket i, where X_y indicates $h(y) = i$; again $\mathbb{E}[X] = 1$. The variance of a sum is the sum of the variances plus covariance terms for pairs of summands. Pairwise independent variables have zero covariance, so

$$\mathrm{Var}[X] = \sum_{y \in S} \mathrm{Var}[X_y] = n \cdot \frac{1}{n}\left(1 - \frac{1}{n}\right) \leq 1,$$

since each X_y is Bernoulli with parameter $\frac{1}{n}$. (Variances do not add in general: indicators of complementary events sum to the constant 1. For constant hash functions, $\mathrm{Var}[X] \approx n$.) Chebyshev's inequality with $t \approx n$, ignoring lower-order terms, gives $\mathbb{P}[X \geq n] \lesssim \frac{1}{n^2}$. Under (P2) alone this is again best possible: some pairwise independent families send all elements to one bucket with probability $\frac{1}{n^2}$.

18.4 Chernoff bounds

Now assume

(P3) all $h(x)$ are independent and uniform in $\{1, \ldots, n\}$; that is, h is a completely random function.

Theorem 18.7 [Tool 4: Chernoff bounds**].** *Let $X_1, \ldots, X_n$ be independent random variables taking values in $[0, 1]$, and let $X = \sum_{j=1}^{n} X_j$.*

(i) For every $\delta > 0$, $\mathbb{P}[X > (1 + \delta)\mathbb{E}[X]] < \left(\dfrac{e}{1 + \delta}\right)^{(1+\delta)\mathbb{E}[X]}$.

(ii) For every $\delta \in (0, 1)$, $\mathbb{P}[X < (1 - \delta)\mathbb{E}[X]] < e^{-\delta^2 \mathbb{E}[X]/2}$.

The deviation probability decays exponentially in both the deviation factor $1 + \delta$ and the expectation $\mathbb{E}[X]$, so it is tiny once either is modestly large. The proof is not needed here; Chernoff bounds are used as a black box, and they are the only one of the five tools with a nontrivial proof. (A sharper form of (i) is $[e^{\delta}/(1 + \delta)^{1+\delta}]^{\mathbb{E}[X]}$, useful when δ is small. Scaling extends the bounds to nonnegative variables bounded by any constant, and the independence assumption can be relaxed, for example to negative correlation.)

Under (P3), the load $X = \sum_{y \in S} X_y$ of bucket i is a sum of independent 0-1 variables with $\mathbb{E}[X] = 1$.

Taking $1 + \delta = \ln n$,

$$\mathbb{P}[X > \ln n] < \left(\frac{e}{\ln n}\right)^{\ln n}. \tag{18.1}$$

Since $(1/e)^{\ln n} = 1/n$, a constant less than 1 raised to a logarithmic power is an inverse polynomial; here the base $e/\ln n$ tends to 0, so the bound is smaller than every inverse polynomial. Compared with Markov and Chebyshev, the deviation is much smaller ($\ln n$ instead of n) and the probability is much smaller too. Taking $1 + \delta = 3\ln n/\ln\ln n$, a short calculation gives

$$\mathbb{P}\left[X > \frac{3\ln n}{\ln\ln n}\right] \le \frac{1}{n^2}. \tag{18.2}$$

The threshold $\ln n/\ln\ln n$ is roughly the solution of $x^x = n$, which matters because of the factor $(1+\delta)^{-(1+\delta)}$ in the bound. Likewise $\mathbb{P}[X \ge n]$ is inverse exponential in n.

18.5 The union bound

Theorem 18.8 [Tool 5: union bound]. *For arbitrary events $E_1, \ldots, E_k$, $\mathbb{P}[$at least one E_i occurs$] \le \sum_{i=1}^{k} \mathbb{P}[E_i]$.*

The events need not be independent. The probability mass of the union is at most the sum of the masses of the events, with equality for disjoint events; it is inclusion-exclusion without the correction terms. Typically the E_i are bad events: if each is unlikely and there are not too many, then with high probability none occurs.

In the hashing example, let E_i be the event that bucket i has load more than $3\ln n/\ln\ln n$. By (18.2) and the union bound over the n buckets, with probability at least $1 - \frac{1}{n}$ no bucket has load more than $3\ln n/\ln\ln n$: the maximum load is $O(\log n/\log\log n)$ with high probability. This is tight up to a constant factor.

The large-expectation regime. The previous application had small expectation and large deviation. Instead hash $|S| = n\ln n$ elements, so every bucket has expected load $\ln n$. Part (i) with $1 + \delta = 4$ gives, for each bucket i,

$$\mathbb{P}[\text{load of } i > 4\ln n] < \left(\frac{e}{4}\right)^{4\ln n} = n^{-4(\ln 4 - 1)} < n^{-3/2},$$

and the union bound shows that with probability at least $1 - n^{-1/2}$ no bucket has load more than four times its expectation. So with light loads there are nontrivial (though only logarithmic) deviations from the expectation, but once loads are modestly large the buckets are evenly balanced with high probability.

18.6 Randomized rounding for edge-disjoint paths

Definition 18.9 [Edge-disjoint paths]. Given a graph $G = (V, E)$, directed or undirected, with $m = |E|$, and pairs $(s_1, t_1), \ldots, (s_k, t_k)$, decide whether there are s_i-t_i paths P_i such that no edge lies on more than one of them.

Paths may share vertices (Figure 45). The problem is NP-hard, for directed graphs even with $k = 2$.

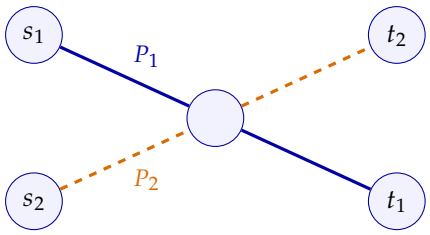


Figure 45: Edge-disjoint paths for two pairs. Both paths pass through the middle vertex but share no edge.

LP rounding solves an LP relaxation, whose optimal solution is expected to be fractional for an NP-hard problem, and then rounds it to a feasible integral solution without degrading the objective much. In randomized rounding the fractional values define a probability distribution and variables are rounded randomly according to it.

The natural relaxation is a multicommodity flow problem: send one unit of flow from each s_i to t_i simultaneously, with every edge of capacity 1. Its 0-1 solutions are edge-disjoint paths, and it is solvable in polynomial time. If it is infeasible, so is the original problem.

Algorithm 18.1: Randomized rounding for edge-disjoint paths

```
Solve the multicommodity flow relaxation; stop if it is infeasible
for i ← 1 to k, independently do
    Decompose the unit s_i-t_i flow into paths carrying amounts that sum to 1
    Choose P_i at random, each path with probability equal to its flow amount
return P_1, ..., P_k
```

The paths are not disjoint in general (that would solve an NP-hard problem), but they are approximately disjoint. This is the original and canonical application of randomized rounding.

Theorem 18.10. *If the LP relaxation is feasible, then with probability at least $1 - \frac{1}{m}$, no edge is used by more than $3 \ln m / \ln \ln m$ of the paths $P_1, \ldots, P_k$.*

Proof. Fix an edge e and let $X = \sum_{i=1}^{k} X_i$, where X_i indicates that P_i uses e. The X_i are independent since the pairs are rounded independently. By linearity of expectation, $\mathbb{E}[X]$ is the total flow on e in the relaxation, which is at most 1 by the capacity constraint. Adding one more independent indicator with probability $1 - \mathbb{E}[X]$ only increases X and makes its expectation

exactly 1. As in the hashing analysis, with edges as buckets and pairs as elements, (18.2) gives $\mathbb{P}[X > 3 \ln m / \ln \ln m] \leq \frac{1}{m^2}$. A union bound over the m edges finishes the proof. □

This one-two punch of a Chernoff bound and a union bound appears in countless analyses. For directed graphs the guarantee is optimal up to a constant factor (assuming P ≠ NP); for undirected graphs the best known lower bound is $\Omega(\log \log n)$.

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 18: Five Essential Tools for the Analysis of Randomized Algorithms.

Lecture 19: Beating Brute-Force Search

For many NP-hard problems there are algorithms whose worst-case running time, while still exponential, is much better than brute-force search. Three examples follow: vertex cover, the traveling salesman problem, and 3SAT.

19.1 Vertex cover and fixed-parameter tractability

Consider vertex cover with unit costs: given an undirected graph $G = (V, E)$ with n vertices and m edges, and a budget k, decide whether G has a vertex cover of size at most k. Trying every k solves the minimization problem, so this is no easier. Think of k as small (say 10 to 20) and G as large.

Brute force checks all $\binom{n}{k} = \Theta(n^k)$ subsets of size k, which is polynomial for constant k but hopeless unless k is tiny. Better would be a running time $\text{poly}(n) \cdot f(k)$, separating the dependence on n and k, or even $\text{poly}(n) + f(k)$, with $\text{poly}(n)$ close to linear. Since the problem is NP-hard for general k, f should be expected to be exponential. An algorithm with such a running time is fixed-parameter tractable (FPT) with respect to k.

Algorithm 19.1: FPT algorithm for vertex cover

Input: Graph $G = (V, E)$ and budget k.

$H \leftarrow \{v \in V : \deg(v) \geq k + 1\}$
$G' \leftarrow G \setminus H$ (delete H and all edges incident to it)
$G'' \leftarrow G'$ with all isolated vertices removed
if G'' has more than k^2 edges **then**
    return "no vertex cover of size $\leq k$"
else
    Compute a minimum vertex cover T of G'' by brute-force search
    return "yes" if and only if $|H| + |T| \leq k$

Theorem 19.1. *The algorithm correctly decides whether G has a vertex cover of size at most k, in time $O(m) + 2^{O(k^2)}$.*

Proof. Let C be a vertex cover with $|C| \leq k$. Every v of degree at least $k + 1$ lies in C: otherwise the other endpoints of its at least $k + 1$ edges all lie in C, so $|C| \geq k + 1$. Hence $H \subseteq C$. The edges of G' are exactly those not covered by H, so the vertex covers of G of size at most k are exactly the sets $H \cup T$ with T a vertex cover of G' of size at most $k - |H|$. Isolated vertices of G' are irrelevant, so G has a vertex cover of size at most k if and only if G'' has one of size at most $k - |H|$.

Every vertex of G'' has degree at most k, since vertices of higher degree went into H. So each vertex of a cover of G'' covers at most k edges, and a cover of size at most k exists only if G'' has

at most k^2 edges. This justifies the "no" answer, and the brute-force branch is correct by the previous paragraph.
The first four steps take linear time. If brute force runs, G'' has at most k^2 edges and, since it has no isolated vertices, at most $2k^2$ vertices, so trying all subsets takes $2^{O(k^2)}$ time. □

The graph G'' is a KERNEL of size $O(k^2)$: the problem on an arbitrarily large graph reduces to the same problem on a graph whose size depends only on k. Linear programming techniques give a kernel of size $O(k)$ for unweighted vertex cover, improving the dependence on k to $2^{O(k)}$, which is essentially the best case for FPT algorithms. Not every problem is FPT; the theory of $W[1]$-HARDNESS plays the role of NP-hardness here. For example, the independent set problem, parameterized by the size of the independent set, is $W[1]$-hard, even though the complement of a vertex cover is an independent set.

19.2 TSP and dynamic programming

The traveling salesman problem (TSP) takes a complete undirected graph on $V = \{1, \dots, n\}$ with nonnegative edge costs c_{ij}, not necessarily satisfying the triangle inequality, and asks for a minimum-cost tour, a simple cycle visiting every vertex exactly once. Trying every tour takes roughly $n!$ time, which grows much faster than c^n for any constant c and is feasible only up to $n \approx 12$ or 13. Dynamic programming takes $O(n^2 2^n)$ time, but also uses exponential space. Whether an $O(c^n)$-time algorithm with polynomial space exists is open.

The subproblems are indexed by a subset $S \subseteq V$ and a vertex j. Let $A[S, j]$ be the minimum cost of a path that

(i) visits every vertex of S exactly once and no other vertex,

(ii) starts at vertex 1 (so $1 \in S$), and

(iii) ends at vertex j (so $j \in S$),

with $A[S, j] = +\infty$ if there is no such path. There are $O(n 2^n)$ subproblems. Since $A[V, j]$ is the cheapest path from 1 to j through all vertices, guessing the vertex before 1 on the tour gives

$$OPT = \min_{j=2,\dots,n} \Big(\underbrace{A[V, j]}_{\text{path from 1 to } j} + \underbrace{c_{j1}}_{\text{last hop}} \Big).$$

For the recurrence, suppose $|S| \geq 3$ and let k be the penultimate vertex of an optimal path for $A[S, j]$. Removing the last hop leaves a path from 1 to k visiting each vertex of $S \setminus \{j\}$ exactly once, and it must be a cheapest such path, or swapping in a cheaper one would improve the path for

$A[S, j]$. So brute-force search over k gives

$$A[S, j] = \min_{k \in S \setminus \{1, j\}} \big(A[S \setminus \{j\}, k] + c_{kj} \big) \qquad (|S| \geq 3).$$

For $|S| = 1$, $A[S, j] = 0$ if $S = \{1\}$ and $j = 1$, and $+\infty$ otherwise. For $|S| = 2$, the only choice is $k = 1$: $A[\{1, j\}, j] = c_{1j}$.

Algorithm 19.2: Dynamic programming for TSP

Input: Costs c_{ij} on the complete graph with vertices $V = \{1, \dots, n\}$.
Output: The minimum cost of a tour.

```
A[{1}, 1] ← 0
for s ← 2 to n do
    for each S ⊆ V with |S| = s and 1 ∈ S do
        A[S, 1] ← +∞
        for each j ∈ S \ {1} do
            if s = 2 then
                A[S, j] ← c_{1j}
            else
                A[S, j] ← min_{k ∈ S\{1,j}} (A[S \ {j}, k] + c_{kj})
return min_{j=2,...,n} (A[V, j] + c_{j1})
```

Subproblems are solved in increasing order of $|S|$, so all needed smaller values are available by constant-time lookup. Each subproblem takes $O(n)$ time and there are $O(n2^n)$ of them, for a total of $O(n^2 2^n)$. This extends the feasible range of n into the 20s.

19.3 Schöning's random search for 3SAT

The 3SAT decision problem asks whether a formula with n variables and m clauses, each a disjunction of three literals, has an assignment satisfying every clause. Brute force tries all 2^n assignments. Schöning's algorithm (1999) is a simple random search; the number T of restarts is fixed by the analysis.

Algorithm 19.3: Random search for 3SAT (Schöning)

Input: A 3SAT formula on variables $x_1, \dots, x_n$; parameter T.

```
for t ← 1 to T do
    Choose a truth assignment a uniformly at random
    for s ← 1 to n do
        if a satisfies every clause then return "satisfiable"
        Choose a clause C violated by a
        Choose one of the three literals of C uniformly at random and flip its variable in a
    if a satisfies every clause then return "satisfiable"
return "unsatisfiable"
```

The algorithm never errs on an unsatisfiable formula, so consider a satisfiable one and fix a satisfying assignment $\mathbf{a}^*$. Track the Hamming distance $\mathrm{dist}(\mathbf{a}, \mathbf{a}^*)$, the number of variables on which they

differ; at distance 0 the algorithm has found $\mathbf{a}^*$. If $\mathbf{a}$ violates C, then since $\mathbf{a}^*$ satisfies C, the two differ on at least one of the three variables of C. So with probability at least $\frac{1}{3}$ the algorithm flips a GOOD VARIABLE and the distance drops by 1; otherwise it flips a BAD VARIABLE, where they agree, and the distance grows by 1. This holds at every step given the history, whatever the earlier choices. The walk is biased away from $\mathbf{a}^*$, which is the source of the exponential running time. Each analysis finds a sufficient condition for success, bounds its probability p per outer iteration from below, and uses

$$\mathbb{P}[\text{all } T \text{ outer iterations fail}] \le (1-p)^T \le e^{-pT},$$

so $T = d \ln n/p$ restarts give failure probability at most n^{-d}.

Version 1. If the initial distance is k, flipping a good variable in each of the first k inner iterations suffices (stopping early at another satisfying assignment is also fine); this happens with probability at least 3^{-k}. By symmetry ($\mathbf{a}$ and its complement are equally likely), $k \le n/2$ with probability at least $\frac{1}{2}$. So $p \ge \frac{1}{2} \cdot 3^{-n/2}$, and $T = \Theta((\sqrt{3})^n \log n)$ restarts give high probability of success, for a running time of $O(1.74^n)$, well below 2^n.

Version 2. Count every initial distance instead of the worst case. The assignments at distance exactly k from $\mathbf{a}^*$ correspond to the $\binom{n}{k}$ sets of variables on which they differ, so $\mathbb{P}[\mathrm{dist}(\mathbf{a}, \mathbf{a}^*) = k] = \binom{n}{k} 2^{-n}$. Conditioning on k and using the binomial formula,

$$\mathbb{P}[\text{success}] \ge \sum_{k=0}^{n} \binom{n}{k} 2^{-n} \left(\frac{1}{3}\right)^k = 2^{-n} \left(1 + \frac{1}{3}\right)^n = \left(\frac{2}{3}\right)^n,$$

so $T = \Theta((\frac{3}{2})^n \log n)$ restarts suffice.

Version 3. Run the inner loop $3n$ times instead of n, a constant-factor change. The algorithm can also succeed after mistakes: one bad flip followed by $k+1$ good flips also reaches $\mathbf{a}^*$. From initial distance k, it suffices that at least $2k$ of the first $3k$ flips are good: without an early stop, the distance would then be at most $k - 2k + k = 0$, so it reaches 0. The number of good flips stochastically dominates a binomial with parameters $3k$ and $\frac{1}{3}$, so

$$\mathbb{P}[\text{success}] \ge \sum_{k=0}^{n} 2^{-n} \binom{n}{k} \binom{3k}{k} \left(\frac{1}{3}\right)^{2k} \left(\frac{2}{3}\right)^k. \tag{19.1}$$

The $\binom{3k}{k}$ factors are handled with Stirling's approximation $n! = \Theta(\sqrt{n}\,(n/e)^n)$ (the hidden constant is $\sqrt{2\pi}$): for $k \ge 1$,

$$\binom{3k}{k} = \frac{(3k)!}{(2k)!\,k!} = \Theta\left(\frac{\sqrt{3k}}{\sqrt{k}\sqrt{2k}} \cdot \frac{(3k/e)^{3k}}{(k/e)^k (2k/e)^{2k}}\right) = \Theta\left(\frac{1}{\sqrt{k}} \cdot \frac{3^{3k}}{2^{2k}}\right), \quad \text{so} \quad \binom{3k}{k}\left(\frac{1}{3}\right)^{2k}\left(\frac{2}{3}\right)^k = \Theta\left(\frac{2^{-k}}{\sqrt{k}}\right).$$

Substituting into (19.1), for some constant $c > 0$ (the $k = 0$ term is 2^{-n} and fits the same bound),

$$\mathbb{P}[\text{success}] \geq c\,2^{-n} \sum_{k=0}^{n} \binom{n}{k} \frac{2^{-k}}{\sqrt{\max(k,1)}} \geq \frac{c}{\sqrt{n}} 2^{-n} \sum_{k=0}^{n} \binom{n}{k} 2^{-k} = \frac{c}{\sqrt{n}} 2^{-n} \left(1 + \frac{1}{2}\right)^n = \frac{c}{\sqrt{n}} \left(\frac{3}{4}\right)^n.$$

Theorem 19.2. *With $T = \Theta((\frac{4}{3})^n \sqrt{n} \log n)$ restarts and inner loops of length $3n$, the random search algorithm decides 3SAT correctly with high probability, in time $(\frac{4}{3})^n \cdot \text{poly}(n)$.*

This has been improved only somewhat since 1999. The EXPONENTIAL TIME HYPOTHESIS (ETH) asserts that every correct algorithm for 3SAT has worst-case running time at least c^n for some constant $c > 1$; it rules out, for example, $n^{\text{polylog}(n)}$ time. It is stronger than P $\neq$ NP, but most experts believe it. Random search extends to k-SAT for each constant k, with running time $O(c_k^n)$ for some $c_k < 2$, but $c_k \to 2$ as $k \to \infty$. The STRONG EXPONENTIAL TIME HYPOTHESIS (SETH) asserts that this is necessary: no algorithm solves general SAT in time $O(c^n)$ for a constant $c < 2$ independent of k. Expert opinion on SETH is mixed; if it holds, it has consequences ranging from fixed-parameter tractability to lower bounds for classic problems such as edit distance.

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 19: Beating Brute-Force Search.

Lecture 20: The Maximum Cut Problem and Semidefinite Programming

Definition 20.1 [Maximum cut]. Given an undirected graph $G = (V, E)$ with weights $w_e \geq 0$, find a cut, a partition of V into sets A and B, maximizing the total weight of the cut edges (those with one endpoint in A and one in B).

Negating the weights does not reduce the problem to minimum cut, because the polynomial-time minimum cut algorithms rely on nonnegative capacities; in fact maximum cut is NP-hard. A $\frac{1}{2}$-approximation is easy (greedy, local search, a random cut, LP rounding, and so on), but none of these techniques gives a factor better than $\frac{1}{2}$. Goemans and Williamson (1994) did much better with semidefinite programming rounding.

20.1 A semidefinite relaxation

Quadratic program. Give each vertex $i \in V$ a variable $y_i \in \{-1, +1\}$ indicating its side of the cut. Then $y_i y_j = +1$ if i, j are on the same side and -1 otherwise, so maximum cut is exactly

$$\max \sum_{(i,j)\in E} w_{ij} \cdot \tfrac{1}{2}(1 - y_i y_j) \quad \text{s.t.} \quad y_i \in \{\pm 1\} \ \text{ for all } i \in V,$$

with a one-to-one, objective-preserving correspondence between cuts and feasible solutions. Its nonlinearity comes from the integrality constraints and the quadratic terms $y_i y_j$.

Vector relaxation. Relax each y_i to a unit vector $x_i \in \mathbb{R}^n$, where $n = |V|$, and replace $y_i y_j$ by the inner product $\langle x_i, x_j \rangle$:

$$\max \ \frac{1}{2} \sum_{(i,j)\in E} w_{ij}\big(1 - \langle x_i, x_j \rangle\big) \quad \text{s.t.} \quad \|x_i\|_2^2 = 1 \ \text{ for every } i \in V.$$

The values ± 1 are the special cases $(\pm 1, 0, \ldots, 0)$, so every cut gives a feasible solution of the same value and

$$\text{vector } OPT \geq OPT.$$

Geometrically, the vertices are mapped to the unit sphere, with the endpoints of each edge pushed toward antipodal points.

Disguised convexity. This relaxation can be solved in polynomial time (up to arbitrarily small error, since the optimum may be irrational). The reason is convexity: a set is convex if it contains the chord between any two of its points, and convex programs are tractable, while the sphere itself

is not convex. To expose the convexity, linearize: introduce a variable p_{ij} for all $i, j \in V$, intended to equal $\langle x_i, x_j \rangle$. A $V \times V$ matrix $P = (p_{ij})$ has the form $p_{ij} = \langle x_i, x_j \rangle$ for some vectors $x_1, \ldots, x_n$ if and only if

$$P = X^\top X \quad \text{for some } X \in \mathbb{R}^{V \times V},$$

since the (i, j) entry of $X^\top X$ is the inner product of columns i and j of X; the columns are the vectors. Such matrices are the (symmetric) POSITIVE SEMIDEFINITE (PSD) matrices. A symmetric matrix is psd if and only if all its (real) eigenvalues are nonnegative, and if and only if

$$z^\top P z \geq 0 \quad \text{for every } z \in \mathbb{R}^n.$$

The forward direction is direct: $z^\top X^\top X z = \|Xz\|_2^2 \geq 0$; the reverse follows from the eigenvalue characterization. For a fixed z, the constraint $z^\top P z = \sum_{i,j \in V} p_{ij} z_i z_j \geq 0$ is *linear* in the decision variables p_{ij}.

The SDP. The vector relaxation is therefore equivalent to the SEMIDEFINITE PROGRAM (SDP)

$$\begin{aligned}
\max \ & \frac{1}{2} \sum_{(i,j) \in E} w_{ij}(1 - p_{ij}) & & \\
\text{s.t.} \ & \sum_{i,j \in V} p_{ij} z_i z_j \geq 0 & \text{for every } z \in \mathbb{R}^n, & \quad (20.1) \\
& p_{ij} = p_{ji} & \text{for every } i, j \in V, & \quad (20.2) \\
& p_{ii} = 1 & \text{for every } i \in V. & \quad (20.3)
\end{aligned}$$

Constraints (20.1) and (20.2) say that P is symmetric psd, so $P = X^\top X$, and (20.3) says that the columns of X are unit vectors.

Solving SDPs. Every constraint is linear, but there are infinitely many of type (20.1). The ellipsoid method runs in time polynomial in the number of variables (n^2 here) given a polynomial-time SEPARATION ORACLE, which either certifies that a proposed solution is feasible or returns a violated constraint. Constraints (20.2) and (20.3) are checked directly. For (20.1), compute the eigenvalues and eigenvectors of P. If some eigenvalue λ is negative, its eigenvector z gives $z^\top P z = \lambda \|z\|_2^2 < 0$, a violated constraint. Interior-point methods also run in polynomial time and work in practice for medium-sized SDPs (much smaller than routinely solved LPs), and extensions of multiplicative weights quickly find approximately optimal solutions. All of these find only near-optimal solutions, which is unavoidable since an SDP with integer coefficients can have an irrational optimum; the running time depends on $\log \frac{1}{\epsilon}$ for accuracy ϵ (inverse polynomially on ϵ for multiplicative weights). A Cholesky decomposition of the optimal P recovers X, hence the vectors x_i.

20.2 Randomized hyperplane rounding

The relaxation was known in the 1980s; the contribution of Goemans and Williamson was the rounding. It is natural to round the vector solution, which has one vector per vertex, rather than the SDP solution, which has one scalar per pair. Two ideas combine: round randomly, and round a vector to a bit by the side of a hyperplane on which it lies.

Algorithm 20.1: Randomized hyperplane rounding

Input: A unit vector $x_i \in \mathbb{R}^n$ for each $i \in V$.
Output: A cut (A, B).

Choose a random unit vector $r \in \mathbb{R}^n$
$A \leftarrow \{i \in V : \langle x_i, r\rangle \geq 0\}$
$B \leftarrow \{i \in V : \langle x_i, r\rangle < 0\}$
return (A, B)

To sample r, draw independent standard Gaussians $g_1, \dots, g_n$ and set

$$r = \frac{(g_1, \dots, g_n)}{\|(g_1, \dots, g_n)\|};$$

the cut does not change without normalizing. The only property needed is spherical symmetry: all vectors at a given distance from the origin are equally likely.

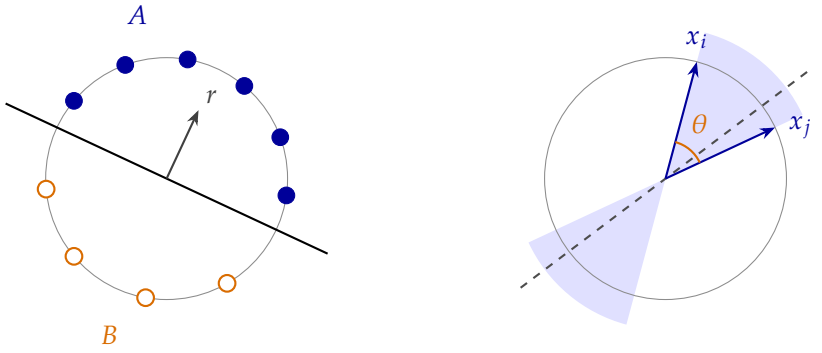


Figure 46: Left: schematic of hyperplane rounding in the plane. The random hyperplane (a line) with normal r puts the vectors with $\langle x_i, r\rangle \geq 0$ in A (filled) and the rest in B (open). Right: in the plane spanned by x_i and x_j, the line separates them exactly when its direction lies in one of the two shaded wedges of angle θ.

Lemma 20.2. *For every edge $(i, j) \in E$,*

$$\mathbb{P}[(i,j) \text{ is cut}] \geq 0.878 \cdot \underbrace{\tfrac{1}{2}\big(1 - \langle x_i, x_j\rangle\big)}_{\text{contribution to the SDP}}.$$

Proof. Project r onto the two-dimensional subspace spanned by x_i and x_j. By spherical symmetry the projection is equally likely to point in any direction, and $\langle x_i, r\rangle$ and $\langle x_j, r\rangle$ depend only on the projection. Let $\theta \in [0, \pi]$ be the angle between x_i and x_j. The edge is cut exactly when the line orthogonal to the projection separates x_i from x_j, which happens for 2θ of the 2π radians of directions (Figure 46, right), so

$$\mathbb{P}[(i,j) \text{ is cut}] = \frac{\theta}{\pi}.$$

Since x_i, x_j are unit vectors, $\langle x_i, x_j\rangle = \cos\theta$, so it remains to check that

$$\frac{\theta}{\pi} \geq 0.878 \cdot \tfrac{1}{2}(1 - \cos\theta) \qquad \text{for all } \theta \in [0, \pi],$$

which is clear from a plot of both sides (Figure 47) and can be verified by first-year calculus. □

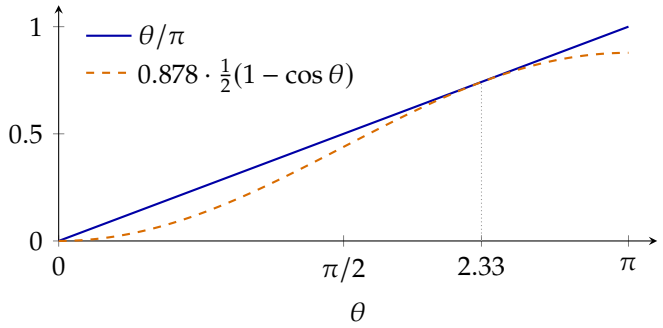


Figure 47: The cutting probability θ/π lies above $0.878 \cdot \frac{1}{2}(1 - \cos\theta)$ on $[0, \pi]$. The ratio of the two sides of the lemma, $2\theta/(\pi(1 - \cos\theta))$, is smallest ($\approx 0.8786$) near $\theta \approx 2.33$, where the curves nearly touch.

Theorem 20.3 [Goemans and Williamson]. *The expected weight of the cut produced by randomized hyperplane rounding of an optimal solution of the vector relaxation is at least* 0.878 *times the maximum weight of a cut.*

Proof. By linearity of expectation over the indicators of cut edges, then Lemma 20.2, then optimality of the x_i for the relaxation,

$$\begin{aligned}\mathbb{E}[\text{weight of } (A,B)] &= \sum_{(i,j)\in E} w_{ij}\mathbb{P}[(i,j) \text{ is cut}] \geq 0.878 \sum_{(i,j)\in E} w_{ij} \cdot \tfrac{1}{2}\big(1 - \langle x_i, x_j\rangle\big) \\ &= 0.878 \cdot \text{vector } OPT \geq 0.878 \cdot OPT.\end{aligned}$$

□

The constant may be optimal: if P ≠ NP and the Unique Games Conjecture holds (its truth is unclear), no polynomial-time algorithm for maximum cut has a better approximation factor.

Source. Tim Roughgarden, A Second Course in Algorithms, Lecture 20: The Maximum Cut Problem and Semidefinite Programming.